Course Material

High Performance Computing

Dr. Chris Cantwell

2023/2024

v4.0

Contents

1 Introduction 1

I Code Development Tools 9

2 Compilers and Linkers 11

3 Debuggers 25

II Linear Algebra Libraries 29

4 The Basic Linear Algebra Subroutines (BLAS) 31

5 Linear Algebra PACKage (LAPACK) 45

III Collaboration and Performance 53

6 Git Version Control 57

7 Build systems 69

8 Testing 77

9 Documentation 81

10 Profilers and Code Optimisation 87

11 Clusters and Job Scheduling (Optional) 99

IV Parallel Programming 107

12 Distributed Parallel Programming with MPI 109

13 Shared-memory Parallel Programming with OpenMP 125

14 Parallel Programming on GPUs with CUDA (Optional) 135

15 ScaLAPACK (Optional) 143

V Supplementary and Advanced Material 149

A Unix/Linux Operating System 151

B List of Common File Types 159

Chapter 1

Introduction

When we think of the term *high-performance computing*, we might think of large complex calculations being undertaken on massive supercomputers housed in a data centre. Examples of these include simulating the aerodynamic performance of an aircraft or Formula 1 car, assessing the structural properties of a new composite material, predicting the chemical reaction of two substances, modelling blood flow through a patient, analysing the human genome, or training a neural network to play Go!

This is not an unreasonable interpretation and is frequently the end-point of the practice of applying high-performance computing techniques. However, to reach this goal requires us to develop an understanding of how computers function, how to develop *robust, maintainable* code, how to *maximise the performance* of that code on an individual computer processor, as well as how to structure the code in such a way that it can run *in parallel* across many computer processors. This will necessitate that we leverage a broader eco-system of tools and libraries of code written by others. These aspects will all be discussed in this course through the C++ programming language and the UNIX/Linux operating system (see Appendix A), although many of the concepts are broadly applicable to other languages.

Let us begin with the following definition:

Definition: High-performance Computing

The practice of leveraging the capabilities of a compute unit, and possibly aggregating many individual compute units, in a way which leads to faster calculations, higher throughput or to be able to undertake calculations otherwise unachievable on an individual unit.

In Part I of this course, we will consider the key tools for code development that we will need in the rest of the course. This includes the compiler, for turning our source code into executable code, and the debugger, to help us identify errors in our code. We will learn how to make use of external libraries of code. Part II looks at two such libraries which implement linear algebra operations, since these form the building blocks of many scientific and engineering codes.

The topics in Part III cover aspects of code development and computer architecture which enable us to write robust and efficient code as part of a team. We will look at version control and build systems to support collaborative development and how to develop test suites to ensure correctness of individual components. We will also look at how we can align our algorithms with the architecture of the computer to improve computational efficiency and use performance profilers which allow us to identify the regions of our code which take the most time to execute, on which we can then focus our optimisation efforts.

Finally, in Part IV, we will look at approaches to combining the power of multiple computer processing cores to undertake a calculation collectively (the *aggregation* part of the definition above).

Before proceeding further, it is useful to gain an understanding of how the computers we are using

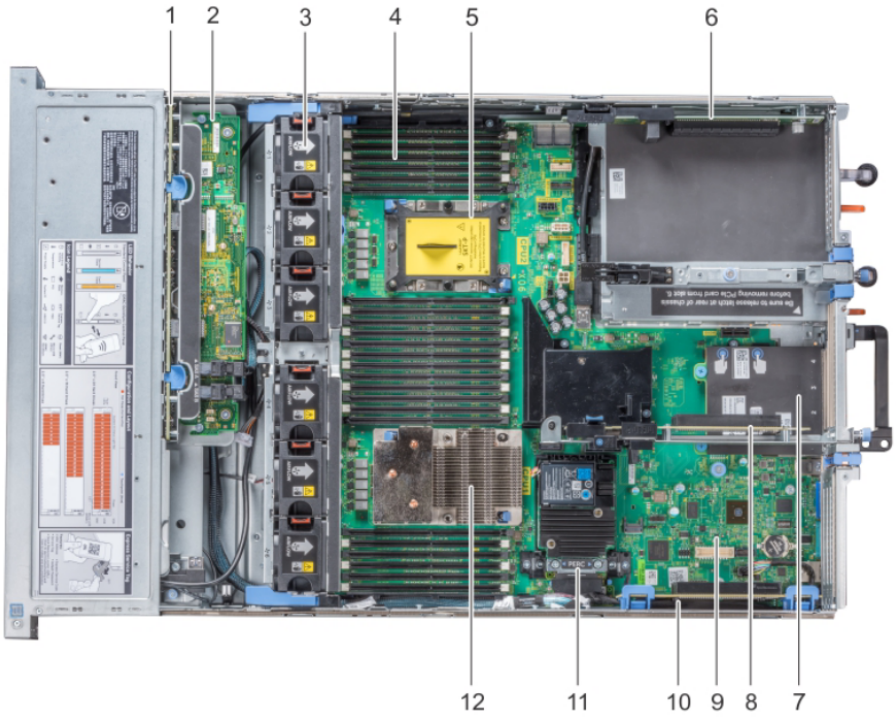


Figure 1.1: Photo of the inside of a Dell R740 compute node. Key components are the two CPUs (5 and 12), the main memory (4), expansion slots (6, 8, 10), the network card (7) and the system board (or motherboard, 9).

work and become familiar with the terminology used. Doing so, will enable us to better understand how to use them most effectively to achieve performant code.

1.1 Computer Architecture

We start by taking a look at a high-performance computer similar to those we will make use of on this course. These computers are designed to be installed in a data centre and so are built differently to the average desktop or laptop computers, but their architecture is fundamentally the same. A top-view of the inside of such a system is shown in Figure 1.1, labelled to show the key components.

The Central Processing Unit (CPU) is the component which will interpret and carry out the instructions we provide in our program. The system shown above is a dual-socket system – that is, it has two CPUs. In order to write meaningful programs, there needs to be a mechanism to temporarily store and retrieve data and this is provided by the main memory. These are connected together through the system board, which also provides access to peripheral components (such as hard disks, network cards and USB ports). The expansion card slots allow the computer to be extended with additional components such as a general-purpose Graphics Processing Unit (GPU).

Figure 1.2 represents these components diagramatically. Each CPU is connected to its own memory through the memory bus. In this example, there are six memory banks, each connected by its own channel to the CPU. This allows data in different memory banks to be accessed concurrently. The width of each channel determines how much data can be transferred at the same time; this is usually measured in bits. A 64-bit computer has a memory bus width of 64 bits (or 8 bytes). The example shown is a dual-socket system, where the CPUs are connected through Intel's Ultra Path Interconnect (UPI). This allows, for example, CPU0 to access the memory of CPU1, although at a slightly slower speed than accessing its own memory. This leads to *non-uniform memory access (NUMA)* which can lead to reduced performance since the operating system is free to move processes between the CPUs.

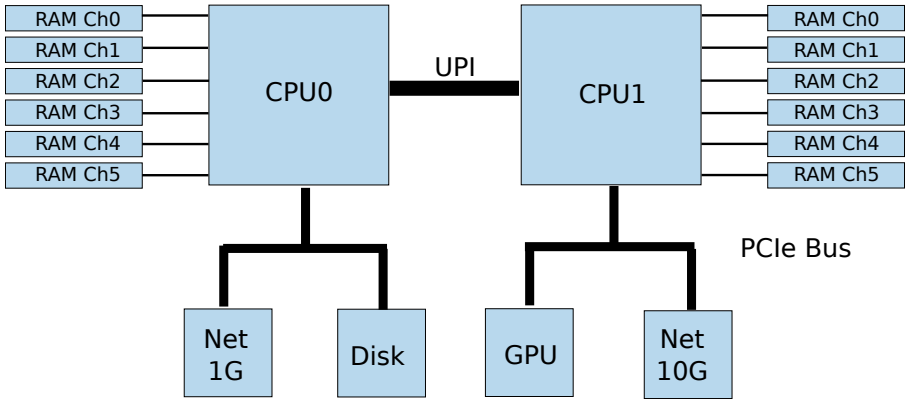


Figure 1.2: Diagram of the connections between key components of a dual-socket high-performance computer.

The processor *clock speed* relates to the frequency at which transistors within the processor can be switched. Every instruction or operation requires a number of clock cycles to complete ranging from one cycle for addition and multiplication to tens of clock cycles for division, up to hundreds of clock cycles to fetch data from main memory. In the early 2000s, physical limits in the number of transistors that processor manufacturers could fit on a single processor *die* while being able to cool it effectively meant that processor clock speeds have saturated at about 4 Ghz. To enable further performance growth, manufacturers were forced to introduce parallel processing within the CPU, by adding support for data parallelism (vectorisation) and instruction parallelism (multiple cores).

Definition: Vectorisation

Vectorisation exploits the concept of Single Instruction Multiple Data (SIMD) where a single program instruction is applied concurrently to multiple data elements. Modern CPUs contain explicit low-level instructions sets (for example, AVX2 or AVX512) for vectorising an algorithm.

Each modern CPU consists of multiple processing *cores*. These can process different instructions concurrently (either from different processes, or different threads of execution within the same process). Often a CPU will also support multiple *hardware threads* per core. These are different to *threads* discussed in the programming context (see Chapter 13). Hardware threads enable multiple instructions to be handled concurrently on a single core, but they share some of the components of the core (e.g. the floating-point unit (FPU)). For applications which do limited compute, this works well, since the CPU is actually idle most of the time waiting for data from memory, disk or user input. For compute-intensive applications, this can lead to a performance bottleneck as threads compete for use of shared components within the core. A useful measure of whether an algorithm is compute-intensive is the *arithmetic intensity*

Definition: Arithmetic Intensity

The arithmetic intensity of an algorithm is the ratio of floating-point operations performed to the number of bytes accessed.

Note that the *number of bytes accessed* is the number of bytes transferred from main memory to the CPU and is different to the amount of memory used by the algorithm. For example, if a single $N \times N$ matrix is used five times during the execution of an algorithm, the bytes accessed would be $5N^2$. Algorithms which can do more with a piece of data while it is on the CPU can therefore be favourable. The arithmetic intensity of an algorithm is not the whole picture and the way in which data is accessed in memory can have a large effect on performance.

1.2 Computer Memory

Computer memory is volatile storage and its contents is lost when the computer is switched off. This is in contrast to disk (magnetic or solid state) which retains data on loss of power. It is a sequence of locations, logically arranged in a linear fashion and each with a unique address. Each location can store a single *binary* number. A binary number is 1 byte = 8 bits, which is eight 0's and 1's. We can access data in memory in constant time, meaning it takes the same time to access the first location as it does the last.

A computer has a hierarchy of memory levels. The level at the bottom is main memory, also known as *Dynamic Random Access Memory (DRAM)* since it needs to be periodically refreshed to avoid the data being lost. It takes around 100 clock cycles to retrieve data from main memory. To help speed up programs which access data repeatedly (i.e. with *temporal locality*), the processor itself has one or more levels of fast memory, termed *cache*. Lower levels such as L1 cache are the smallest and are closer to the processor and faster to access, while higher levels such as L3 cache are larger, but slightly slower to access. L1 access times can be 10-100 times faster than main memory.



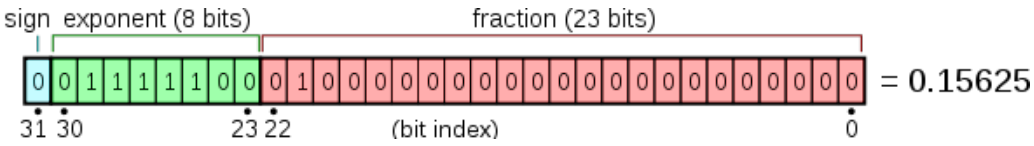
Note

The management and usage of cache on the processor is handled automatically by the computer hardware and is not managed explicitly when we write computer programs in C++. However, it can be helpful to understand how the cache is likely being used in order to write more performant algorithms.

The main cost to accessing memory is *latency*. To further improve the performance of algorithms, when the processor requests a value from memory, an entire *cache line* is fetched. This exploits the idea of *spatial locality*, that a program is likely to access the next value in memory and thereby it can retrieve that value directly from cache without a further request to main memory.

1.2.1 Representing data in memory

The smallest addressable memory unit is 1 byte (8 bits); this allows 256 possible combinations. Often the data we want to represent, for example floating-point numbers, requires more combinations than 8 bits allows. Multiple memory locations can be addressed together to represent data which can represent many more values. A *datatype* tells the computer how many memory locations to use and how to interpret the binary digits within them. For example, a floating-point number typically uses 4 bytes of memory and is interpreted as shown below:





Note

Different computers (e.g. 32bit and 64bit) may represent the same data type in different ways.

1.3 Using High-performance Computers

Most Top500 supercomputers are distributed memory machines. See a complete list at:
https://www.top500.org

They achieve their computational power by connecting many individual computers (nodes) together using a very fast computer network. Software which uses these types of computers is run on each of the nodes and needs to be able to break up the task into chunks which can be distributed between the nodes of the supercomputer. The different chunks communicate with each other, usually using a paradigm called *message-passing*, which will be covered in Chapter 12. Within a node, each chunk can be broken up further into multiple *threads* of execution using a different programming model we cover in Chapter 13.

1.3.1 High-performance computing at Imperial College

Different types of computation work most efficiently on different types of computers. To cater for these needs, Imperial College has several research computing systems:

- **CX1/CX3: High-throughput cluster**
  Many (mixed specification) nodes connected with ethernet. Good for many concurrent serial jobs, but does not perform well with node-parallel jobs.
- **CX2: High-performance parallel system**
  Large number of homogeneous nodes of identical specification. Infiniband interconnect for low-latency, high-bandwidth MPI communication. Good for large-scale distributed massively-parallel jobs with thousands of processes.

For more information, see the Research Computing Service webpages:
https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/.
Usage is restricted to students undertaking computational research projects.

The Department of Aeronautics also runs a small Linux cluster. This can be used by undergraduate and postgraduate students for any taught module, group design project or individual project:
https://www.imperial.ac.uk/aeronautics/local/computing-services/.

See Section 11.2 for guidance on how to submit jobs to any of these clusters.

1.4 C++ Programming Language

We will use the C++ programming language in this course. C++ is a general-purpose language used to instruct a computer to perform a sequence of tasks or calculations. It is an intermediate-level language in the sense that instructions, or *source code* are given using human-readable text, yet it does contain some low-level capabilities which, for example, allow direct access to, and manipulation of, computer memory. As a consequence, it can be used to write very efficient, high-performance code. It is a *compiled language*, which means the source code must be converted or *compiled* into a machine-readable file of instructions that the computer processor can run directly.

Many high-performance computing libraries use native C++ arrays and pointers to exchange data, so you should ensure you are familiar with these topics. To refresh your memory of C++, have a go at some of the revision exercises below.

Exercises

Exercise 1.1

Using recursion, write a program which computes the first 15 Fibonacci numbers. You should write a function which computes the n-th number in the Fibonacci sequence x_n, given that

$$x_n = x_{n-1} + x_{n-2} \quad \text{and} \quad x_0 = x_1 = 1$$

Exercise 1.2

Write a program which computes an approximation to π using both of the following formulae:

$$\frac{\pi}{4} \approx \sum_{k=0}^{n_1} \frac{(-1)^k}{2k+1} \qquad \frac{\pi}{2} \approx \prod_{k=1}^{n_2} \frac{4k^2}{4k^2-1}$$

and hence find n_1 and n_2 such that these approximations are accurate to five significant figures. Which approximation converges faster?

Exercise 1.3

1. Write a function `vecadd`, which, given two vectors $\mathbf{a}$ and $\mathbf{b}$ of the same length, calculates their element-wise sum $\mathbf{a} + \mathbf{b}$.
2. Write a function `dot`, which, given two vector $\mathbf{a}$ and $\mathbf{b}$ of the same length, calculates their dot product $\mathbf{a} \cdot \mathbf{b}$.
3. Write a function, `multiply`, which, given two square $n \times n$ matrices $\mathbf{A}$ and $\mathbf{B}$, computes the product $\mathbf{AB}$ into another matrix $\mathbf{C}$.

Write a program which tests your functions. Allocate the vectors and matrices in your program using dynamic memory. Pass the vectors and matrices to the functions using pointers. Generate some test vectors and matrices to test your code.

Exercise 1.4

The matrix product can be computed using a recursive algorithm. Each matrix is split into quarters, and the sub-blocks are multiplied together (like with 2x2 matrices). Then:

$$\mathbf{C}_{ij} = \sum_k \mathbf{A}_{ik}\mathbf{B}_{kj}.$$

Write a program which allocates three $n \times n$ matrices, each represented as a single contiguous block of memory (i.e. the matrices are stored row-by-row in memory), and a function `recursiveMultiply` which implements the recursive matrix-matrix multiplication algorithm. Your function will need the following parameters:

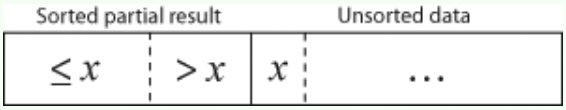
- The starting position of the sub-matrices of $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ to be multiplied.
- The size m of the sub-matrices to be multiplied, e.g. $m = n$, $m = n/2$, $m = n/4$, etc.
- The size of the original full matrices (i.e. n). This is needed so you know how far to move in memory to move from one row of the matrix to the next.

Remember to implement the stopping condition when $m == 2$. Test your code with some example matrices and/or compare with your code for Exercise 1.3.

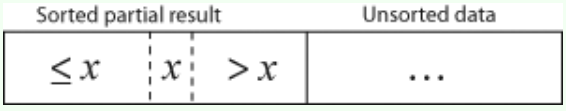
Exercise 1.5

In this exercise we will sort an array using the *Insertion Sort* algorithm.

This is an *in-place* sorting algorithm. It starts with the first element and iterates through the array in order. It *inserts* each item into the correct place in the preceding (sorted) elements. For example, at a given iteration,



the value x is moved into the correct position and those values $> x$ are moved up the array:



When the last element has been positioned in the correct location, the array is sorted.

1. Write a function `insert` which takes an array `a`, a position `p` (which must be strictly less than the total size of `a`), and a floating-point value `x` to insert. Assuming that `a` is already sorted up to position `p`, this function should insert `x` in the correct position (overwriting `a[p+1]`). Obviously there is nothing to do if `p=0`. Now all the values of `a` up to position `p+1` are sorted in ascending order.
2. Write a function `insertionSort` which takes an array `a`, and it's length `l`. For each element `i` of the array in order, make a call to `insert` passing `a`, `(i-1)` as the position, and the value of `a[i]` as the value to insert.

Part I

Code Development Tools

Chapter 2

Compilers and Linkers

Objectives

In this chapter you will learn about:

- how to use multiple source files to better organise large codes;
- the range of options which can be used to modify the behaviour of the compiler;
- the C++ linker and its purpose;
- how to use and link to external libraries of code.

This chapter discusses the compiler/linker. Typically, as is the case with the GNU C++ compiler `g++`, these two processes are handled by the same executable. One can perform the entire compilation and linking process for an application using a single `g++` command. However, there are benefits to breaking up the process into stages for larger projects.

The `g++` compiler/linker can be invoked as follows in order to compile and link multiple C++ source files into an executable.

```
g++ file1.cpp file2.cpp
```

This will first perform compilation of each source file into a corresponding object file, before linking these together (with some system libraries) to produce an executable called `a.out`. We now explore in more detail what happens during the compilation and linking processes and discuss some of the available optional arguments to the `g++` tool.

2.1 The Compilation Process

The compilation of C++ source code into an executable code which can be directly run by the computer's processor is a complex process. It involves several stages in which the human-readable text, often combined with other codes, is interpreted and translated into machine-readable binary instructions that can be executed directly by the Central Processing Unit (CPU).

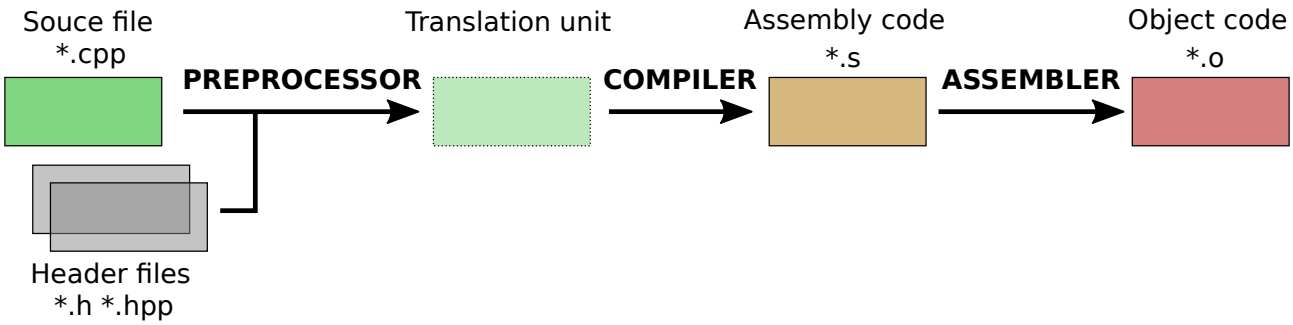


Figure 2.1: The compilation process consists of three distinct stages. Filename extensions shown are conventions used on Linux-based systems.



Note

Only C++ source files are passed to the compiler. Header files are loaded implicitly through the use of `#include` commands in the source file.

Conceptually, this process is broken down into three stages, which is illustrated in Figure 2.1, although in general (i.e. particularly outside of this chapter) we refer to the three stages together as *compiling*:

- **Preprocessing**: the preprocessor actions preprocessor statements and merges in header files to create a single *translation unit*.
- **Compilation**: the compiler translates this translation unit into assembly code instructions.
- **Assembling**: the assembler takes the assembly instructions produced by the compiler and creates *relocatable object code*.

We are very rarely interested in anything but the end result of this process (the object code).

2.1.1 Preprocessing

In practice, the preprocessing step is usually executed automatically before the compilation process begins. The main tasks undertaken during preprocessing include:

- Inserting the contents of header files in place of the `#include` preprocessor statements.
- Expanding macros such as `#define` preprocessor statements.
- Remove code within conditional `#if`-`#endif` preprocessor statements if the condition evaluates to `false`.

Even the simplest of programs usually results in many thousands of lines of code due to included header files.

To see the result of the pre-processing step, one can ask `g++` to stop after this step by using the `-E` argument, for example:

```
g++ -E file1.cpp -o file1.ii
```

The `-o file1.ii` argument instructs the `g++` to write the expanded source code to file instead of the terminal.

2.1.2 Compilation

The compiler translates C++ instructions from the translation unit into low level assembly code. Assembly code (or symbolic machine code) consists of instructions which are specific to the target CPU. These are typically mnemonics. Many simple statements in C++ will result in several low-level statements in the corresponding assembly code.

To see the result of the compilation process, use the `-S` argument:

```
g++ -S file1.cpp
```

The resulting file `file1.s` then contains assembly code for the translation unit.

As an example, the trivial lines of C++ code

```
int x = 42;
int y = 99;
int z = x + y;
```

results in the following assembly code on a typical Intel processor:

```
movl    $42, -4(%rbp)
movl    $99, -8(%rbp)
movl    -4(%rbp), %edx
movl    -8(%rbp), %eax
addl    %edx, %eax
movl    %eax, -12(%rbp)
```

The symbols `eax` and `edx` are two of the CPU registers (memory locations in the CPU itself which hold one value). The instruction `movl` moves an integer value into one of those registers. The `addl` is used to add the contents of the first register to the second.

Beyond an appreciation for the process of compilation, we do not need to understand assembly instructions in this course.

2.1.3 Assembly

Finally, the assembler converts the assembly code into binary machine code instructions which can be directly understood by the processor and which is generated in such a way that it can be put together with other object code (relocatable) by the linker.

To stop after this step we could use

```
g++ -c file1.cpp
```

2.2 The Linking Process

Linking is the process of connecting the compiled object code generated from the different source code files into a single executable. The source code may make reference to *symbols* (such as functions) provided by external libraries. It is also the job of the linker to make sure these libraries and required symbols are available and that the executable knows the position in the library to go to when the symbol is required. The linking process is illustrated in Figure 2.2.

Unless requested to stop at an earlier point in the compilation process using one of the options discussed above, the linking process will automatically be performed to generate executable code.

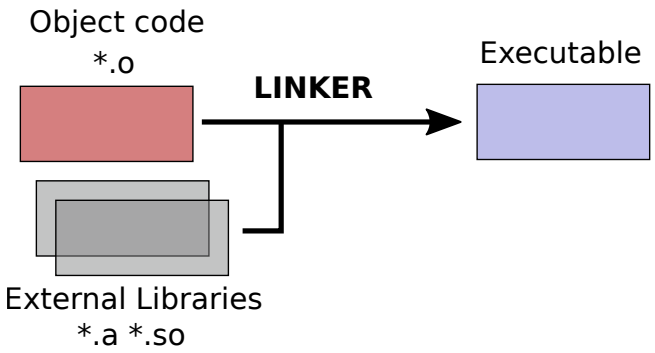


Figure 2.2: The linking process puts together the individually compiled translation units with external libraries to create an executable.

To perform the linking process on previously generated machine code (files typically with the `*.o` extension), one or more of these files are specified on the command-line, instead of the source code files. For example,

```
g++ file1.o file2.o
```

will link the code in `file1.o` and `file2.o` and generate an executable called `a.out`.

Rather than accepting the default output name of `a.out`, the name of the executable can be specified using the `-o` command-line argument, for example,

```
g++ -o myprog file1.o file2.o
```

2.3 Compiling multiple source files

As a program becomes larger and larger it is common practice to separate the program source code into several (if not many) files. This helps keep the program code organised and manageable, and therefore easier to work with. There are two important types of files we should use: source files (.cpp) and header files (.h). See Appendix B for more information on the purpose of these file types.

We start by moving functions into separate files. The choice on how to organise functions and classes in files is down to the programmer, but by convention we usually aim to collate related functions together, or restrict ourselves to one class per source file (naming the source file the same as the class). Furthermore, we extract function prototypes and class declarations into corresponding header files. An example of this is shown in Example 2.1.

We can now compile the source files separately and link everything together. Note that the header file is not compiled, although it is included in both `main.cpp` and `myfunction.cpp` by the preprocessor.

```
# Compile main code
g++ -Wall -O2 -o main.o -c main.cpp
# Compile myfunction
g++ -Wall -O2 -o myfunction.o -c myfunction.cpp
# Link everything together
g++ -o myprog main.o myfunction.o
```

2.4 Compiler/Linker options

Compilers can take many arguments which influence how the code is compiled. Some of these are essential, such as specifying the source code files and libraries needed by the program, while others

Example 2.1: Splitting up code into multiple files.

`myfunction.h`

```
1 void tripleValue( double & );
```

`myfunction.cpp`

```
1 #include "myfunction.h"
2 void tripleValue( double & pVal) { pVal *= 3.0; }
```

`main.cpp`

```
1  #include <iostream>
2  #include "myfunction.h"
3
4  int main(){
5      double vMyVal = 4.0;
6      std::cout << "Initial value is: " << vMyVal << std::endl;
7      tripleValue(vMyVal);
8      std::cout << "Triple value is:  " << vMyVal << std::endl;
9      return 0;
10 }
```

might tune the compilation process and resulting executable code, include additional information to assist with debugging (identifying why code does not work as expected) or provide warnings about possible problems with the code.

It is important to remember that the way a code is compiled can make a big difference in performance.

2.4.1 Search Paths

The majority of applications, which perform anything but the most trivial of tasks, rely on external code that is made available in pre-compiled libraries and/or external header files. The compiler will search through a number of standard locations when looking for these files, for example `/usr/lib` for libraries and `/use/include` for headers on Linux systems. Libraries installed through the Linux package manager will almost always install libraries into these locations, so no additional command-line arguments are needed.

If a library is located elsewhere, the compiler must be told additional paths in which to search. This is accomplished with the `-L` option, placed before the library name itself:

```
g++ -o myprog file1.o file2.o -L/path/to/my/lib -lmylib
```

Similarly, when compiling, if header files are located in a non-standard path, the compiler can be instructed to search these paths using the `-I` option:

```
g++ -c -o file1.o -I/path/to/my/headers file1.cpp
```

These can be combined if compiling and linking together:

```
g++ -o myprog -I/path/to/my/headers -L/path/to/my/lib file1.cpp -lmylib
```

2.4.2 Warnings

The compiler and linker is a very complex and powerful tool. In the process of converting C++ code to machine code, it must decipher each and every symbol in the code and the relationships between them. As a consequence, it can identify places in the code where mistakes may have been made, there are ambiguities over what is intended, or the resulting code may be sub-optimal.

The options below control how much of this information is displayed.

Option	Description
`-Wall`	turns on all warnings, including those below
`-Wcomment`	displays nested comments warnings
`-Wformat`	displays incorrect use of format strings in `printf`
`-Wunused`	displays any variables which are declared but not used
`-Wimplicit`	displays functions which are used without declaration (header is missing)

Some additional warning flags which may be useful include:

Option	Description
`-W`	common programming errors (e.g. logical expressions which can never be true)
`-Wconversion`	conversion errors (e.g. passing the wrong type to a function)
`-Wshadow`	redeclaration of a variable

For more details see https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

2.4.3 Optimisations

If instructed, the compiler will try to optimise the code to try and produce a faster or smaller executable. As a rule, one should initially compile without optimisation flags, check that the executable is functioning correctly and then try higher optimisation levels. The reason for this is that the process of optimisation can make the process of debugging code more challenging.

There are four levels of optimisation:

Option	Description
`-O0`	no optimisation at all, no code rearrangement
`-O1`	common optimisations with no space trade-off, no instruction scheduling
`-O2`	turns on instruction scheduling but does not increase executable size or memory requirement; good for production level
`-O3`	turns on options that make executable larger, e.g. function inlining

For example, we can compile our earlier example with some optimisations enabled:

```
g++ -O2 -c -o file1.o -I/path/to/my/headers file1.cpp
```

2.4.4 Debugging and Profiling

The compiler can be instructed to generate an executable that can be used by debuggers and profilers, by retaining references to the original source code alongside the machine code. A consequence of this is that the executable is larger. Optimisations should also be disabled as these may break the association between machine code symbols and source code.

Debug symbols are added when the `-g` argument is specified. It should be specified for both the compilation and linking stages. For example:

```
g++ -g -o myprog file1.cpp
```

For more details on debugging code, see Chapter 3.

2.4.5 Language standard

C++ is an evolving language and its specification has been defined several times over the years in different specification documents, called *standards*. GCC supports the original ISO C++ standard published in 1998, as well as newer 2011, 2014, 2017 and 2020 revisions. Some newer language features are only available in the later revisions of the standard and other features have been deprecated or removed.

To tell the compiler which standard to adhere to, we use the `-std` argument. For example:

```
g++ -std=c++11 -o myprog file1.cpp
```

The options are

- `-std=c++98`, or `-std=c++03` - Original standards
- `-std=c++11` - 2011 revision (default for our compiler version)
- `-std=c++14` - 2014 revision
- `-std=c++17` - 2017 revision
- `-std=c++20` - 2020 revision

2.5 External Libraries

The syntax of C++ is rich enough to enable us to construct a broad range of complex algorithms to perform the wide variety of tasks we expect and demand of modern computers.

However, particularly in scientific and engineering applications, the same complex algorithms reappear repeatedly in many different codes. Reimplementing these algorithms efficiently in each code is time-consuming and increases the likelihood of introducing errors. A solution to this is to package these algorithms into a separate unit (or *library*), external to each application code, but which can be leveraged by a number of such codes.

Definition: External Library

An external library is a binary-compiled set of data structures and algorithms which are accessed by one or more application codes through a documented application programming interface (API).

There are now an enormous range of scientific libraries available which can be used to add a rich set of capabilities to your code and provide a number of advantages. Well-established scientific/numerical libraries typically contain code which is well tested and documented by the community and are often supplied in source-code form, particularly if they are academic codes and *open-source*. This allows you to inspect the exact algorithm implemented, which can be helpful if things do not work quite as you expect. Sometimes, libraries are commercial and may only be supplied in compiled binary form. The most common example of this in HPC is for vendor-tuned versions of well-established libraries which are optimised to work efficiently on the vendor's own HPC hardware.

2.5.1 Specifying External Libraries

To tell the linker that an external library will provide some of the functionality needed by a program, these must be listed as arguments on the command-line. Library names on Linux typically start with `lib` and either have a `.a` or `.so` extension. The former indicates the library is designed for static linking, while the latter for dynamic linking. Often a library with both extensions is available on the system. The part between the `lib` and the extension is the *basename*. The compiler will choose to link dynamically by default if only the basename is provided with the `-l` option. Alternatively the full path to the library can be provided on the command-line, without using the `-l` option.

For example:

```
g++ -o myprog file1.o file2.o -lblas
```

or

```
g++ -o myprog file1.o file2.o /usr/lib/libblas.so
```



Note

Dynamic linking just adds a reference to the dynamic library file (`*.so`) in the executable. When the executable is run, the dynamic library must also be available and is loaded by the operating system.

Static linking actually *copies* the used symbols from the library file into the resulting executable; this allows the executable to be moved to another system (of the same CPU type, of course) and run without requiring the library files. However, the executable is of course potentially much bigger in size.

2.5.2 Library dependencies and ordering

Some libraries depend on other libraries and all must be specified for the linking process to complete. Furthermore, the order in which libraries are specified on the command-line matters. When the compiler encounters a symbol (such as a function) in a translation unit or library which does not exist in that unit or library, it looks for it in the libraries which follow using the ordering specified on the command-line. It will not look at libraries specified earlier.

For example, the LAPACK library uses routines from the BLAS library. Therefore, both libraries must be specified and the LAPACK library must be specified *before* the BLAS library:

```
g++ -o myprog file1.o file2.o -llapack -lblas
```

2.5.3 Undefined References

If the linker cannot find a symbol it reports an *undefined reference* and an error such as the following is shown:

```
/tmp/cc49I6tc.o: In function ‘main()’:
test.cpp:(.text+0x9fe): undefined reference to ‘dgbmv_’
```

We might have forgotten to link against the library which provides the function (in this case BLAS), listed the libraries in the wrong order on the command-line, or declared a function in our code and not defined its implementation.

2.5.4 Using an External Library

External libraries typically consist of the compiled object code, a set of header files defining the API and some documentation describing the API and its usage. The file extension of the compiled object code is different on various operating systems. On Linux, they will have the file extension `.so` or `.a`, which distinguishes between dynamic and static linking (see Section 2.5.1). On OS X, libraries typically have the extension `.dylib`, while on Windows they have the extension `.dll` or `.lib`.

To use a library, which here we suppose is called `ext`, we typically follow the 5 steps below:

1. Read the documentation for the library!
2. `#include` the header files, probably called `ext.h`.
3. Write our code using the library API to access functions, classes, etc, as required.
4. Compile our source code (we may need to specify *include directories* - see Section 2.4.1)
   ```
   g++ -c -I/path/to/headers -o test.o test.cpp
   ```
5. Link the external library file, probably called `libext.so`, when building the executable:
   ```
   g++ -o test test.o /path/to/library/libext.so
   ```
   OR
   ```
   g++ -o test -L/path/to/library test.o -lext
   ```

In practice, `ext` is replaced with the basename of the library you need to use (e.g. `fftw3`, `boost_program_options`, `blas`, `lapack`). We will discuss some of these libraries in the subsequent chapters.

In Codelite, to link against an external library, you must add the library to the project as follows:

- Right-click on the appropriate project in the Workspace and select *Settings...*
- Select *Linker* from the list on the left.
- Click the *Libraries* entry in the table and then the ... button which appears.
- Enter `ext` as the box which appears. Multiple libraries can be specified, with each on a separate line.
- Click OK twice to return to the main window.
- When you build your project, you should see `-lext` at the end of the g++ command in the *Build* window.

Again, use the basename of the library you need, in place of `ext`.

2.6 External Library Examples

2.6.1 Popular Numerical Libraries

Below are some examples of external libraries. In Sections 2.6.2 and 2.6.3 we look at two of these in a little more detail to illustrate the general principles of using external libraries. We then discuss some of the key libraries in high-performance computing in later chapters.

Computational Linear Algebra

- BLAS: Basic Linear Algebra Subprograms (Chapter 4)
  Perform vector-vector, matrix-vector and matrix-matrix operations.
- LAPACK: Linear Algebra PACKage (Chapter 5)
  Solve linear systems and eigenvalue problems.
- ScaLAPACK: Scalable Linear Algebra PACKage (Chapter 15)
  Solve linear systems and eigenvalue problems in parallel.
- ARPACK: ARnoldi PACKage
  Solve eigenvalue problems using iterative Arnoldi algorithms.
- Intel MKL: Intel Math Kernel Library (commercial)
  Implements much of the above, but optimised for Intel architectures.

Partial Differential Equations

- PETSc: Portable, Extensible Toolkit for Scientific Computation
  Data structures and routines for solving PDEs in parallel.

Scientific Computing

- MPI: Message Passing Interface (Chapter 12)
  Distributed parallel programming library using message passing.
- OpenMP: Multi-processing extension (Chapter 13)
  Shared-memory parallel programming library.
- FFTW: Fastest Fourier Transform in the West (Section 2.6.2)
  Efficient calculation of Fourier transforms.
- GSL: GNU Scientific Library
  General-purpose numerical library for scientific computation.
- Trilinos
  Reusable scientific solvers, particularly for linear and non-linear systems.
- NAG mathematical and statistical routines (commercial)
  General-purpose numerical library.

General C++ libraries

- Boost (Section 2.6.3)
  Collection of free peer-reviewed portable C++ source libraries.

2.6.2 FFTW

FFTW is a library for efficiently performing real or complex discrete Fourier transforms of arbitrary size. It performs best for sizes with small prime factors (e.g. 900), supports one-dimensional or multi-dimensional transforms and from version 3.3 it supports performing transforms in parallel using MPI.

An example of using FFTW from an application code is given in Example 2.2. On line 1, we include the header file which provides the prototypes for the functions in the FFTW library. Lines 4 and 5 declare arrays of type `fftw_complex` and a variable of type `fftw_plan`; these are data types/structures provided by FFTW and defined in the `fftw3.h` header file. Line 9 uses a function provided by the FFTW API to optimise the transform algorithm for the provided array size `N`. Line 10 uses another API call to actually perform the transform and line 11 uses another API call to request FFTW frees any memory it allocated during the process.

To link with FFTW, we would use a command such as

```
g++ myprog.cpp -lfftw3
```

assuming the library is installed in a standard system location.

Example 2.2: Using the FFTW library to efficiently perform the discrete Fourier transform of an array of values.

```
1  #include <fftw3.h>
2
3  int main() {
4      const int N = 64;
5      fftw_complex in[N], out[N];
6      fftw_plan p;
7
8      // populate 'in' here
9
10     p = fftw_plan_dft_1d(N, in, out, FFTW_FORWARD, FFTW_ESTIMATE);
11     fftw_execute(p);
12     fftw_destroy_plan(p);
13 }
```

2.6.3 Boost

Boost is actually a set of libraries, which provide a useful collection of algorithms and data structures for use in a broad range of application codes. Portability across operating systems is also strongly emphasised, which means it helps us write codes which could be run on Linux, Windows and OS X, if desired.

Some of the functionality available includes:

- Cross-platform filesystem library.
- Powerful formatting library.
- Input/output streams, including compression.
- Lexical casts, e.g. int to string.
- Mathematical functions and statistical distributions.
- Library for parsing command-line options.

Extensive documentation for these and all the other libraries is available at:
http://www.boost.org/doc/libs/1_74_0/

Boost contains two types of libraries: compiled libraries and *header-only* libraries. In the case of the latter, the algorithms are implemented entirely in templated header files which are compiled as part of the application code compilation process; therefore, there is no binary code associated with these libraries and no explicit linking is required.

Some Boost libraries also have inter-dependencies. To use these libraries, you must link to both the target library, plus all of its dependencies. For example, the `boost_filesystem` library uses functionality from the `boost_system` library. To link against the `boost_filesystem` library, we must therefore use a command such as

```
g++ myprog.cpp -lboost_iostreams -lboost_filesystem -lboost_system
```

Remember, the order of the libraries in the command is important (see Section 2.5.2).

Parsing command-line arguments

Options can be given to our programs when they are run from the command-line, for example:

```
./myprog 0 10
```

We can access these values in our code by parsing the `argc` and `argv` parameters to the `main` function. The `argc` parameter tells us the number of parameters (including the program name itself), while the `argv` parameter is an array-of-character-arrays which contains the actual arguments given.

Example 2.3: Using the `program_options` library to parse command-line arguments.

```
1  #include <iostream>
2  #include <boost/program_options.hpp>
3  namespace po = boost::program_options;
4
5  int main(int argc, char* argv[]) {
6      po::options_description opts(
7          "Available options.");
8      opts.add_options()
9          ("start", po::value<int>()->default_value(0),
10          "Starting value.")
11         ("end",   po::value<int>()->default_value(10),
12          "Ending value.")
13         ("help",       "Print help message.");
14
15     po::variables_map vm;
16     po::store(po::parse_command_line(argc, argv, opts), vm);
17     po::notify(vm);
18
19     if (vm.count("help")) {
20         std::cout << opts << std::endl;
21     }
22     const int start = vm["start"].as<int>();
23     const int end   = vm["end"].as<int>();
24 }
```

Parsing the latter directly can be a little tedious, but Boost includes the `program_options` library to make the process easier. Example 2.3 illustrates this. We include the necessary header file on line 2 and declare an alias for the `boost::program_options` namespace on line 3 to reduce typing. On

lines 6-13, we tell boost about the options our program should accept. We specify the data type of the values expected and can optionally provide a default value (should the user not specify the option). Lines 15-17 parses the options given and populates the map of given options in `vm`. We can then extract the values from the map and use them (lines 19-23).

We can now call our program using more descriptive options:

```
./myprog --help
./myprog --start 2 --end 5
./myprog --start 2          # end will be 10 by default
```

Exercises

Exercise 2.1

Download the files `compiler-main.cpp`, `compiler-norm.cpp` and `compiler-norm.h` from Blackboard Learn. This program uses the BLAS library (`/usr/lib/libblas.so`, see Chapter 4) and a manual routine to compute the norm of a randomly generated vector of numbers.

Open a terminal and change to the directory in which you downloaded the files.

1. Use the compiler and an appropriate command-line argument to pre-process the `compiler-main.cpp` file. Examine the output to observe the effect of preprocessor statements.
2. Use appropriate compile flags to compile each of the files separately into object code.
3. Use the compiler to link the object code into an executable.
4. Does this code meet the C++98 standard? Use the compiler to test it.
5. Execute the code; notice that it just produces zero. Use compiler warning flags to help you identify the errors in the code.
6. When the code produces a non-zero result and both algorithms match, examine the effect of using optimisation flags on the runtime of the code, using the Linux `time` command.

Exercise 2.2

Make a copy of your solution to Exercise 1.2 and update it to approximate Pi to 7 significant figures.

1. Use a terminal to compile and run your code to check it works.
2. Use Boost `auto_cpu_timer` (https://www.boost.org/doc/libs/1_74_0/libs/timer/doc/cpu_timers.html) to time how long the program takes to run.
3. Time the compilation process by prefixing your compiler command with the `time` command.
4. Investigate the different compiler optimisation flags and their impact on compilation time, execution time and the size of the executable.

Exercise 2.3

Rather than presenting the user with a menu to choose program parameters, one can specify parameters on the command-line. The *Boost* library `program_options` offers a straight-forward way to do this.

To learn about how the Boost `program_options` library works, have a look at the tutorial:
http://www.boost.org/doc/libs/1_74_0/doc/html/program_options/tutorial.html
and the example codes at:
http://www.boost.org/doc/libs/1_74_0/libs/program_options/example/

Copy your insertion sort program from Exercise 1.5 and modify it, using the Boost `program_options` library to allow the user to specify the size of the array, the numerical range of floating-point numbers with which to populate it and whether to sort ascending or descending (with appropriate defaults). You should then be able to run your program with command-line arguments, such as:

```
./insertion-sort
./insertion-sort --size=50
./insertion-sort --size=100 --min=0 --max=100 --descending
```

Note: Ensure your program accepts command-line arguments. The definition of the main function should now look like:

```
int main(int argc, char* argv[])
```

Exercise 2.4

Download the `signal.txt` file from Blackboard Learn. This is a superposition of sine waves of different frequencies and amplitudes polluted with Gaussian white noise. The sample-rate of the signal is 1kHz (i.e. 1000 consecutive samples in the file corresponds to 1 second).

1. Write a program which reads the `signal.txt` file and uses the FFTW library to compute the frequency power spectrum. For this you should use a 1D real-to-complex discrete Fourier transform (DFT).

   Some guidance on using FFTW is available at:
   http://www.fftw.org/doc/Tutorial.html#Tutorial

   Hints: For a signal with n samples, the transform will produce $(n/2 + 1)$ frequency bins. If the signal has a sample rate of r, the frequency bin size is given by r/n. Power is given by $p = 2|z_i|$, where z_i is the complex output.

2. Identify the dominant frequencies in the signal in order of power.

3. Reconstruct a filtered clean version of the signal without the noise using the complex-to-real DFT, plot it, and compare with the original signal.

Chapter 3

Debuggers

Objectives

In this chapter you will learn about:

- how to compile code for use with a debugger;
- breakpoints and how to step through your program;
- the backtrace, moving between frames and inspecting variables.

3.1 Bugs, Debugging and Debuggers

A *bug* is a colloquial term to describe when our computer program does not quite behave as expected. Usually these are problems which occur at runtime (i.e. the syntax of your code is valid such that it compiles) and bugs may occur during the normal use of your program, or only in certain edge cases.

Definition: Debugging

Debugging is the process of finding and fixing bugs.

A *debugger* is a tool to help with this process. Many bugs can be quite subtle and difficult to spot from merely looking at the code. Tracking and inspecting the *state* of the program as it executes can therefore be very valuable – this is what a debugger enables.

The GNU Debugger, `gdb`, can be attached to an executable when it is run to allow you to debug your code. If your program crashes, for example due to a segmentation fault, it will allow you to see exactly where the error occurred (assuming you compiled with debug symbols, see Section 2.4.4). You can then investigate the value of variables and examine the contents of arrays to identify the cause of the problem. The debugger also allows you to step through your code to identify the origins of the erroneous state.

3.2 Compiling and Running with GNU debugger

The simplest way to use a debugger is to compile your code with debug symbols and run the executable through gdb directly. For example,

```
g++ -g -o myprog file1.cpp file2.cpp
gdb ./myprog
```

This will launch the GDB "shell" through which you can issue commands; note the `(gdb)` to indicate we are in the GDB shell. We have not actually run our program yet, only instructed GDB which executable it is going to monitor. To run our code, we simply type `run` or `r`, followed by any arguments we would have otherwise given on the command-line, and we should see something like the following:

```
(gdb) run
Starting program: /tmp/myprog
[Inferior 1 (process 25918) exited normally]
```

In this case, the program completed successfully without error.

If an error occurs, information detailing where the error occurred will be printed, similar to the following example:

```
Program received signal SIGSEGV, Segmentation fault.
0x0000555555554631 in badfunc () at file1.cpp:4
4           x = *y;
(gdb)
```

Your code will be paused and you will be returned to the GDB shell.

3.3 Backtrace

A backtrace lists the functions in the order they were called to get to the current point in the execution of the code. We can get a backtrace using the `backtrace` and `bt` GDB commands:

```
(gdb) bt
#0  0x0000555555554631 in badfunc () at file1.cpp:4
#1  0x000055555555464f in main () at file1.cpp:10
```

The most recent (lowest level) function call is at the top and the earliest (highest level) function call is at the bottom. Typically, the last line should be the `main()` function. The backtrace includes the name of the function and information about the file and line number at which the error occurred.



Note

If you see question marks instead of file and line information, you have not compiled your code correctly with debugging symbols (`-g`).

3.4 Frame context

Each of the lines (function calls) in the backtrace is a *frame* and by default our debugging context is frame #0. If we think the source of our problem happens earlier in our program, we can change to

one of the other frames with the `frame` or simply `f` GDB command.

```
(gdb) frame 1
#1  0x000055555555464f in main () at file1.cpp:10
10          int z = badfunc();
(gdb)
```

GDB prints information on the new frame and the line of code.

3.5 Listing code

To see the surrounding lines of code where the error occurs, we can use the `list` GDB command.

Running `list` a second time will print the lines which follow those printed by the first `list` command. To see lines before the first `list` command, run `list -`.

A line number, or function name can also be given to specify the region of code to list.

3.6 Inspecting variables and arrays

To inspect the current value of variables in our code we use the `print`, or shorthand `p`, GDB command. For example

```
(gdb) p x
$1 = 1
```

tells us that the variable `x` has the value 1. For floating-point numbers, such as `double` variables, GDB will interpret the data type appropriately. You can also print values from an array using, for example, `print myarray[4]`. If an array was not allocated on the heap (i.e. using the `new` keyword), the whole array can be printed using `print myarray`. GDB can even interpret and print `struct`s.

It is often useful to examine memory at a lower level; for example, to examine whole arrays allocated on the heap. The GDB command `x` can be used to examine a block of memory. It takes two additional arguments: a format expression to tell it how to interpret the data in memory; and an address from where to start examining. For example:

```
(gdb) x/20dw var
```

would examine 20 decimal (integer) values from the pointer `var`. For details on the meaning of the format expressions, run the GDB command `help x`.

3.7 Breakpoints

Sometimes your program does not generate an error but just produces the wrong numbers. In this case, it can be useful to stop your program in the middle of execution to examine the results. This is achieved using breakpoints and they can be set using the `breakpoint` (or just `b`) GDB command,

```
(gdb) b file1.cpp:7
Breakpoint 1 at 0x555555554933: file file1.cpp, line 7.
```

The above example sets a breakpoint at line 7 of file1.cpp; GDB will now pause every time that line is encountered during the execution of the program, allowing you to examine the state of the program as above.

3.7.1 Advancing through a program

When paused, we can instruct GDB to evaluate the next statement, and then pause again, using the command `next`, or just `n`. If that statement involved calling a function, the entire function would be run. If instead, we wanted to *step into* the function, we can call the `step` command, or just `s`.

3.7.2 Continuing Execution

To continue execution as normal, following a breakpoint, use the GDB command `continue`, or `c`. Execution will stop again if this, or another, breakpoint is encountered (or an error occurs).

3.8 Getting help

To get help with GDB commands use the `help` command.

3.9 Exiting GDB

To exit GDB, simply type `quit`.

Exercises

Exercise 3.1

Download the file `dbg.cpp` from Blackboard Learn. This is a simple program which populates an $N \times N$ matrix with

$$M[i,j] = (e^x)^2/(e^y)^2$$

where $x = i/N$ and $y = j/N$. Compile and run the program. You should observe that the program fails with the error

```
Segmentation fault
```

Use the debugger `gdb` to investigate the cause of the error and fix the code.

Hint: Change the value of N in the code to make debugging easier.

Part II

Linear Algebra Libraries

Chapter 4

The Basic Linear Algebra Subroutines (BLAS)

The BLAS actually describes a specification for fundamental linear algrebra routines, rather than a specific implementation. There is a *reference* implementation which illustrates the expected behaviour of each function in the API, but many hardware vendors will provide their own (often *closed-source*) implementations to take advantage of their intimate knowledge of their hardware.

The reference BLAS implementation can be found on the Netlib website:
http://www.netlib.org/blas/

The term *Basic* refers to the scope of the linear algebra operations considered. BLAS includes core explicit operations on matrices and vectors, such as matrix-vector and matrix-matrix multiplications, but more advanced linear algebra operations, such as solving linear systems and computing eigenvalues, are instead provided by the Linear Algebra PACKage (LAPACK), which we explore in Chapter 5.

4.1 BLAS fundamentals

BLAS consists of three *levels*:

- Level 1 BLAS: vector-vector routines
- Level 2 BLAS: matrix-vector routines
- Level 3 BLAS: matrix-matrix routines

Until a decade or two ago, FORTRAN, developed in the 1950s, was the mainstay language of scientific programming as it was fast and better suited to manipulating arrays. Modern C++ and modern Fortran 90 are comparable on speed and the features of C++ provide arguably more flexibility in creating powerful robust engineering codes.

The *reference BLAS* implementation, along with many of the vendor implementations and other legacy numerical libraries, are actually still written in Fortran, rather than C or C++ to maintain compatibility with the many legacy scientific and engineering codes still in use. Therefore, we must briefly digress to understand how we can link to a Fortran 90 library.



Note

There is a C *wrapper* interface (also available from the Netlib URL above) which provides C functions that *wrap* the Fortran 90 calls, but this, understandably, uses a modified API and is not necessarily available on all HPC systems. However, either the main BLAS interface or the C wrapper may be used during this course.

4.2 Linking to BLAS

The implementation for the BLAS routines is not part of our code; it is a pre-compiled *external library*.

The file containing the actual object code for the BLAS routines is:

```
/usr/lib/libblas.so
```

To link to the BLAS library using `g++`, we use the `-l` linker option

```
# Standard practice, assumes libblas.so in library search path
g++ myprog.cpp -lblas
```

In CodeLite you must add the library to the project:

- Open the *Workspace* menu and select *Open Active Project Settings...*. Select *Linker* from the list on the left.
- In the *Libraries* field, enter `blas`. To add multiple libraries, separate their names with semi-colons.
- Click *Apply* and *OK* to return to the main window.
- When you build your project, you should see `-lblas` at the end of the g++ linker command in the *Build Log* window.

4.3 Using Fortran libraries in C++

As with any external library, in order to use one of the API functions we must tell the compiler the function prototype, so that it can keep track of what arguments it should expect from our code and their data types. This is often provided in a C/C++ header file which we can just `#include` in our code. However, for libraries written in Fortran, we need to manually provide these declarations.

Suppose we have the Fortran 90 function

```
DOUBLE PRECISION FUNCTION DDOT(N,DX,INCX,DY,INCY)
    ! Integer scalar arguments and double precision arrays
    INTEGER INCX,INCY,N
    DOUBLE PRECISION DX(*),DY(*)
```

This is a BLAS function which computes the dot product of two vectors. It takes five parameters, whose data types are integer or double-precision floating-point numbers.

The way Fortran and C++ compilers generate compiled forms of a function differ, so in order to use a Fortran code, we must tell the C++ compiler to look for the compiled symbol with a slightly different name than what it would otherwise expect. There are two main changes:

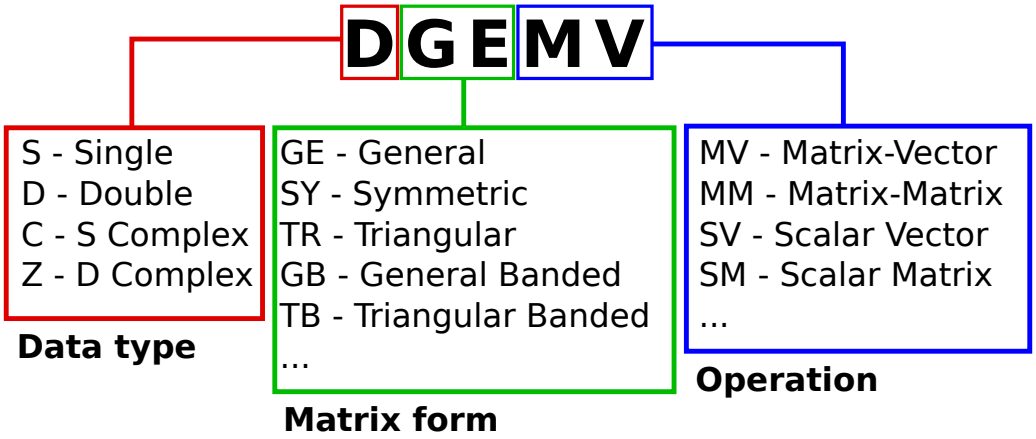


Figure 4.1: Naming structure of BLAS routines.

- Fortran appends the function name with an underscore (_), whereas C/C++ compilers do not.
- Since C++ supports function overloading, it "mangles" the function names with the parameter datatypes to ensure they are always unique.

In order to call Fortran 90 functions from our C++ code, we therefore need to add the underscore and turn off the name mangling. The corresponding C function prototype would therefore be

```
// A convenient macro to fix differences in symbol names between C and Fortran
#define F77NAME(x) x##_

// Function prototypes for the BLAS functions we will use
// Tell the compiler to use "C"-style symbol names (without name mangling)
extern "C" {
    double F77NAME(ddot) (const int& n,
                          const double *x, const int& incx,
                          const double *y, const int& incy);
}
```

We have used a preprocessor `#define` for adding the underscore simply to make clear to people reading our code that we are calling a Fortran function. To turn off the name-mangling we use `extern "C"`, which tells the compiler to use C naming (C does not support function overloading).

We can then call this function from our C code using

```
double a = F77NAME(ddot)(n, input1, 1, input2, 1);
```

4.4 BLAS Routine Naming

BLAS provides an API which follows a concise logical naming structure that describes the particular linear algebra operation performed by the routine. This is illustrated in Figure 4.1. The first character represents the data type of the numbers in the matrices and vectors; the second two letters represent the structure of the matrix (for Level 2 and Level 3 routines); and the last letters represent the operation to be performed.

The routine shown in the example, `dgemv`, therefore performs a general (full) matrix-vector multiplication where all matrix and vector entries are double-precision floating-point numbers.

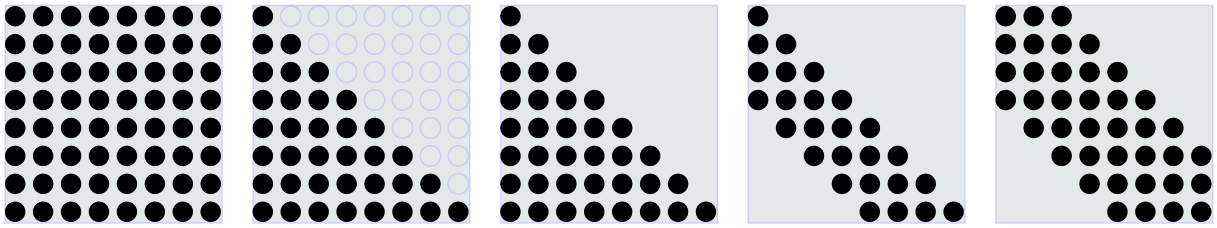


Figure 4.2: Some common forms used in BLAS for real-valued matrices. From left-to-right: General matrix, symmetric matrix, triangular packed matrix, triangular banded matrix, general banded matrix

4.5 Matrix forms

To improve the computational performance of linear algebra operations and minimise memory usage, different matrix *forms* can be used which take advantage of structure in the matrix. Generally we assume the notation a_{ij} refers to the matrix element on row i and in column j.

4.5.1 General

Use this to represent a fully dense matrix in which all $M \times N$ entries, a_{ij}, are potentially non-zero.

4.5.2 Symmetric

Symmetric matrices are those which satisfy $a_{ij} = a_{ji}$. It is also assumed that $M = N$.

4.5.3 Triangular

Triangular matrices may be *upper triangular* or *lower triangular*, which is typically indicated to BLAS routines which accept such matrices by the parameter `UPLO`. For a lower triangular matrix, it is assumed $a_{ij} = 0$ for $i < j$. Similarly, for an upper triangular matrix, it is assumed $a_{ij} = 0$ for $i > j$.

4.6 Matrix storage schemes

While the form of a matrix describes which elements of the matrix may be zero or non-zero, the storage scheme defines how the non-zero data is arranged in memory. Some of the matrix form and storage scheme combinations known to BLAS are illustrated in Figure 4.2. A fundamental aspect to these schemes is whether to store data by-row or by-column, which is discussed below.

4.6.1 Column-major vs Row-major

Memory in a computer is addressed in a linear fashion. There are therefore two ways in which we can represent a two-dimensional matrix in memory. We can either store the matrix in *row-major* format, or in *column-major* format.

In row-major format, elements along the same row will be arranged adjacent in memory:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \quad \text{stored as} \quad [\underbrace{a_{11}\, a_{12}\, a_{13}}_{\text{Row 1}}\ \underbrace{a_{21}\, a_{22}\, a_{23}}_{\text{Row 2}}].$$

This is conventionally the format we use when programming in C or C++.

In column-major format, elements down the same column will be arranged adjacent in memory:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \quad \text{stored as} \quad [\underbrace{a_{11}\, a_{21}}_{\text{Col. 1}}\ \underbrace{a_{12}\, a_{22}}_{\text{Col. 2}}\ \underbrace{a_{13}\, a_{23}}_{\text{Col. 3}}].$$

This convention is adopted by Fortran codes. Therefore, if we are directly calling the Fortran interface, they expect matrices to be laid out in memory using the column-major format. If we use the CBLAS wrappers (see Section 4.10, we can choose to use row- or column-major formats.

4.6.2 Conventional storage

Conventional storage assumes memory is allocated for all $M \times N$ entries of the matrix. This is the simplest storage format to understand. In the case of triangular matrices, for example, we would need to allocate a full matrix, but are safe in the knowledge that only the triangular elements (upper or lower) will actually be accessed.

4.6.3 Packed storage

Packed storage can be used with symmetric or triangular matrices where the non-zero data is stored as follows:

$$\begin{pmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ a_{41} & a_{42} & a_{43} & a_{44} & \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} \end{pmatrix}$$

would be stored in memory as:

$$[\underbrace{a_{11}\, a_{21}\, a_{31}\, a_{41}\, a_{51}}_{\text{Column 1}}\ \underbrace{a_{22}\, a_{32}\, a_{42}\, a_{52}}_{\text{Column 2}}\ \underbrace{a_{33}\, a_{43}\, a_{53}}_{\text{Column 3}}\ \underbrace{a_{44}\, a_{54}}_{\text{Col. 4}}\ \underbrace{a_{55}}_{\text{Col. 5}}]$$

4.6.4 Banded storage

Banded storage assumes the diagonal, first K_L lower off-diagonals and first K_U upper off-diagonals contain non-zero entries. The data is organised in a $(K_L + K_U + 1) \times N$ matrix, where each row of this matrix corresponds to a diagonal of the original matrix. A general banded matrix (in column-major format), in which $K_L > 0$ and $K_U > 0$, is therefore represented as follows:

$$\begin{pmatrix} \mathbf{a_{11}} & a_{12} & & & \\ a_{21} & \mathbf{a_{22}} & a_{23} & & \\ a_{31} & a_{32} & \mathbf{a_{33}} & a_{34} & \\ & a_{42} & a_{43} & \mathbf{a_{44}} & a_{45} \\ & & a_{53} & a_{54} & \mathbf{a_{55}} \end{pmatrix} \rightarrow \begin{pmatrix} * & a_{12} & a_{23} & a_{34} & a_{45} \\ \mathbf{a_{11}} & \mathbf{a_{22}} & \mathbf{a_{33}} & \mathbf{a_{44}} & \mathbf{a_{55}} \\ a_{21} & a_{32} & a_{43} & a_{54} & * \\ a_{31} & a_{42} & a_{53} & * & * \end{pmatrix} \begin{matrix} \\ \leftarrow \text{Upper diagonal} \\ \leftarrow \text{Diagonal} \\ \leftarrow \text{First lower diag.} \\ \leftarrow \text{Second lower diag.} \end{matrix}$$

resulting in the ordering in memory:

$$[\underbrace{* \, \mathbf{a_{11}}\, a_{21}\, a_{31}}_{\text{Column 1}}\ \underbrace{a_{12}\, \mathbf{a_{22}}\, a_{32}\, a_{42}}_{\text{Column 2}}\ \underbrace{a_{23}\, \mathbf{a_{33}}\, a_{43}\, a_{53}}_{\text{Column 3}}\ \underbrace{a_{34}\, \mathbf{a_{44}}\, a_{54}\, *}_{\text{Column 4}}\ \underbrace{a_{45}\, \mathbf{a_{55}}\, *\, *}_{\text{Column 5}}]$$

The $*$ symbols denote array elements which are allocated, but not accessed by the BLAS routine.

4.7 Examples

To give a practical introduction to the use of the BLAS, we illustrate it with some examples.

4.7.1 BLAS Level 1

Consider the dot product of two vectors $\mathbf{x}$ and $\mathbf{y}$:

$$z \leftarrow \mathbf{x}\mathbf{y}^\top.$$

This can be performed using the BLAS routine `ddot`. To call this routine through the standard Fortran interface, we need the corresponding function prototype to be defined at the top of our source code:

```
double F77NAME(ddot) (const int& n,
                      const double *x, const int& incx,
                      const double *y, const int& incy);
```

This routine requires us to specify 5 parameters:

- `n` specifies the number of elements in the vector
- `x` and `y` are pointers to arrays of doubles specifying the input vectors
- `incx` and `incy` specify the *stride* through the vectors `x` and `y`.

The result z, a scalar quantity, is provided as the return value.

Definition: Definition: Stride

Stride is the increment $s > 0$ of the index of an array, used when performing vector operations.

Stride is typically 1, but may be greater in some cases, such as when performing a vector operation on a row of a matrix that is stored in column-major format.

A complete example of using the `ddot` routine is given in Example 4.1.

4.7.2 BLAS Level 2

The multiplication of a general full matrix and a vector can be performed using the level 2 BLAS routine `dgemv`. This actually performs the operation:

$$y \leftarrow \alpha A x + \beta y.$$

The BLAS routine prototype is given by

```
void F77NAME(dgemv) (const char&   trans, const int&    m,
                     const int&    n,     const double& alpha,
                     const double* A,     const int&    lda,
                     const double* x,     const int&    incx,
                     const double& beta,  double* y,
                     const int&    incy);
```

The first thing to note is that the routine actually performs the matrix-vector product of A and x, **plus** a scalar multiple of y. Therefore, we must pay close attention to the contents of our `y` array before entering the routine, or ensure `beta` is set to zero.

Most of the parameters a reasonably intuitive. Recall that the matrix, given by the double-precision array `A` should be stored as a full matrix in column-major ordering. The `trans` parameter is a single character which indicates whether to use A as given (`'N'`), or to operate with the transposition of A instead (`'T'`). The `LDA` parameter describes the leading dimension of the matrix.

Example 4.1: Using the `ddot` routine in BLAS

```
#include <iostream>
#include <cstdlib>
using namespace std;

#define F77NAME(x) x##_
extern "C" {
    double F77NAME(ddot) (const int& n,
                          const double *x, const int& incx,
                          const double *y, const int& incy);
}

int main() {
    const int N = 64;                       // Size of vectors
    double* x = new double[N];              // Allocate vectors
    double* y = new double[N];
    double z = 0.0;

    srand(time(0));                         // Initialise random numbers
    for (int i = 0; i < N; ++i) {           // Populate vectors
        x[i] = (double)(rand()) / RAND_MAX; //   with numbers in [0,1]
        y[i] = (double)(rand()) / RAND_MAX;
    }

    z = F77NAME(ddot)(N, x, 1, y, 1);       // Compute dot-product using BLAS

    cout << "Dot product result: " << z << endl;

    delete[] x;                             // Deallocate memory!
    delete[] y;
}
```

Definition: Leading dimension

For a matrix, represented by a linear array, the leading dimension is the length of the fastest-changing index.

In contrast, `m` specifies the number of rows, and `n` the number of columns, on which we want to operate. Together these parameter allows us to operate on sub-matrices by choosing `m < lda` (see Section 4.8).

A complete example using `dgemv` is given in Example 4.2.

4.7.3 BLAS Level 3

The general matrix-matrix multiply operation is given by

$$C \leftarrow \alpha AB + \beta C$$

and the corresponding BLAS routine prototype by:

```
void F77NAME(dgemm) (
         const char&   trans, const char&   transb,
         const int&    m,     const int&    n,
         const int&    k,     const double& alpha,
         const double* A,     const int&    lda,
         const double* B,     const int&    ldb,
```

Example 4.2: A complete example using the level 2 BLAS routine `dgemv`. Try adding code to print out the matrix and vectors and verify it is correct.

```
#include <cstdlib>
using namespace std;

#define F77NAME(x) x##_
extern "C" {
    void F77NAME(dgemv) (const char&   trans, const int&    m,
                         const int&    n,     const double& alpha,
                         const double* A,     const int&    lda,
                         const double* x,     const int&    incx,
                         const double& beta,  double* y,
                         const int&    incy);
}

int main() {
    const int N = 8;                            // Number of columns
    const int M = 6;                            // Number of rows
    double* A = new double[N*M];                // Matrix allocation
    double* x = new double[N];                  // Input/output vectors
    double* y = new double[M];

    for (int i = 0; i < N; ++i) {               // Populate matrix and vector
        for (int j = 0; j < M; ++j) {
            A[i*M+j] = double(rand())/RAND_MAX;
        }
        x[i] = double(rand())/RAND_MAX;
    }

    // Perform matrix-vector product.
    F77NAME(dgemv)('N', M, N, 1.0, A, M, x, 1, 0.0, y, 1);
}
```

```
          const double& beta,  double*       C,
          const int&    ldc);
```

Again, take care to set `beta` to zero, or the matrix elements of `C` to zero if only the product of `A` and `B` is required.

For each matrix we must specify a leading dimension. Note that since the number of columns of A must equal the number of rows of B, we are only asked to provide three matrix dimensions, illustrated in Figure 4.3.

4.8 Operating on Sub-matrices

BLAS routines are designed to be sufficiently flexible to operate on sub-matrices. This is illustrated in Figure 4.4. Here the full matrix has M rows and N columns, but we wish to operate on a sub-matrix of size m rows and N columns with a starting offset of q rows and p columns.

We do the following:

- Set the `lda` parameter to M as this describes how long a column is *in memory*. Equivalently, this is the number of elements in memory we must move in order to get to the adjacent element along a row.
- Use the values of m and n to specify the size of the sub-matrix.

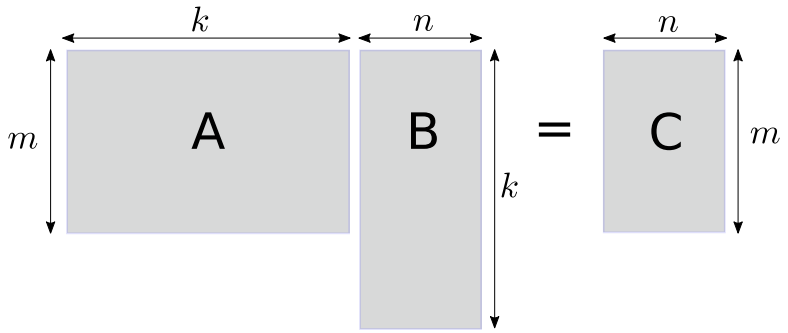


Figure 4.3: Diagram illustrating the dimension parameters for a `dgemm` matrix-matrix multiplication.

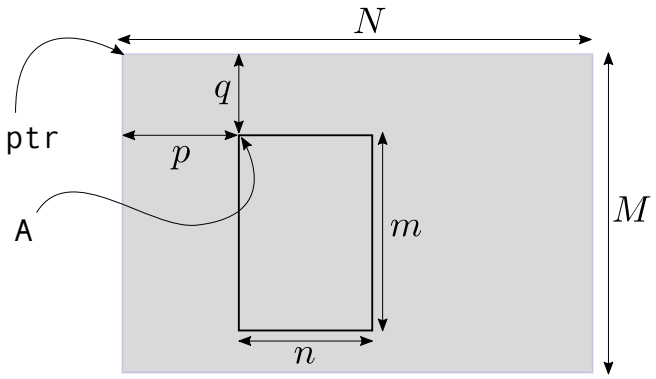


Figure 4.4: Operating on sub-matrices with BLAS routines.

- Use pointer arithmetic to provide the starting point of the *sub-matrix*, in this case:

```
A = ptr + (p * M) + q
```

Notice that we do not need to specify N, although BLAS is assuming that $p + n < N$.

4.9 Learning about BLAS routines

The BLAS API is very well documented. For a list of the available routines and full details on the operation and expected input/output for each routine, consult the documentation at http://www.netlib.org/blas/index.html.

A quick reference guide is also available to download from Blackboard Learn.

4.10 CBLAS Wrapper for BLAS

So far we have directly called the Fortran functions which implement the BLAS routines. This has required us to

- Provide our own routine declarations at the top of our source file.
- Ensure the symbol names expected by the compiler match those in the BLAS library, for example with the following,

```
#define F77NAME(x) x##_
extern "C" {
    double F77NAME(dnrm2) (
        const int& n, const double *x, const int& incx);
}
```

- Ensure our matrix data is organised in column-major ordering.

As a convenience for C/C++ programmers, there are CBLAS wrappers which make calling the BLAS library routines similar to calling native C++ functions. These wrappers are defined in the header file `cblas.h`, which needs to be included at the top of your code. You can then call a BLAS routine by prefixing its name with `cblas_`:

```
double cblas_dnrm2(const int N, const double *X, const int incX);
```



Note

In addition to avoiding the need to define the BLAS function prototypes, the CBLAS wrappers also allow us to choose to store our matrices in row-major ordering. While this in isolation is not particularly advantageous, it may simplify inter-operation with other C++ libraries which expect matrices to be provided in row-major format.

However, this does result in the parameter lists of CBLAS wrapper routines not exactly matching those of the reference BLAS implementation.

To find out more about the CBLAS wrappers and the parameters required for each CBLAS wrapper routine, open the header file

```
/usr/include/cblas.h
```

Example 4.3 shows the same code as Example 4.2 using the CBLAS wrappers. Note the presence of an additional parameter to specify the layout at the beginning of the parameter list and the use of C/C++ *enums* (which simply associated labels to integer values) to specify the layout and transposition flags.

Example 4.3: Using the CBLAS wrapper for calling `dgemv`.

```
1  #include <cstdlib>
2  using namespace std;
3
4  #include <cblas.h>
5
6  int main() {
7      const int N = 8;     // Columns
8      const int M = 6;     // Rows
9      double* A = new double[N*M];
10     double* x = new double[N];
11     double* y = new double[M];
12
13     for (int i = 0; i < N; ++i) {
14         for (int j = 0; j < M; ++j) {
15             A[i*M+j] = double(rand())/RAND_MAX;
16         }
17         x[i] = double(rand())/RAND_MAX;
18     }
19
20     cblas_dgemv(CblasColMajor, CblasNoTrans, M, N, 1.0, A, M, x, 1, 0.0, y, 1);
21 }
```

4.11 Other BLAS implementations

The BLAS we are using in this course is the free reference implementation of BLAS. Other BLAS implementations exist which may offer improved performance for some applications and on certain system architectures. These include ATLAS, OpenBLAS, MKL and GotoBLAS.

Exercises

Exercise 4.1

The aim of this exercise is to learn the basics of using the BLAS library.

Recall that full documentation for the BLAS interface can be found at `http://www.netlib.org/blas` and a quick reference guide to the available routines is available to download from Blackboard Learn.

1. Create a new program which allocates an array of N double-precision floating-point numbers, x_i, and populates them with pseudo-random numbers between 0 and 1. Choose N small to start with.
2. Using BLAS routines, compute the three quantities

$$d = \sqrt{\left(\sum_{i=1}^{N} x_i^2\right)} \quad \text{and} \quad s = \sum_{i=1}^{N} |x_i| \quad \text{and} \quad m = \max_i |x_i|$$

3. Write a function which manually computes these quantities and verify you have used the BLAS correctly.

Exercise 4.2

Write a program which uses BLAS to compute the product, C_1, of two matrices A and B of size $N \times N$, where $N = 2^n$, for some integer $n > 0$. Write an additional function which performs the calculation manually, storing in matrix C_2.

Calculate the difference, using BLAS, between C_1 and C_2 to verify your solutions match:

$$\epsilon = \sum_{i=1}^{N} \sum_{j=1}^{N} |C_1[i][j] - C_2[i][j]|$$

Exercise 4.3

The conjugate gradient method is an iterative algorithm for solving a system of linear equations of the form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is a symmetric positive-definite matrix. It proceeds as follows:

$\mathbf{r_0} := \mathbf{b} - \mathbf{Ax_0}$
$\mathbf{p_0} := \mathbf{r_0}$
$k := 0$
repeat
  $\alpha_k := \dfrac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A}\mathbf{p}_k}$
  $\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$
  $\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k$
  if $|\mathbf{r}_{k+1}| < \epsilon$, break
  $\beta_k := \dfrac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$
  $\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$
  $k := k + 1$
end repeat
return $\mathbf{x}_{k+1}$

1. Write a function which generates an $n \times n$ matrix $\mathbf{M}$ of random double-precision floating-point numbers with $-1 < m_{ij} < 1$. Write another function which generates a random vector $\mathbf{x}$ of length n, with $-1 < x_i < 1$.
2. Calculate the symmetric positive-definite matrix $\mathbf{A} = \mathbf{M}^\top \mathbf{M}$.
3. Compute the matrix-vector product $\mathbf{b} = \mathbf{Ax}$.
4. Write a function which implements the conjugate gradient algorithm *using BLAS routines*.
5. Now solve the linear system $\mathbf{Ay} = \mathbf{b}$, for $\mathbf{y}$ using your implementation and verify that the resulting vector $\mathbf{y}$ matches the original vector $\mathbf{x}$.
6. Optimise your code to take advantage of the symmetric nature of the matrix.

Exercise 4.4

The Helmholtz equation for u in one-dimension on the interval $[a, b]$ is given by

$$\frac{\partial^2 u}{\partial x^2} - \lambda u = f$$

for a given forcing function f and with boundary conditions $u(a) = u_a$ and $u(b) = u_b$. This equation can be discretised using finite differences as

$$\frac{u(x_{i-1}) - 2u(x_i) + u(x_{u+1})}{(\Delta x)^2} - \lambda u(x_i) = f(x_i),$$

where Δx is the distance between adjacent grid points. On a grid of n grid points, this gives

rise to a matrix equation of n equations of the form

$$\begin{pmatrix} 1 & 0 & 0 & & & & 0 \\ \beta & \alpha & \beta & 0 & & & \\ 0 & \beta & \alpha & \beta & 0 & & \\ & \ddots & \ddots & \ddots & \ddots & \ddots & \\ & & 0 & \beta & \alpha & \beta & 0 \\ & & & 0 & \beta & \alpha & \beta \\ 0 & & & & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{n-3} \\ u_{n-2} \\ u_{n-1} \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n-3} \\ f_{n-2} \\ f_{n-1} \end{pmatrix},$$

where $u_i = u(x_i)$, $f_i = f(x_i)$, $\alpha = -\frac{2}{(\Delta x)^2} - \lambda$ and $\beta = \frac{1}{(\Delta x)^2}$. The first and last rows correspond to the ends (boundaries) of the domain and in this case are set to a fixed value. In order to enforce these Dirichlet boundary conditions at $x = a$ and $x = b$, we eliminate the first and last rows/columns from the system of equations, adding the missing contributions from the second and second-to-last rows to the right-hand side, to create the reduced matrix system:

$$\begin{pmatrix} \alpha & \beta & 0 & & 0 \\ \beta & \alpha & \beta & 0 & \\ \ddots & \ddots & \ddots & \ddots & \ddots \\ & 0 & \beta & \alpha & \beta \\ 0 & & 0 & \beta & \alpha \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-3} \\ u_{n-2} \end{pmatrix} = \begin{pmatrix} f_1 - u_0/(\Delta x)^2 \\ f_2 \\ \vdots \\ f_{n-3} \\ f_{n-2} - u_{n-1}/(\Delta x)^2 \end{pmatrix}.$$

1. Write a function which generates the reduced matrix system above, using *symmetric* matrix storage, for $n = 21$ and domain $[a, b] = [0, 2]$. Assume $\lambda = 1$.

2. Write a function which calculates the forcing function

$$f = -(\lambda + \pi^2)\sin(\pi x)$$

on the grid points. The boundary conditions for this problem are $u(0) = u(2) = 0$.

3. Use the conjugate gradient implementation from Exercise 4.3 to solve the system of equations for u. Verify your solution matches the analytical solution of $u = \sin(\pi x)$.

4. Modify your code to solve for the forcing function

$$f = -(\lambda + \pi^2)\cos(\pi x)$$

on the domain $[a, b] = [0, 3]$ with boundary conditions $u(0) = 1$, $u(3) = -1$ and verify your solution matches the analytical solution of $u = \cos(\pi x)$.

Exercise 4.5

Further optimise the memory and computational efficiency of your solution to Exercise 4.4 by using banded storage and calling appropriate BLAS routines.

Exercise 4.6

The transient heat equation in one dimension is given by the following partial differential equation (PDE) and boundary conditions:

$$\begin{aligned} \frac{\partial u}{\partial t} &= \frac{\partial^2 u}{\partial x^2} && \text{on } \Omega \\ u(0,t) &= \gamma_0(t) && \text{on } \partial\Omega, \\ u(1,t) &= \gamma_1(t) && \text{on } \partial\Omega \\ u(x,0) &= u_0(x) && \text{initial condition.} \end{aligned}$$

Consider the domain Ω to be the interval $0 \leq x \leq 1$. This can be discretised in space and time using finite differences as

$$\frac{U_i^{k+1} - U_i^k}{\Delta t} = \alpha \frac{U_{i-1}^k - 2U_i^k + U_{i+1}^k}{(\Delta x)^2},$$

where k is the time-step and i is the index of the N_x grid-points used to discretise the solution in space. U is therefore the solution at the points x_i.

This can be rearranged to give:

$$U_i^{k+1} = \nu U_{i-1}^k + (1-2\nu)U_i^k + \nu U_{i+1}^k, \qquad \nu = \frac{\Delta t}{(\Delta x)^2} < 0.5.$$

or, equivalently, as a matrix equation: $\mathbf{U}^{k+1} = \mathbf{A}\mathbf{U}^k$. We use Forward Euler time-integration, which is *conditionally stable*.

Implement the following:

1. Prompt the user for the number of grid points to use, the time-step size and the number of time steps.
2. Write a function which populates the (tridiagonal) matrix **A**. Recall from Ex. 4.4 how to handle boundary conditions.
3. Generate the vector $\mathbf{U}$ at time zero by discretising the initial condition $u(x,0) = \sin(\pi x)$, $\gamma_0 = \gamma_1 = 0$.
4. Repeatedly perform the matrix-vector operation above to time-integrate the PDE and write out the final solution to file.

   Test your code using $N_x = 20$, $\Delta t = 0.001$ and $N_t = 100$. The initial sine function should decay in amplitude.

Exercise 4.7

(Advanced) Implement the matrix-matrix product using the BLAS and the recursive approach of Exercise 1.4, taking advantage of the ability of the BLAS routines to operate on sub-matrices.

Hint: Take care when specifying array offsets, the sub-matrix dimensions and `LDA`, `LDB` and `LDC`. Remember, these *leading dimension* arguments relate to the layout of the original complete matrix in memory, not the sub-matrix being operated on.

Chapter 5

Linear Algebra PACKage (LAPACK)

5.1 Overview of LAPACK

LAPACK is library which builds upon the basic operations of BLAS, containing routines for more advanced linear algebra operations such as:

- solving systems of simultaneous linear equations;
- least-squares solutions of systems of linear equations;
- eigenvalue problems;
- and singular value problems.

The naming scheme of the routines closely follows that of BLAS, and full documentation is available at http://www.netlib.org/lapack/explore-html



Note
While there is a library called CLAPACK, this is not a wrapper but is a complete reimplementation of LAPACK written in C and is not commonly available in HPC systems.

5.2 Linking to LAPACK

Since LAPACK uses BLAS routines, we need to link to the BLAS library, even if we do not explicitly use a BLAS routine in our program. In order for the linker to find the BLAS symbols required by the LAPACK routines, we must put the linker flag for BLAS, `-lblas`, **after** the linker flag for LAPACK, `-llapack`. For example,

```
g++ myprog.cpp -llapack -lblas
```

5.3 Solving a general linear system of equations

To solve a general system of linear equations, $\mathbf{AX} = \mathbf{B}$ one can use the LAPACK function `dgesv`. We allow for $\mathbf{X}$ and $\mathbf{B}$ to be matrices, thereby solving for multiple right-hand sides in one call. The prototype for the `dgesv` routine is:

```
#define F77NAME(x) x##_
extern "C" {
    void F77NAME(dgesv)(const int& n, const int& nrhs, const double * A,
                        const int& lda, int * ipiv, double * B,
                        const int& ldb, int& info);
}
```

The approach used to solve the system is to compute the LU decomposition of $\mathbf{A}$ with partial pivoting and then solve for $\mathbf{X}$. Many of the parameters are familiar from the BLAS. Here, `n` is the rank of the system, which must necessarily be square to avoid an under- or over-determined system. `nrhs`, specifies the number of columns of $\mathbf{B}$ and $\mathbf{X}$.

This routine actually computes an LU decomposition of $\mathbf{A} = \mathbf{PLU}$, where $\mathbf{P}$ is a permutation matrix arising from pivoting. $\mathbf{P}$ is captured by an array of pivot indices, for which we must provide the routine with an empty integer array, `ipiv`, of size `n`.



Warning

Many LAPACK routines overwrite their input variables during execution. For example, on exit from the `dgesv` routine, the input matrix referenced by the pointer `A` has been overwritten with the factors L and U from the LU-decomposition of $A = PLU$.

A further difference with BLAS is the inclusion of an `info` parameter which provides information on whether the operation was successful, or the cause of any error which occurred. For example, in `dgesv`, this value may be:

- 0 if the operation was successful.
- $-k$ if the k-th parameter was invalid.
- k if $U(i,i) = 0$ and therefore U is singular.

A complete example code using the LAPACK routine `dgesv` is given in Example 5.1.

5.3.1 Improving solve performance

Storage formats

LAPACK also has routines for solving matrix systems where the matrix is stored in one of the more compact storage formats such as packed or banded storage (see Section 4.6). The code excerpt in Example 5.2 shows its usage.

Of particular note is the size of the matrix A. Unlike the BLAS routine `dgbmv`, the size of the matrix for `dgbsv` must be $(1 + 2K_L + K_U) \times N$. The matrix storage then looks as follows:

$$\begin{pmatrix} \mathbf{a_{11}} & a_{12} & & & \\ a_{21} & \mathbf{a_{22}} & a_{23} & & \\ a_{31} & a_{32} & \mathbf{a_{33}} & a_{34} & \\ & a_{42} & a_{43} & \mathbf{a_{44}} & a_{45} \\ & & a_{53} & a_{54} & \mathbf{a_{55}} \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & a_{12} & a_{23} & a_{34} & a_{45} \\ \mathbf{a_{11}} & \mathbf{a_{22}} & \mathbf{a_{33}} & \mathbf{a_{44}} & \mathbf{a_{55}} \\ a_{21} & a_{32} & a_{43} & a_{54} & * \\ a_{31} & a_{42} & a_{53} & * & * \end{pmatrix} \begin{matrix} \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \leftarrow K_L \text{ padding rows} \\ \\ \leftarrow \text{Upper diagonal} \\ \leftarrow \text{Diagonal} \\ \leftarrow \text{First lower diag.} \\ \leftarrow \text{Second lower diag.} \end{matrix}$$

resulting in the linear storage

$$\underbrace{* \; * \; * \; \mathbf{a_{11}} \; a_{21} \; a_{31}}_{\text{Column 1}} \; \underbrace{* \; * \; a_{12} \; \mathbf{a_{22}} \; a_{32} \; a_{42}}_{\text{Column 2}} \; \underbrace{* \; * \; a_{23} \; \mathbf{a_{33}} \; a_{43} \; a_{53}}_{\text{Column 3}} \; \underbrace{* \; * \; a_{34} \; \mathbf{a_{44}} \; a_{54}}_{\text{Column 4}} \; \underbrace{* \; * \; a_{45} \; \mathbf{a_{55}} \; * \; *}_{\text{Column 5}}$$

Example 5.1: Using `dgesv` to solve a linear system $\mathbf{Ax} = \mathbf{y}$.

```
#include <iostream>
#include <cstdlib>
using namespace std;

#define F77NAME(x) x##_
extern "C" {
    // LAPACK routine for solving systems of linear equations
    void F77NAME(dgesv)(const int& n, const int& nrhs, const double * A,
                        const int& lda, int * ipiv, double * B,
                        const int& ldb, int& info);
    // BLAS routine for performing matrix-vector multiplication
    void F77NAME(dgemv) (const char&   trans, const int&    m,
                        const int&    n,     const double& alpha,
                        const double* A,     const int&    lda,
                        const double* x,     const int&    incx,
                        const double& beta,  double* y,
                        const int&    incy);
}

int main() {
    const int n    = 10;                    // Define system size
    const int nrhs = 1;
          int info = 0;
    double*   A    = new double[n*n];        // Allocate matrix and vectors
    double*   x    = new double[n];
    double*   y    = new double[n];
    int*      ipiv = new int[n];             // Vector for pivots

    // Populate A and x with random numbers
    srand(time(0));
    for (int i = 0; i < n*n; ++i) A[i] = (double)(rand())/RAND_MAX;
    for (int i = 0; i < n;   ++i) x[i] = (double)(rand())/RAND_MAX;

    // Compute product y
    F77NAME(dgemv)('N', n, n, 1.0, A, n, x, 1, 0.0, y, 1);

    // Solve system in-place (y is RHS on input and solution on output)
    F77NAME(dgesv)(n, nrhs, A, n, ipiv, y, n, info);

    // Print out result and compare with original input
    for (int i = 0; i < n; ++i) cout << x[i] << ", " << y[i] << endl;
}
```

The K_L padding rows can be unset on input. The banded matrix should be stored starting at position `A + kl * n`. After executing this routine, the first $1 + K_L + K_U$ rows contain the factorisation of the banded matrix, while the last K_L rows contain multipliers used during factorisation.

Pre-factoring matrices

The LAPACK function `dgbsv` solves the problem by first factorising the matrix using LU decomposition and then solving the resulting system.

If the system of equations is to be solved repeatedly, the matrix factorisation need only be performed once. This is achieved by first calling the LAPACK function `dgbtrf`:

```
void F77NAME(dgbtrf) (const int& m, const int& n, const int& kl,
```

Example 5.2: Code excerpt for solving a matrix system stored using the banded format with 1 lower diagonal and 2 upper diagonals.

```
int n     = 50;           // Problem size
int kl    = 1;            // Lower diagonal bandwidth
int ku    = 2;            // Upper diagonal bandwidth
int nrhs  = 1;            // Number of RHS vectors
int ldab  = 1 + 2*kl + ku;// Number of rows in compressed matrix
int ldb   = n;            // Size of RHS vector
int info;
double* A = new double[ldab*n];  // Banded matrix + 'fill' row
int* piv  = new int[n];          // Pivot data
double* b = new double[n];       // RHS vector / output vector

// ... Populate matrix A in banded format and vector b here

// Solve banded matrix system Ax=b.
// On input, b contains RHS vector. On output, b contains solution.
F77NAME(dgbsv) (n, kl, ku, nrhs, A, ldab, piv, b, ldb, info);
if (info) {
    cout << "ERROR: An error occurred in DGBSV: " << info << endl;
}
```

```
        const int& ku, double* A, const int& lda,
        int* ipiv, int& info);
```

which performs an LU factorisation of the matrix, storing the result in `A`. Once factored the system can be solved quickly for any given solution using the routine

```
void F77NAME(dgbtrs) (const char& trans, const int& n, const int& kl,
        const int &ku, const int& nrhs,   const double* A,
        const int& lda, const int* ipiv, double* b,
        const int& ldb, int& info);
```

5.4 Finding Eigenvalues and Eigenvectors

LAPACK includes routines for finding matrix eigenvalues/eigenvectors. Symmetric matrices have real eigenvalues and can be found using the `dsyev` routine, which has prototype

```
void F77NAME(dsyev)(const char& v, const char& ul, const int& n,
                    double* a, const int& lda, double* w,
                    double* work, const int& lwork, int* info );
```

A complete example of using the `dsyev` routine is given in Example 5.3.

The parameter `v` indicates if eigenvectors should be computed (`'V'`), or not (`'N'`). If computed, these are stored in `A`. In either case, the original matrix is overwritten. Since the matrix is assumed symmetric only the upper or lower triangle need be provided; this is indicated with the `ul` parameter using `'U'` or `'L'`, respectively. The array `w` is used to output the computed eigenvalues.

The parameters `work` and `lwork` provide the LAPACK routine with some temporary memory, called a *workspace*.

5.4.1 LAPACK Workspaces

Finding eigenvalues and eigenvectors requires *workspace* - an area of memory for holding intermediate temporary data produced during the course of the algorithm. Workspaces are required

by many of the algorithms in LAPACK. The size of the workspace is specified using a parameter `lwork`, while the memory itself is referenced through the pointer `work`.

The amount required depends on the routine and the provided parameters. For `dsyev`, it depends on whether eigenvectors are requested. Instead of trying to compute the necessary size *a priori*, we can call `dsyev` with `lwork=-1` and a workspace `work` of size 1. The routine will only calculate the optimal workspace size and put this (integer) into `work[0]`:

```
double wkopt;
int lwork = -1;
F77NAME(dsyev)('V', 'U', n, A, lda, w, &wkopt, lwork, &info);
```

We can typecast this value to an integer and make a second call to `dsyev` with the correctly allocated workspace to perform the actual eigen computation.

```
lwork = (int)wkopt;
work = new double[lwork];
F77NAME(dsyev)('V', 'U', n, A, lda, w, work, lwork, &info);
```

Example 5.3: Eigenvalues/eigenvectors of symmetric matrix

```cpp
#include <iostream>
#include <cstdlib>
using namespace std;

#define F77NAME(x) x##_
extern "C" {
    void F77NAME(dsyev)(const char& v, const char& ul, const int& n,
                        double* a, const int& lda, double* w,
                        double* work, const int& lwork, int* info );
}

int main() {
    const int n = 10, lda = n, ldv = n;
    int info = 0, lwork = -1;
    double wkopt;
    double* A  = new double[n*n];
    double* w  = new double[n];
    double* work;

    srand(time(0));
    for (int i = 0; i < n; ++i)     // Populate random symmetric matrix
        for (int j = i; j < n; ++j)
            A[i*n+j] = A[j*n+i] = (double)(rand())/RAND_MAX;

    // Query for optimal workspace size
    F77NAME(dsyev)('V', 'U', n, A, lda, w, &wkopt, lwork, &info);

    // Allocate workspace
    lwork = (int)wkopt;
    work  = new double[lwork];

    // Compute eigenvalues - A is replaced with eigenvectors
    F77NAME(dsyev)('V', 'U', n, A, lda, w, work, lwork, &info);

    // Print eigenvalues.
    for (int i = 0; i < n; ++i) {
        cout << "EV: " << i << ": " << w[i] << endl;
    }

    delete[] A, w, work;
}
```

Exercises

Exercise 5.1

In Exercise 4.4, the matrix of equations was solved using an iterative conjugate gradient technique which only required the use of explicit linear algebra operations (i.e. matrix-vector multiplications).

Copy your solution from this exercise and write an additional function which solves the system directly using routine(s) from LAPACK.

Check your solutions match.

Exercise 5.2

In Exercise 4.6, we time-advanced the heat equation using an explicit time-integration scheme. The stability of the scheme for a chosen space-step and time-step can be determined from the eigenvalues of the evolution matrix operator. If at least one eigenvalue is outside the unit circle (i.e. $|\omega_i| \geq 1$), the corresponding eigenvector will grow unbounded, leading to instability of the numerical method.

1. Use LAPACK to compute the eigenvalues of the explicit operator from your solution to Exercise 4.6 to confirm the scheme is stable for $\nu < 0.5$.

2. Use LAPACK to perform implicit time-integration in which

$$\frac{U_i^{k+1} - U_i^k}{\Delta t} = \alpha \frac{U_{i-1}^{k+1} - 2U_i^{k+1} + U_{i+1}^{k+1}}{(\Delta x)^2},$$

   and you are therefore now required to repeatedly solve the matrix system $\mathbf{A}\mathbf{U}^{k+1} = \mathbf{U}^k$. Choose the LAPACK routine(s) and storage formats carefully to maximise the efficiency of your program.

3. **(Advanced)** Measure the time taken to perform the complete time-integration part of your code using the Boost Timer library:
   http://www.boost.org/doc/libs/1_74_0/libs/timer/doc/cpu_timers.html

   Compare the performance of your implicit and explicit solvers for different values of time-step and space-step.

Exercise 5.3

(Advanced) The singular value decomposition (SVD) of a non-zero real matrix $\mathbf{A}$, with m rows and n columns, is given by

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

where $\mathbf{U}$ is an $m \times m$ matrix whose columns are the left singular vectors, $\mathbf{\Sigma}$ is a diagonal matrix of singular values such that $\Sigma_0 > \Sigma_1 > \ldots > \Sigma_{\min(m,n)}$, and $\mathbf{V}$ is an $n \times n$ matrix, whose rows are the right singular vectors.

Therefore the matrix can be expressed as a sum of outer products of the form

$$\mathbf{A} = \sum_{i=0}^{\min(m,n)} \mathbf{u}_i \Sigma_i \mathbf{v}_i^T$$

SVD can be used as a rudimentary form of image compression by representing the image as a rectangular matrix, decomposing it, and retaining only the first $k << \min(m,n)$ singular vectors.

1. Download the image `plane.jpg` from Blackboard Learn.
2. Use the Boost Generic Image Library to read the jpeg image and extract the red, green and blue components of the image pixels to three separate matrices. An example of how to do this is given on Blackboard Learn. Boost GIL documentation: http://www.boost.org/doc/libs/1_74_0/libs/gil/doc/html/index.html
3. Use LAPACK to perform the SVD on the three matrices and obtain the singular values and left/right singular vectors.
4. Combine the first k (user-defined) singular vectors to generate an approximation of the image matrix.
5. Use the Boost GIL library to combine the three approximate matrices back to a colour image and write to file.
6. Test your program by first choosing $k = \min(m,n)$. Explore how the image quality changes with reduced values of k.

Part III

Collaboration and Performance

Many C++ codes, particularly those used in high-performance computing environments, are large and often consist of hundreds of source files with hundreds of thousands of lines of source code. They often implement complex and cutting-edge algorithms in order to perform a given task in a computationally efficiency and scalable manner. There is often the need to try out new algorithms, extensions or ideas which may subsequently be included into the main code or discarded.

They are also typically written by teams of developers in a collaborative manner. While they may all sit together in the same office, teams are often geographically distributed in different institutions around the world; this is particularly true of open-source projects.

With these considerations in mind, there are a wide range of tools and best practices which are becoming increasingly essential for developing HPC codes in order to maintain the quality, reliability and sustainability of projects. These tools generally help developers to write better code, more efficiently.

In this part of the course we will examine a number of these best practices and the associated tools:

- Collaborative version control for teams of developers
- Build systems to manage the compilation process
- Profilers used to optimise code
- Documentation to capture the meaning of code
- Testing to check that the program was implemented according to its specifications
- Cluster usage for running on high-performance computing systems

Chapter 6

Git Version Control

Objectives

In this chapter you will learn about:

- the difference between centralised and distributed version control systems;
- the key concepts of code version control;
- how to use Git to manage a local repository;
- branches and how to use them;
- remote repositories and how to collaborate on code.

Almost all large-scale (and many small-scale) software projects use a version control system (VCS) to track the development of the code. These allow developers to save and keep track of different versions of the source files in some form of a *repository* as they are developed. This aspect is particularly useful when multiple developers are working on different parts of the code in parallel. Each version of the code in the repository is known as a *revision* or *commit*. Developers can often label important milestones such as the implementation of key functionality or public releases of the software.

6.1 Centralised and Distributed VCS

There are two fundamental types of VCS. The first is a centralised VCS in which a master copy of the code and all the revisions are kept on a central server and all developers send revisions to and from the server. This requires developers have an active connection to the server in order to work. Only the revision being worked on are held on the developers computer. Examples of centralised VCSs are Subversion and CVS.

The second type is known as a distributed VCS. In this case, each developer holds a copy of the entire repository, including all revisions, on their computer. This makes such VCSs very fast to use as there is no need to communicate over the network to a central server. To share code changes, groups of commits are sent between developers in a peer-to-peer manner. Examples of distributed VCSs are Git and Mercurial. Often a server is still used to act as a master copy of the code, but this

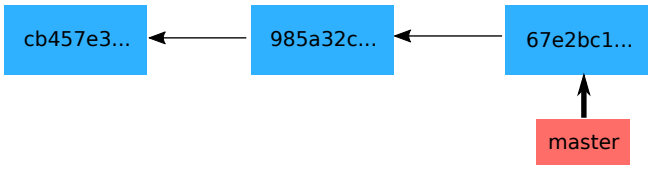


Figure 6.1: The chronology of changes to a set of files are represented in Git as a linked-list of commits. A human-friendly label usually points to the most recent commit.

is more for convenience than out of necessity; examples include GitHub or BitBucket. Distributed VCSs are considered more flexible and powerful, but at the cost of additional complexity.

We will examine the Git distributed VCS in the remainder of this chapter.

6.2 The `git` Version Control System

The Git distributed VCS is a system for recording the chronology of code development. It was originally designed and developed to manage the source code of the Linux kernel. Operations are performed locally using the `git` command.

It is a feature-rich tool, supporting a wide range of features, including

- Remote repositories (distributed development)
- Code branching and merging
- Tagging of important milestones
- Forking of repositories
- Data integrity

6.3 Core concepts

A git repository manages changes to a set of files within a directory. It represents the chronology of development as a linked list of *commits*. The files visible within that directory, known as the working copy, are the versions of the files at a specific point in that chronology (often the most recent commit). Git only records changes to files specifically marked to be tracked by the developer.

A simple illustration of commits is shown in Figure 6.1. Each commit records a snapshot of all the tracked files at a given instance. It has an ID (or hash) to uniquely identify the changes encapsulated in the commit, and a commit message describing the changes made can be added by the developer. Each commit also has one (or sometimes two) ancestor commits. To make it easier for developers, a label (e.g. “master”) can be used as a human-friendly alias to refer to the most recent commits. A *refspec* refers to either such a label, or a specific commit hash.

The development chronology can be allowed to diverge, in which case a *branch* is created. The branch can also be given a label and at some point *merged* back into the original line of code development. This ability to seamlessly branch and merge is one of the most powerful features of Git.

6.4 Initialising a new repository

If you start a new project and want to use `git` to track changes to the source files you must first initialise the repository. After creating a directory for your project (and possibly creating some initial

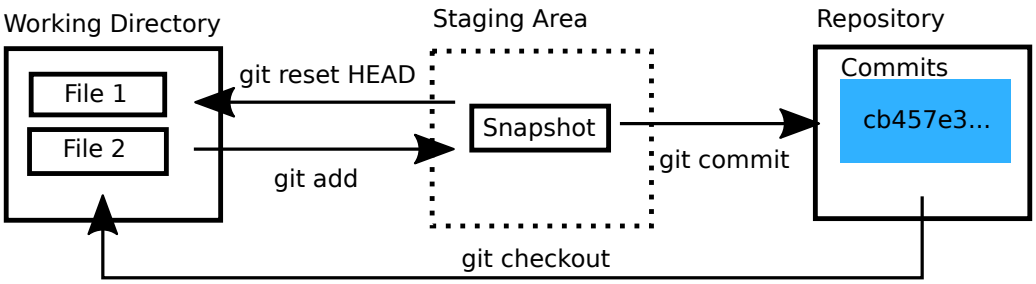


Figure 6.2: Basic workflow in Git

code) you can initialise a `git` repository using the `git init` command. This creates a hidden directory (called `.git`). After this you can start telling `git` which files to track and record your first commit.

6.5 Workflow

The working directory contains your source code for your computer program, along with other types of files such as machine code and executables. Git uses a *staging area* to explicitly collate together all the changes which will form the next commit.

This is illustrated in Figure 6.2. New files (or changes to existing files) can be added to the staging area using `git add`. When all required changes are staged, a commit is made using `git commit`. The staging area is then reset as all the previously staged changes are now recorded in the repository. The working directory can be updated to reflect the state of the files at a different point in the development chronology using the `git checkout`.

Usage of these commands is illustrated in Example 6.1.

Example 6.1: Basic `git` commands in action.

```
git add hello.txt          # Mark (changes to) a file to be committed

git rm hello.txt           # Delete a file. The file will still exist in
                           # earlier commits

git mv hello.txt bye.txt   # Move/rename a file under version control

git checkout cb17576       # Go back to an earlier commit cb17576...
git checkout master        # Go to commit pointed to by label 'master' (usually
                           # the most recent commit on the main branch of development)
```

To see the chronology of commits, along with the messages describing each commit, the `git log` command can be used. To see the actual changes to files between two commits, one can use `git diff`.

Example 6.2 illustrates some of these commands in action.

6.6 Remote repositories

Every user of a `git` repository has a complete copy of the code and its entire history. They can theoretically work completely independently of all other developers and disconnected from the rest of the world. However, most projects are developed by teams who will wish to share their work with each other. Remote repositories describe any copy of the repository that is not the developer's local

Example 6.2: Basic `git` command-line usage in a working directory called `myrepo`.

```
mkdir myrepo; cd myrepo     # Create new directory
git init                    # Initialise a new git repository

touch hello.txt             # Create a blank file
git add hello.txt           # Mark the new file to be committed
git commit -m "1st commit" # Make our first commit
# The 32 char hex string is the commit-id

git log                     # Review the log of commits
echo hello > hello.txt      # Modify the file
git diff                    # Review the changes we have made
git add hello.txt           # Mark the changes to be committed
git diff --cached           # Review the changes to be committed
git commit -m "2nd commit" # Make a second commit
git log                     # Review the log of commits
```

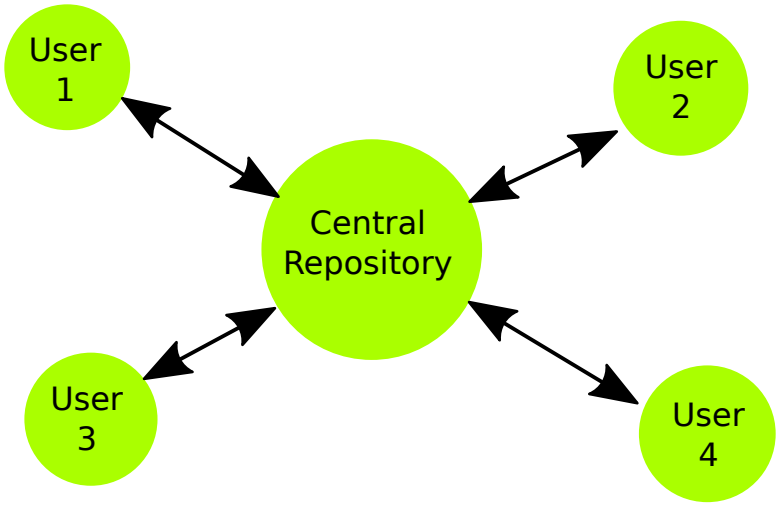


Figure 6.3: Many projects use a central repository to coordinate developers. Each user sees this as a remote repository to which they can push and pull commits.

copy. A remote repository may be a copy on another developer's computer, or may be a central repository on a server (such as GitHub). In fact, most projects maintain a central copy of the repository to coordinate the work of a large team of developers. Users work primarily in their local repositories, but use the central repository to publish code and share with other developers. This is illustrated in Figure 6.3.

Remote repositories can be listed, added and removed manually using the `git` command. See `git help remote` for more information.

6.6.1 Cloning Remote Repositories

To work with an existing remote repository, it must first be *cloned* locally, with makes an entire copy of the repository locally and checks out the latest 'master' branch commit:

```
git clone <repository-location> [<directory>]
```

The `repository-location` may be a local directory, but is typically a remote repository URL. Git can connect to repositories using standard HTTP or HTTPS, or more securely using the Secure SHell protocol (SSH). The latter is usually preferred, as encryption keys can be set up which avoid the

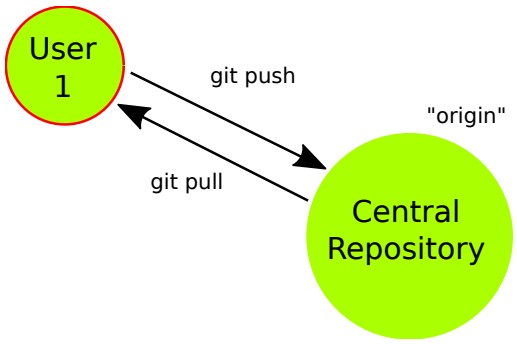


Figure 6.4: Commits can be transferred to and from a remote repository using the push and pull commands.

developer needing to repeatedly enter their password on every operation or to cache it on disk.

The optional `directory` argument allows you to specify the name of the directory to be created on your computer. By default, this will just be the name of the repository.

For example, we can clone the OpenBLAS project using:

```
git clone https://github.com/xianyi/OpenBLAS.git
```

This will clone the central repository on your computer. Note that we have used HTTPS and cloned the repository anonymously; we do not have permission to *push* commits to OpenBLAS. We can still make commits on our local repository.

After cloning a repository, the original URL is automatically added as a remote repository (with the label `origin`). You can see this by listing the remote repositories:

```
git remote -v
```

The ouptut will be:

```
origin  https://github.com/xianyi/OpenBLAS.git (fetch)
origin  https://github.com/xianyi/OpenBLAS.git (push)
```

6.6.2 Pushing and pulling

The action of sending commits to a remote repository is called *pushing*, while the action of downloading commits from a remote repository is called *pulling*. This is illustrated in Figure 6.4.

Typically, we add one or more commits to our repository and then wish to share them with other users. To do this we use

```
git push [<repository-label> [<refspec>]]
```

where the `<repository-label>` is the name of the label associated with the remote repository, for example `origin`. The optional `<refspec>`, e.g. `master` or a commit hash, specifies the commits to be pushed. It defaults to the current branch.

To retrieve new commits from a remote repository, we use the corresponding command

```
git pull [<repository-label> [<refspec>]]
```

where the `<repository-label>` defaults to `origin`.

A more complete example is shown in Example 6.3.

Example 6.3: Working with remote repositories

```
# First clone repository
$ git clone http://github.com/myuser/myrepository.git myrepo
$ cd myrepo

$ git remote -v       # See details of remote repositories
origin  http://github.com/myuser/myrepository.git (fetch)
origin  http://github.com/myuser/myrepository.git (push)

# ... Now do some coding ...

$ git add myfile.txt  # Stage files
$ git commit          # Make a commit (this is local)

# ... Possibly more coding and more commits...

$ git push origin     # Send our commit(s) to remote repository
```

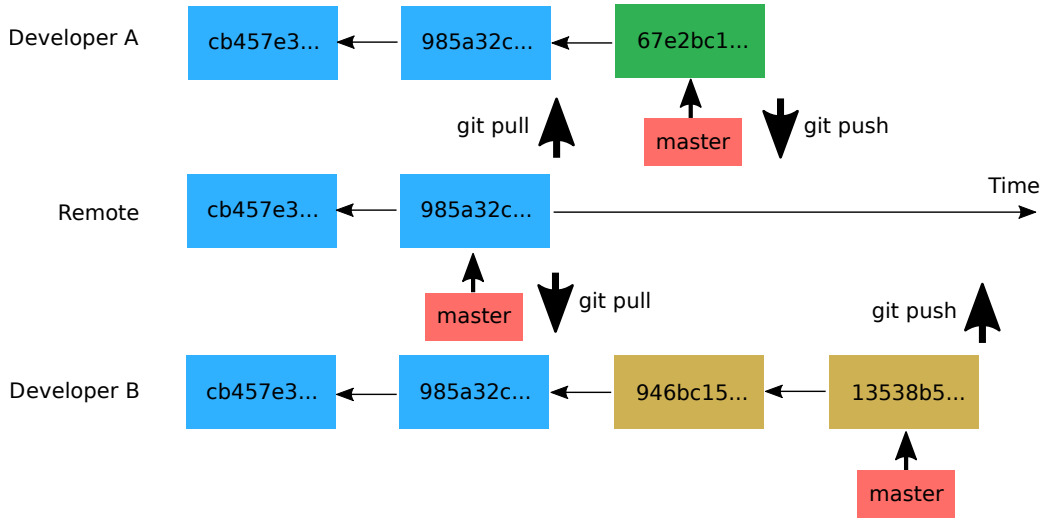


Figure 6.5: Conflicts can arise when two developers make changes to the same piece of code.

6.7 Collaborating

Git makes collaborating on code development easy and efficient. However, difficulties can arise when two developers make different changes to the same part of the code.

Figure 6.5 illustrates this problem, where two developers are working locally and pushing their commits up to a remote repository. Developer A makes some changes, starting from commit `985a32c...` and pushes his commits up to the remote repository. Developer B meanwhile has also made some changes, starting from the same commit. When she pushes her commits up to the repository a conflict occurs as the label `master` now could refer to either commit ancestries.

There are two possible solutions:

- Developer B pulls the new commit from Developer A and merges those changes into her branch.
- Developer B reapplies her changes on top of those from Developer A.

The latter is the cleaner way and is called *rebasing*.

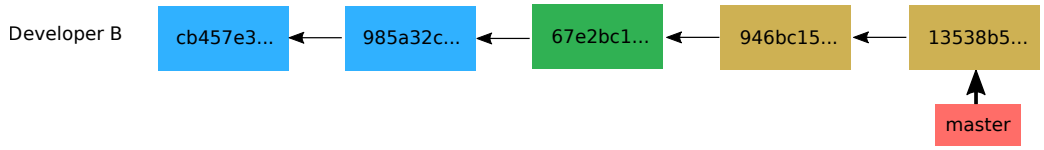


Figure 6.6: After rebasing Developer B's changes onto those of Developer A, a single sequence of commits is maintained.

6.7.1 Rebasing

Rebasing involves rolling back a sequence of commits and reapplying them on top of another commit. This is often used when resolving conflicts like those illustrated in Figure 6.5. Developer B can instead use

```
git pull --rebase
```

The following then takes place:

1. Commits by Developer B are reversed back to the common ancestor, in this case commit `985a32c...`
2. Changes from Developer A are pulled from the server and applied; this is called *fast-forwarding*.
3. Commits from Developer B are applied on top.
4. Any conflicting changes to the code are resolved by Developer B.

Developer B can now push her commits up to the remote repository and they will now be placed after those of Developer A, as shown in Figure 6.6.



Note

When reapplying the commits of Developer B on top of Developer A, new commit hashes will be generated for the commits of Developer B when reapplying them.

This is because the hash of a commit depends not only on the changes in the commit, but also on the date, author and hash of the parent (previous) commit (which has now changed). However, for clarity of concept, this is now shown in Figure 6.6.

6.7.2 Branching

Branches allow new code to be developed and tested away from the main code development path. This feature is useful to use when developing a significant new feature as it enables multiple intermediate commits to be recorded in the repository without breaking the code for other developers working on the main 'master' branch.

To create a branch use

```
git branch <branch-name>
```

This command adds a new label (`<branch-name>`) to the most recent commit. We can then change to the branch

```
git checkout <branch-name>
```

We are now on <branch–name> instead of master.

To see a list of branches we can do

```
git branch [-a]
```

With the –a argument we can see remote branches too.

To push a specific branch:

```
git push origin <branch-name>
```

6.7.3 Merging

Merging is the process of combining the changes from one branch into a target branch. Typically the target branch is master into which one merges changes from another branch. To do this, change to the target branch and run

```
git merge <branch-name>
```



Note

Merging implicitly happens when using `git pull` to bring in changes from a remote repository (if not explicitly forcing a rebase using ––rebase).

An example of merging a branch (maybe containing a new code feature) into the `master` branch is given in Example 6.4.

Example 6.4: Merging the branch `mynewfeature` into the target branch `master`

```
1 git checkout master    # Change to the branch we want to merge into
2 git pull --rebase      # Check we are up to date
3 git merge mynewfeature -m "Merged my new feature into the main code"
4 git push origin master # Push your changes back to remote repository
```

If there are conflicting changes, you will need to resolve these manually. Git will report a list of files which have conflicts and leave the two versions of the conflicting sections of code in the source files annotated with tags: <<< and >>>. For each conflicting file, you need to edit it, keep the version of the code you want (and remove the annotations), and then stage it using `git add`. After fixing all conflicting files, the merge process can be completed by running `git commit`.

6.7.4 Tagging milestones

Tags provide a way to bookmark specific points in a projects history. e.g.

- New release versions of the code;
- Significant new features added;
- Overhaul of a library API.

Tags are conceptually just like labels, e.g. master or mybranchname, but are tied to a specific commit.

To create a tag, run `git tag`, for example

```
git tag v2.0 -m "Completed version 2.0"
```

will create a new tag called `v2.0` with an associated message.

To list the available tags run

```
git tag -l
```

Finally, to delete a tag, use `git tag -d`, for example

```
git tag -d v2.0
```

6.7.5 Forking repositories

We were able to clone the OpenBLAS repository by doing

```
git clone https://github.com/xianyi/OpenBLAS.git
```

We are not members of the project and cannot push commits. However, we can make a *fork* (copy) of the repository. This is particularly useful if you want to propose a change to another person's project. On GitHub, you could click **Fork** on the OpenBLAS repository to create a linked copy on your account. You can now experiment, push changes, new features, fixes, etc, to your copy of the repository. To submit your work to the original project you open a *pull request* on the GitHub website.

Exercises

Exercise 6.1

Create a `git` repository to store a simple *Hello World!* program.

Before using `git` for the first time on a computer, you should configure it so that your name and email address appear correctly in commit messages:

```
git config --global user.email "your-name@imperial.ac.uk"
git config --global user.name "Your Name"
```

This only needs to be done once. It will be used for all future git repositories you work with using your account.

1. Change into the directory containing your solutions to previous exercises and initialise a new `git` repository.
2. Create a new project with a code that prints “Hello World!” to the terminal.
3. Add `hello.cpp` to the git staging area and commit it.
4. Review the git log to confirm the commit has been made successfully.
5. Make a change to the code so that it prints “Goodbye World”, save the code, compile it and test it.
6. Use the `git` command to examine the changes made to this file.
7. Stage and commit the revised version of the file to the repository.
8. Examine the commit log again to confirm.

The frequency with which you commit changes is not prescribed, but you should typically make a commit for each new feature or fix in your code.

To practise the use of version control, use this repository to add the code you developed in previous exercises and to manage the code you write in the exercises which follow. You should probably be committing your updates after completing each exercise at a minimum.

Exercise 6.2

In this exercise we will create a branch, make some changes and merge the branch back into the repository.

1. Create a new directory, change into it, and initialise an empty git repository.
2. Create a simple program in `hello.cpp` as follows:

```cpp
#include <iostream>

int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

   Save it, add it to the staging area and commit it. Check `git log` to confirm the commit is there.

3. Create a new branch called 'goodbye', but do not switch to it yet.
4. Still on the 'master' branch, edit `hello.cpp` to add `using namespace std;` and to remove the `std::`. Commit this change.
5. Switch to the 'goodbye' branch. Check you have done this successfully by running 'git branch'.

   Edit `hello.cpp` again, changing "Hello" to "Goodbye". Note that the namespace change is **not** present in this file as it was made after the branch was created. Commit the change. Run `git log` to see the sequence of commits on the 'goodbye' branch.
6. Switch to 'master' again. Merge the 'goodbye' branch. Note that a merge conflict should occur as the `cout` line was changed differently on each branch. Edit the file and resolve the conflict. Commit to complete the merge.

Exercise 6.3

Repeat steps 1-5 in Exercise 6.2.

Instead of merging branch 'goodbye' into 'master', instead *rebase* the commits in the 'goodbye' branch onto the latest 'master'. In rebasing, we roll back the changes we made on the 'goodbye' branch and reapply them on top of the latest commits from the 'master' branch (resolving conflicts as we go).

Now merge the 'goodbye' branch into 'master' and notice there are no conflicts to resolve. Git does what is called a *fast-forward* merge and does not create an additional *merge commit*.

Note: Rebasing is good for keeping a clean history. However, once you have pushed your branch up to a remote repository and shared with others, you should not rebase it as it will rewrite the entire branch. This could cause problems if another developer has based work on one of your commits.

Chapter 7

Build systems

Objectives

In this chapter you will learn about:

- how to use the Make command and Makefiles to specify dependencies between source files;
- how to use CMake to simplify Makefile generation.

Most of the exercises discussed so far have been quite trivial to compile. For example, if we only have a couple of source code files, we can just run something like

```
g++ -std=c++11 -Wall -O2 myprog.cpp myfunction.cpp
```

every time we change our code. As our code grows bigger and we have many source code files to compile this can become slow as the compiler must process all the code in the project every time we make even the smallest change. In addition, the command-line can become somewhat long and unmanageable, particularly if we need to also link to multiple libraries and use various compiler flags.

To address this problem, we might consider compiling each source file separately and then linking the resulting machine code,

```
# Compile main code
g++ -std=c++11 -Wall -O2 -o myprog.o -c myprog.cpp
# Compile myfunction
g++ -std=c++11 -Wall -O2 -o myfunction.o -c myfunction.cpp
# Link everything together
g++ -o myprog myprog.o myfunction.o
```

This is now much more efficient, since we can just compile those files which are changed and then re-link the result.

With large projects it can still be challenging to keep track of dependencies between files. For example, changing a header file requires all source code files which include that header to be recompiled. For large projects this becomes an untenable situation.

Build systems, and their associated build tools, are designed to address this problem. They automate the compilation process by calculating which tasks (e.g. compiling, linking) need to be run,

and in which order, so as to accomplish an objective. The goal is often to generate an executable.

We will briefly introduce two build systems. The first is called Make, while the second is called CMake. The latter is a sort of meta-build-system, since it generates files for Make under Linux.

7.1 Make

Make is the standard tool used on Linux to help build software packages. It is configured by one or more text files, typically named `Makefile`, located in the source code directories. Each of these files contain *rules* which describe how to build *targets*, along with any necessary *dependencies*. The targets and dependencies are typically files, but non-file targets may also be used.

Rules have the following syntax:

```
<target>: <dependencies>
  <command>
  ...
```

The dependencies list the other targets that must be generated first before we are able to build this target. When these are available, this target is generated by running the commands listed.



Note

The commands for a rule are indented by exactly one TAB character. This is part of the syntax. This is not the same as indenting with spaces, and Make will complain if you do that!

The `make` manual has full and comprehensive documentation and is available at http://www.gnu.org/software/make/manual/make.html.

7.2 Basics

An example of a simple `Makefile` is given in Example 7.1

Example 7.1: A simple Makefile to compile a program. Note that the g++ lines are preceded by a tab character, and not spaces.

```
1  default: myprog
2
3  myprog.o: myprog.cpp
4      g++ -std=c++11 -Wall -O2 -o myprog.o -c myprog.cpp
5
6  myfunction.o: myfunction.cpp myfunction.h
7      g++ -std=c++11 -Wall -O2 -o myfunction.o -c myfunction.cpp
8
9  myprog: myprog.o myfunction.o
10     g++ -o myprog myprog.o myfunction.o
```

There are four rules defined in the example.

- The first rule (line 1) simply defines the default target which should be built when `make` is run.

- The second rule (line 3) describes how we should make the object code `myprog.o`. This understandably depends on the corresponding source file `myprog.cpp`. The command to build the target is the command `g++`, with a number of arguments (line 4).
- The third rule (line 6-7) describes how we should make the object code `myfunction.o`.
- The fourth rule (line 9-10) describes how to link together our object codes and make an executable.

To see this in action, we can invoke the `make` command in the directory containing our `Makefile` (and source code). This should produce output similar to the following:

```
$ make
g++ -std=c++11 -Wall -O2 -o myprog.o -c myprog.cpp
g++ -std=c++11 -Wall -O2 -o myfunction.o -c myfunction.cpp
g++ -o myprog myprog.o myfunction.o
```

When `make` is run for the first time, the specific commands are printed as each of the compiler commands is executed. If we now run `make` a second time (without changing our source files), we get the message

```
$ make
make: 'myprog' is up to date.
```

Make has checked all the dependencies of myprog to ensure none of them are newer than the current version of the myprog executable and deduced that nothing needs doing.

We will now discuss a number of features available with `make` and in the process simplify our build system considerably.

7.2.1 Variables

Variables can be used to simplify Makefiles. Variables can have any name not including the characters: :, # and =. For example, they could be used to simplify our example above by avoiding the repeated flags.

```
flags = -std=c++ -Wall -O2
myprog.o: myprog.cpp
    g++ $(flags) -o myprog.o -c myprog.cpp
```

This reduces the risk of errors and inconsistencies and allows the compiler flags for all source files to be easily modified in just one place. Notice that the variable is prefixed with a dollar sign ($) when reading/referencing the variable. It is also recommended when referencing a variable value to enclose the name in round braces.

Some variables are predefined, but may be overridden by the user. These are often used in Implicit Rules (see Section 7.2.2). Examples include:

CXX	The C++ compiler to use. Default value: g++
CXXFLAGS	Extra flags to give to the C++ compiler. Default value: (empty)
LDLIBS	Libraries to link. Default value: (empty)
LDFLAGS	Extra flags to be given when linking. Default value: (empty)

7.2.2 Implicit rules

Compiling a C++ source code file into an object file is a standard task. Make therefore includes a number of *implicit rules* for such tasks. These rules use the file extension of the target and dependencies to decide which commands to run. The commands often make use of predefined variables for defining the compiler name, flags, etc. For example:

- The rule `x.o : x.cpp` would result in running

```
$(CXX) $(CPPFLAGS) $(CXXFLAGS) -c -o  x.o  x.cpp
```

- The rule `x : x.o y.o z.o` would result in running

```
$(CC) $(LDFLAGS) x.o y.o z.o $(LOADLIBS) $(LDLIBS)
```

These are often sufficient for most programs, if the variables `CXXFLAGS`, `LDFLAGS` and `LDLIBS` have been set appropriately (if needed).



Note

As a small caveat, when using implicit rules for C++ programs, we need to override $(CC) to be g++ since otherwise linking would be done by a C compiler.

Example 7.2: Simplifying a makefile with implicit rules.

```
1  CC=g++
2  CXXFLAGS=-std=c++11 -Wall -O2
3  HDRS=myfunction.h
4
5  default: myprog
6
7  myprog.o: myprog.cpp
8
9  myfunction.o: myfunction.cpp $(HDRS)
10
11 myprog: myprog.o myfunction.o
```

7.2.3 Pattern rules

If more control is required than offered by implicit rules, custom rules can be defined called *pattern rules*. A pattern rule is essentially an ordinary rule, except the target contains the `%` symbol. `%` symbol can match any non-empty string. The contain a `%` symbol to show where they must contain the *stem* that was matched in the target.

For example, the pattern rule

```
%.o : %.cpp
```

would match `myfunction.o` and result in the prerequisite `myfunction.cpp`

7.2.4 Automatic variables

Automatic variables allow the substitution of targets and dependencies in pattern rules.

Common automatic variables include

$@	The name of the target
$<	The name of the first prerequisite
$^	The names of all the prerequisites
$?	The names of all the prerequisites newer than the target

For example, given the rule

```
%.o : %.cpp $(HDRS)
    $(CC) -c $< -o $@
```

a request to build the target myfunction.o would expand this to

```
myfunction.o: myfunction.cpp myfunction.h
    $(CC) -c myfunction.cpp -o myfunction.o
```

We can now simplify Example 7.1 using the features above, as shown in Example 7.3. In this

Example 7.3: Improved Makefile

```
1  CXX = g++
2  CXXFLAGS = -std=c++11 -Wall -O2
3  HDRS = myfunction.h
4  OBJS = myprog.o myfunction.o
5  LDLIBS = -llapack -lblas
6
7  %.o : %.cpp $(HDRS)
8      $(CXX) $(CXXFLAGS) -o $@ -c $<
9
10 myprog: $(OBJS)
11     $(CXX) -o $@ $^ $(LIBS)
12
13 all: myprog
```

example we have used made use of variables, pattern rules and automatic variables. To add extra source files, we just add it to the HDRS and OBJS variables. We have also linked to some external libraries.

7.2.5 Cleaning

It is useful to create a rule to *clean* (reset) the source directory. This would remove all the files generated during the build, such as the executable and the intermediate object files. We can do this by writing a *phony* target:

```
.PHONY: clean          # Specify that 'clean' is not a real file
target

clean:
    -rm -f *.o myprog   # Clean up (and ignore any errors)
```

7.2.6 Default rules

When we type `make`, without specifying a specific target, it will build the first target it finds in the Makefile. We should therefore define a default target at the top

```
default: myprog
```

One or more specific targets from the Makefile can also be built by explicitly specifying them on the command-line:

```
make <target-name>
```

7.3 CMake

CMake is a cross-platform build system for build systems. By cross-platform we mean that it runs on Linux, Windows and OSX. It knows about the different sets of tools, such as compilers and linkers, available on each system. These are known as *toolchains*. Its purpose is not to directly build the software itself, but to automatically generate the configuration for a platform-specific build system. On Linux, it can generate the Makefiles we are already familiar with while on Windows it can generate Visual Studio projects. It can also generate CodeLite projects.

CMake is an alternative to Make, which improves the ability of people to compile your code on a range of operating systems (assuming the necessary libraries are available). The principles are similar to Makefiles. We describe targets, their dependencies and how to make them. However, CMake also includes a range of scripts to automatically find common libraries in an operating-system agnostic way.

Remember, CMake does not actually build the software, but generates a operating system-specific build configuration using the appropriate tools for the platform.

7.3.1 Basics

Similar to the way that Make requires a `Makefile` to be created in the directory containing the source files, a file `CMakeLists.txt` is required to be in the same directory for CMake.

Example 7.4 shows the simplest `CMakeLists.txt` file.

Example 7.4: A simple CMake file `CMakeLists.txt`

```
cmake_minimum_required(VERSION 2.8.9)
project (myprog)
add_executable(myprog myprog.cpp myfunction.cpp)
```

The first line specifies the minimum version of CMake required for the project. Since newer versions of CMake include new or changed functionality, this ensures the commands in the `CMakeLists.txt` files are compatible with the available version of CMake. The second line simply defines the name of the project.

The third line states that an executable called `myprog` is to be generated from the source code `myprog.cpp` and `myfunction.cpp`. On Linux, CMake will automatically generate Makefiles with rules for building each source file and linking them together.

7.3.2 Running CMake

CMake can be run using the `cmake` command:

```
cmake ./
```

To avoid cluttering the source code directory with intermediate CMake, Make and build files, we can build *out-of-tree*. We create a sub-directory (e.g. `build`) and run CMake there. For example, from our source code directory, we might run:

```
mkdir build
cd build
cmake ../
```

7.3.3 Variables

CMake supports variables, however they are not assigned in the same was as in a Makefile. Instead we must use the `set` command. For example,

```
set(sources "myprog.cpp␣myfunction.cpp")
```

Variables are referenced using the dollar symbol ($) and curly braces.

```
add_executable(myprog ${sources})
```

7.3.4 Finding Libraries

CMake includes a number of scripts to find common libraries. For example, if we require the BLAS library, we can use the `find_package` command.

```
find_package(BLAS REQUIRED)
```

The first argument is the name of the library to find. The second optional argument tells CMake that it should stop if it cannot be found. This function will populate a number of CMake variables. For example, `BLAS_FOUND` is true if the library was found. `BLAS_LIBRARIES` contains the (list of) libraries needed to link against BLAS.

7.3.5 Linking Libraries

To link libraries in CMake we use the `target_link_libraries` command. This takes two parameters: the name of the target and the libraries to link. For example, now that we have found BLAS, we can link our executable against it using

```
target_link_libraries(myprog ${BLAS_LIBRARIES})
```

7.3.6 Configuring builds

As well as the `cmake` tool, there is a configurable cmake tool called `ccmake`. This provides a menu system which allows the build of a project to be configured. For example, after running `ccmake`, we might change the `CMAKE_BUILD_TYPE` variable to `DEBUG` in order to build a version of our code with

Debug symbols. After adjusting the configuration we press `c` to validate the configuration, followed by `g` to generate the Makefiles.

7.3.7 Further information

We have provided only a brief introduction to CMake here. For details on the full rich set of features, consult the documentation at: https://cmake.org/cmake-tutorial/

Exercises

Exercise 7.1

Split your code from Exercise 1.1 into three files: `main.cpp`, `fibonacci.h` and `fibonacci.cpp`

1. Using the terminal, compile the whole program using a single command.
2. Using the terminal, compile each `*.cpp` file separately and then link the resulting object files to create an executable.
3. Write a Makefile that will compile the program using explicit rules for each `*.cpp` file.
4. Create another Makefile that will compile the program using implicit rules.
5. Create a rule to clean the source directory of the files generated during compilation.
6. Define 'default' and 'all' target rules.

Exercise 7.2

Using the C++ files created in Exercise 7.1:

1. Write a `CMakeLists.txt` that will generate the makefiles.
2. Create a subdirectory `build` and within it run:

```
cmake .. # .. is the parent directory (where CMakeLists.txt is)
make
```

Chapter 8

Testing

Objectives

In this chapter you will learn about:

- the different types of tests and their purpose;
- strategies for testing your code for correctness;
- how to use the Boost test framework to simplify generating tests.

Testing is the process of asserting the correctness of code and verifying it performs to its design specification. There are several levels of testing which can be used in combination to check your code.

8.1 Types of tests

Definition: Unit test

A unit test checks an individual code unit or component of a piece of software.

Unit testing is typically used for verifying the correctness of individual functions, classes or other data structures. We might want to check a function produces the correct output for specific inputs, for example. For a class, we may check that all interface routines behave as expected under the design.

Each unit test should only test one specific concept. It will also test one specific code path. For more complicated units with multiple code paths, there should be at least as many unit tests as there are possible code paths.

Definition: Integration test

An integration test checks if multiple components that have been tested independently, behave correctly when combined together.

Definition: System test

A system test checks a complete system, comprising all the required components, meets the design specification.

Integration testing should happen after unit testing and system testing usually happens at the end. Integration testing might test that two related classes interact correctly. A system test might test the complete application code as a whole works correctly.

For this course, we will just consider unit testing of our code.

8.2 Basic unit testing

Suppose we wish to test the following function:

```
int mymultiply (int a, int b) {
    return a * b;
}
```

We might write a code which checks this for a few different inputs:

```
int main() {
    if ( ! (mymultiply(2,3) == 6) ) {
        cout << "Failed test mymultiply(2,3)" << endl;
        return 1;
    }
    return 0;
}
```

If the test fails, an error message is printed and the program exits with a non-zero return value. If it passes, the program exits successfully. There is only one code path in our function, and so we have covered them all.

Now consider a second example,

```
double myabs(double x) {
    if (x >= 0) {
        return x;
    }
    else {
        return -x;
    }
}
```

As you can see, there are now two code paths to test. We might design two tests as follows to cover both code paths:

```
int main() {
    if ( ! (myabs(5) == 5) ) {
        cout << "Failed test myabs(5)" << endl;
        return 1;
    }
    if ( ! (myabs(-7) == 7) ) {
        cout << "Failed test myabs(-7)" << endl;
        return 1;
    }
    return 0;
}
```

8.3 Test frameworks

Generating tests manually can become very laborious and (ironically) error-prone, particularly for more complicated functions and classes. A testing framework is designed to simplify the process of creating tests for units of code.

One example is the Boost Test framework, although there are many others with pros and cons.

Example 8.1: Unit testing code with the Boost Test framework.

```
#include "mymath.h"

#define BOOST_TEST_MODULE MyMath
#include <boost/test/included/unit_test.hpp>

BOOST_AUTO_TEST_CASE( MyMultiply )
{
    BOOST_CHECK( mymultiply(2,3) == 6 );
}

BOOST_AUTO_TEST_CASE( MyAbs )
{
    BOOST_CHECK( myabs(5) == 5 );
    BOOST_CHECK( myabs(-7) == 7 );
}
```

Example 8.1 shows how to use Boost Test to write unit tests for the code in the previous section, which we suppose are declared in a file called `mymath.h`. The `BOOST_AUTO_TEST_CASE` macro indicates the start of a new test. In this case, we have defined one test for each routine. For classes or more complicated routines, where there are a multiple aspects to be tested, there may be multiple test cases, which each check multiple items.

To run our testing code, we first need to compile it, along with the code we are actually testing. Behind the scenes, Boost creates the `main()` routine for our testing code, so this does not need to be written explicitly.

```
g++ -o mymath-test mymath-test.cpp mymath.cpp
```

Running the code, should produce successful output:

```
$ ./mymath-test
Running 2 test cases...

*** No errors detected
```

Boost also ensures that our test code returns 0 if the tests were all successful or a non-zero value otherwise.

In Example 8.1 we used the `BOOST_CHECK` macro to test for equality. If this fails, it would produce an error similar to the following (where we have explicitly introduced a failure!):

```
./mymath-test
Running 2 test cases...
mymath-test.cpp(13): error: in "MyAbs": check myabs(5) == 6 has failed

*** 1 failure is detected in the test module "MyMath"
```

However, Boost continues the test case to the end.

If instead we decided it was meaningless to continue the test case in the event that a check failed, we might instead use

```
BOOST_REQUIRE( myabs(5) == 5 );
```

There is also an explicit test for equality,

```
BOOST_CHECK_EQUAL( myabs(5), 5 );
```

which has the added benefit of showing the incorrect value in the error message:

```
./mymath-test
Running 2 test cases...
mymath-test.cpp(13): error: in "MyAbs": check myabs(5) == 6 has failed [5 != 6]

*** 1 failure is detected in the test module "MyMath"
```

Finally, for floating-point values, we should test whether they are equal to within a prescribed tolerance using,

```
BOOST_CHECK_CLOSE( exp(1.0), 2.718, 1e-03 );
```

Exercises

Exercise 8.1

Generate an appropriate set of unit tests for Exercise 3.1 using the Boost Test framework.

Chapter 9

Documentation

Objectives

In this chapter you will learn about:

- how to effectively document your code using comments;
- doxygen and how to generate HTML documentation for your program.

Documentation of a computer program provides a description, in plain language, of the function of aspects of the code. There are several forms of documentation which are often produced, the most important of which are:

- User Guide: documentation provided for the benefit of the user of the program. This generally describes how the user can interact with the software through some form of user interface (such as command-line or graphical).
- Code comments: this form of documentation is provided for the benefit of the developers who write the code. It usually provides a reference for variables, functions, datatypes, etc.

Other forms of documentation might include

- Developers Guide, explaining higher-level structural aspects of the code;
- Change log, listing bug fixes and new features with each release version;
- Contributing Guide, for new developers joining open-source projects;
- Licence, detailing how, what, and when the code can be distributed.

In this section, we will focus on code comments and how to make effective use of them. While many of the exercises in this course lead to quite simple codes and therefore in need of little documentation, it is essential for larger projects with multiple developers. It allows other developers to easily understand your code and vice versa; it also reminds you how your code works when you look back at it in two years time!

9.1 Creating comments

Comments in C++ can be created in two ways:

- Single line comments (double backslash, or C-style)

```
// a single-line comment

/* an old c-style comment */
```

- Multiple line comments with `/*` and `*/` (the intermediate `*` are optional)

```
/*
 * a comment spanning
 * multiple lines
 */
```

9.2 What to document

What you choose to document is usually governed by the policy of the project, or otherwise personal preference. However, at a minimum you should document the following which provides a good overview of how your code works with the least effort.

- What the program does (name, author, date and description).
- A brief description of each routine and what the parameters are.
- A description of what the routine returns.

It is usually helpful to also add comments to more complicated code segments detailing **in a human language** what is being done by the algorithm! Make sure comments contribute insight into the code and do not just repeat the code in words. For example, the following is a useless comment:

```
// Set a[p] equal to val
a[p] = val
```

while the following is more helpful

```
// Put 'val' in the correct place in the sorted array
a[p] = val
```

A more illustrative example is given in Example 9.1.

9.3 Doxygen

Reading comments in code can sometimes be annoying, since they are interspersed with the code. Doxygen is a tool which extracts the comments in your code and produces more user-friendly HTML web-page documentation describing the classes, functions and variables in your code. As well as presenting the comments in a more visually pleasing form, it can also perform additional analysis, such as generating dependency diagrams between classes, functions, variables, etc, to show the relationships between different parts of the code. It also generates hyperlinks to make it easier to move around larger codes. An example of the output is shown in Figure 9.1.

Example 9.1: Illustrative documentation for part of an insertion sort program.

```
/*
* Perform a single insert step in the insertion sort.
* a[] is the array of values to sort.
* p is the position up to which all values are sorted.
* val is the value to insert in the correct place.
*/
void insert(float a[], int p, float val) {
    // Move items up array until the right place is reached
    while (p > 0 && val < a[p-1]) {
        a[p--] = a[p-1];
    }
    // Put 'val' in the correct place in the sorted array
    a[p] = val;
}
```

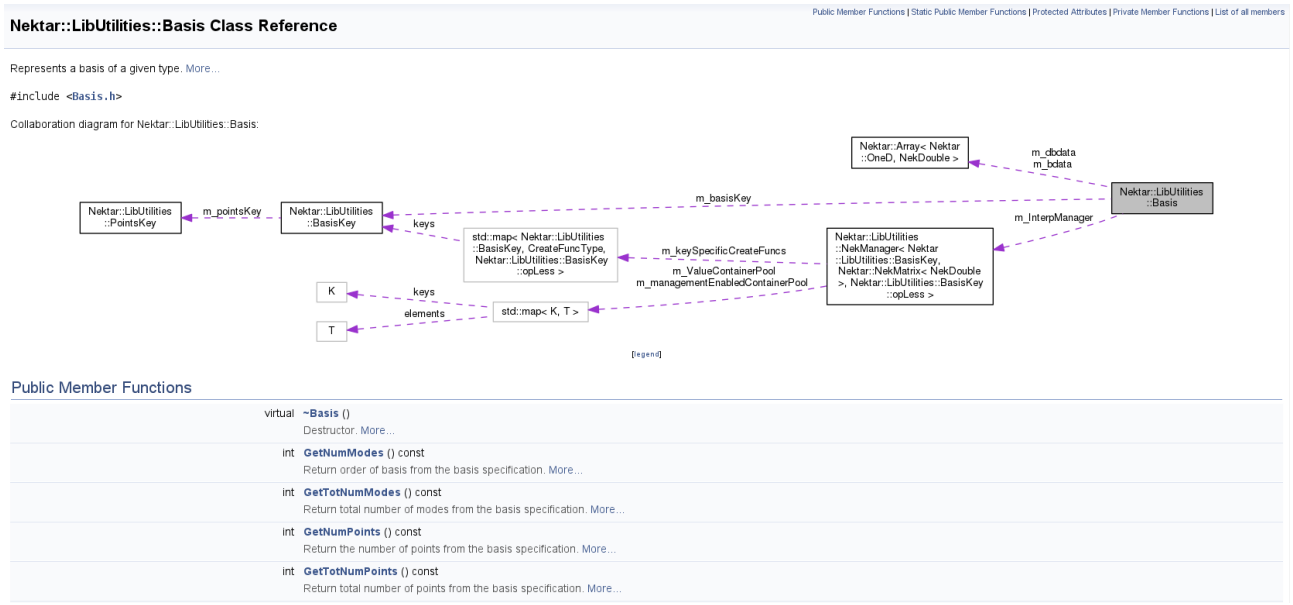


Figure 9.1: Sample of doxygen output showing a dependency diagram and list of member functions for a class.

9.3.1 Annotating code

Doxygen uses a special syntax for code comments to allow it to better interpret the information they contain.

Doxygen assumes comments relate to code which follows them. It uses a modified form of the comment syntax to indicate those comments which should be processed by doxygen. All other comments are ignored.

- `///` for one-line comments, or `///<` for comments on same line as the code being documented.
- `/**` ... `*/` for lengthy multi-line comments.

Within comments, a special mark-up syntax is used to annotate code. For example,

- `@brief` indicates short description of a class or function which follows.
- `@param <name> <desc>` is used to describe a function parameter.
- `@returns <desc>` is used to describe a return value.

- `\f$...\f$` can be used to include mathematics as LaTeX. These formulae are rendered to images for the HTML documentation.

An example of how we might use these to document a function is given in Example 9.2, while a more substantial example of documenting a class is given in Example 9.3.

Example 9.2: Using Doxygen to document a function

```
/**
 * @brief Perform a single insert step in the insertion sort.
 * @param a     array of values to sort.
 * @param p     position up to which all values are sorted.
 * @param val   the value to insert in the correct place.
 */
void insert(float a[], int p, float val) {
}
```

Example 9.3: Using Doxygen to document a class.

```
/**
 * @class Point
 * @brief Describes a position on the 2D plane.
 */
class Point {
    public:
        /// Default constructor
        Point();
        /// Construct point with a given coordinates
        Point(double x, double y);
        /// Create a copy of an existing point
        Point(const Point& pSrc);
        /// Clean up and deallocate memory
        ~Point();

        /// Get the x-coordinate of the point
        double getX();
        ...
    private:
        double mX;     ///< x-coordinate of point
};

/**
 * Create a point at \f$ (x,y)\in\mathbb{R}^2 \f$.
 * @param pX    x-coordinate of point
 * @param pY    y-coordinate of point
 */
Point::Point(double pX, double pY) : mX(pX), mY(pY) {
    // Normal comment using only two slashes (not parsed by doxygen)
}
```

9.3.2 Configuring and running doxygen

Doxygen is configured through a text file. It has the default name of `Doxyfile` and is formatted similar to a Makefile using syntax of the form `TAG = VALUE`.

A template configuration file can be easily generated by running

```
doxygen -g
```

This can then be edited as required. Alternatively, one can use the `doxywizard` graphical interface.

The HTML documentation is then generated using

```
doxygen Doxyfile
```



Note

By default Doxygen will only generate documentation for classes. If you use it with a code with standard functions, you should set the `EXTRACT_ALL` option to `YES`.

9.3.3 Further information

Further details can be found at:

http://www.stack.nl/~dimitri/doxygen/manual

Exercises

Exercise 9.1

Try annotating some of your solutions to the exercises in Chapter 1 using Doxygen.

Make sure to describe the purpose of each function and function parameter in the program, as well as the return values.

Chapter 10

Profilers and Code Optimisation

Objectives

In this chapter you will learn about:

- how to use a profiler to collect and analyse performance data;
- performance bottlenecks and how to find them;
- how to optimise your code to improve performance.

We now have a detailed understanding of how to write C++ instructions and compile them into executable code. While we have the capability to debug code when it goes wrong using a debugger, we do not yet have the means to examine the efficiency of our correct code. To do this we need another tool.

10.1 What is a profiler?

Definition: Profiling

Profiling is the process of gathering information about the running of a program and analysing it, typically for the purpose of program optimisation. It is usually undertaken using a *profiler*.

A profiler is a tool which *observes* our program while it is running and makes measurements about its resource usage and performance. It can record information such as how many times a function is called, and from where, as well as how much time is spent in each function (or even in each line of code). Profilers may also record memory allocations and deallocations to identify how memory is used by our program over time. After execution is complete, we can then *analyse* this data to draw conclusions about how well our code performs and identify regions where improvements can be made.

Effective code profiling and performance optimisation necessarily requires a detailed understanding of how the compiler translates C++ code into machine instructions as well as how the hardware of the computer operates. This section therefore only touches on a few basic optimisations we can look for in our code, but which can still offer significant gains.

10.1.1 When should we perform profiling?

We should avoid performing profiling on an incomplete or broken code. Improving the performance of incomplete code may lead to sub-optimal performance when the complete algorithm is implemented and therefore wasted effort. Furthermore, optimising code often makes it less readable and can obfuscate the high-level structure of the underlying algorithm making it more difficult to debug. Implementing optimisations may require intentional or inadvertent changes in the algorithms, so it is important to have a working reference implementation before starting, and ideally a set of input conditions which test all parts of the algorithm, with which to compare.

10.1.2 Oracle Developer Studio

To introduce the concepts and techniques for profiling and optimising code, we will use parts of the *Oracle Developer Studio* suite. This includes a command-line tool for collecting performance statistics and a graphical tool for examining and analysing the collected data.

These tools are currently only available on the *Remote Desktop Linux Environments*, `spitfire.ae.ic.ac.uk` and `hurricane.ae.ic.ac.uk`. For this part of the course, please use one of these machines.

To use *Oracle Developer Studio* tools, we need to make them available to us by enabling an environment module. From a terminal, run the command

```
module load dev-studio
```

We can test the tools are available to us using

```
which analyzer
```

If they are, this should print the full path to the executable `analyzer`, i.e.

```
/apps/dev-studio/12.6/bin/analyzer
```

If nothing is printed, there is a problem as the command `analyzer` cannot be found.

10.2 A test problem

To illustrate the concepts of performance optimisation, suppose we wish to compute a matrix $M[i,j]$ of size $N \times N$, for which the entries are given by:

$$M[i,j] = \sum_{k=0}^{K-1} \frac{N^{k/K}}{e^{k/K}} f(x) f(y),$$

where $x = i/N$, $y = j/N$, and

$$f(z) = \sin^2 z.$$

Our initial naive implementation is shown in Example 10.1 and quite closely follows the mathematical representation. We suppose the implementation is in a source code file called `perf.cpp`.

Our first step in optimising this code is to get a baseline measurement of how long it takes to run. We can do this using the Linux `time` command. For now, we make sure to turn off compiler optimisations and include debugging symbols which will be useful for us later.

Example 10.1: Code for implementing the test problem without any profiling or optimisations applied.

```cpp
#include <cmath>

double func(int i, int n) {
    double alpha = (double)i / double(n);
    return pow(sin(alpha), 2);
}

int main() {
    const int n = 10000;
    const int K = 10;
    double* matrix = new double[n*n];

    for (int k = 0; k < K; ++k) {
        for (int j = 0; j < n; ++j) {
            for (int i = 0; i < n; ++i) {
                matrix[i*n + j] += pow(n, (double)k/K) / exp((double)k/K)
                                * func(i, n) * func(j, n);
            }
        }
    }
}
```

```
g++ -O0 -g -o perf perf.cpp
time ./perf
```

This should produce an output similar to:

```
real    2m03.940s
user    2m03.340s
sys     0m0.600s
```

For our purposes, the `real` time is of interest.

10.3 Collecting profiling data

We now seek to better understand which aspects of our code are taking the longest and identify how we might improve the overall performance of the code.

To do this, we use the `collect` tool to accumulate performance measurements of our code.

```
collect -o initial.er ./perf
```

This runs our program and samples performance statistics as it runs. These data are stored in the directory `initial.er`.

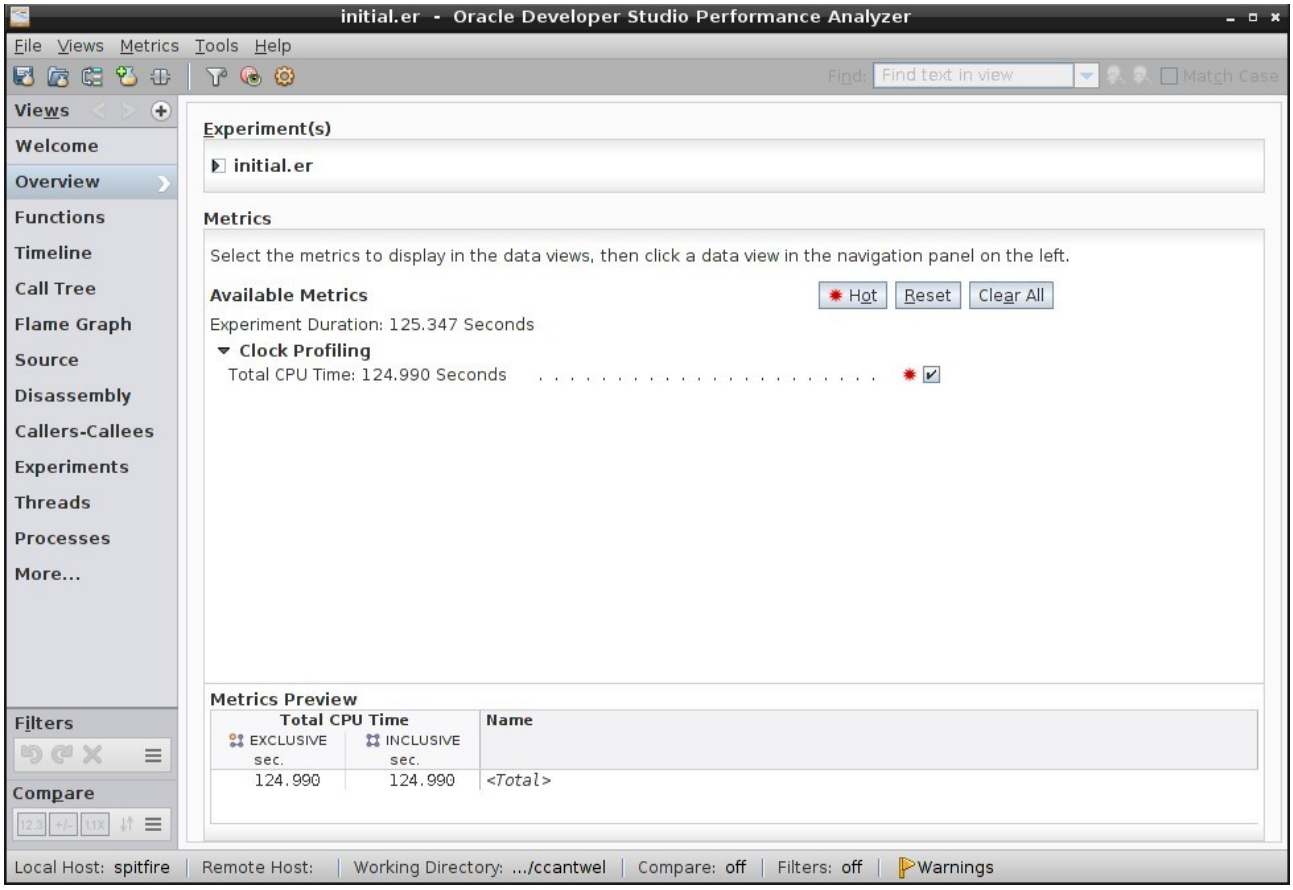


Figure 10.1: Overview display of the analyzer tool. The performance can be analysed using the different views on the left.



Note

There are a range of command-line arguments to the `collect` tool which can be used to control the type of information collected and the scope of the program to be analysed.

These options can be listed using `collect --help`, but we will not discuss them during this course.

10.4 Analysing the profiling data

To examine the profiling statistics we launch the *Performance Analyzer* tool

```
analyzer initial.er
```

A screenshot of the initial interface is shown in Figure 10.1. This shows an overview of the collected statistics. In this case, we have only performed the default clock profiling, which measures how long it takes the CPU to execute each command. Therefore, this is the only metric available.

The performance analyzer software has a number of ways of viewing the data, shown in the menu on the left hand side. This provides various granularities of analysis from the whole program execution (on the overview screen) to the timings for individual statements.

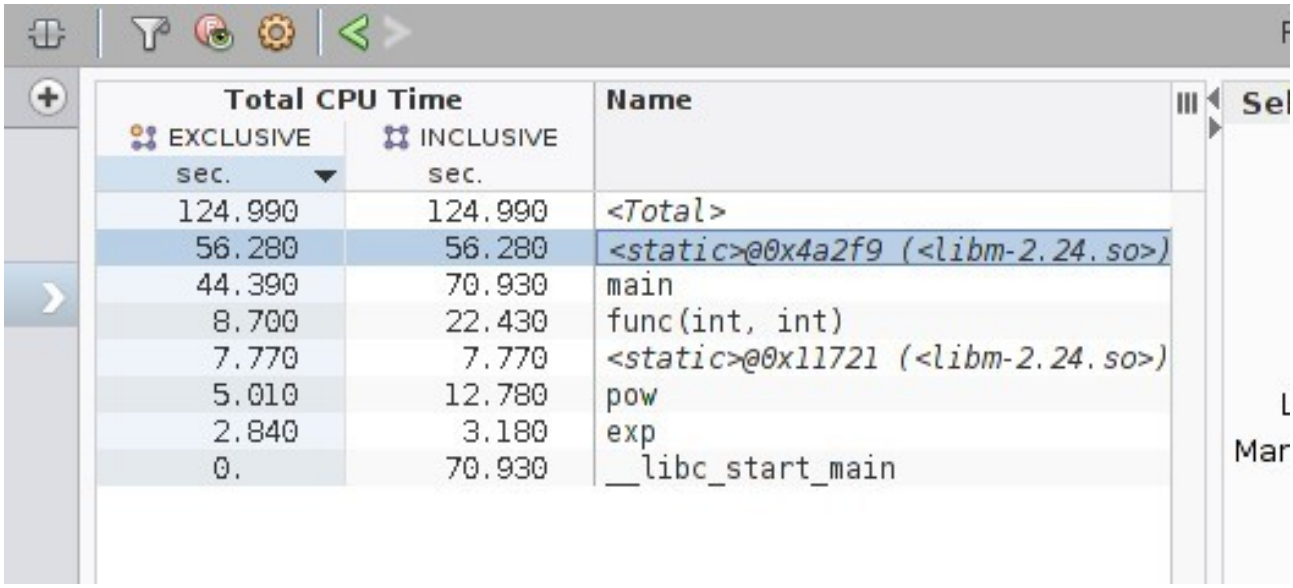


Figure 10.2: Analysing the cumulative cost of each function.

10.4.1 Functions and Call Tree

Our test program is quite simple with only a `main()` routine and one additional function. However, a good place to start is usually the *Functions* view. Here we can identify the functions in our program that we spend cumulatively the longest time executing. In our present case, this is the `main()` routine, as shown in Figure 10.2.

The *Call Tree* shows the same functions as a tree in the order in which they were called. This has the added advantage of showing the program path taken to the most expensive functions. Additionally, if a function is called from multiple places in the code, the function will appear multiple times in the tree with the total time of that function for each program path shown.

Double-clicking the `main()` function in either view takes us to the *Source* view.

10.4.2 Source

The source view shows an annotated form of our source code. Initially, the start of the function we double-clicked on in the *Functions* view is highlighted. By default, the numbers down the left side show the time (in seconds) that each line took to execute. This is *inclusive time*, so it also includes the time taken to execute any functions that are called. The term *exclusive time* refers to the time taken executing the fundamental operations on that line (e.g. arithmetic operations).

The source view for our test program can be seen in Figure 10.3. The yellow and orange highlighting indicates “hot” areas where the code is spending a high percentage of the runtime. The yellow on line 16 indicates a high *inclusive* time. This is not surprising since this line and the functions it calls incorporates all the complex mathematical operations in the program. In contrast, the orange indicates high *exclusive*. Line 4 is highlighted orange not because dividing two numbers is particularly expensive in itself, but because it is called a very large number of times!



Warning

For the source display to work, the code needs to be compiled with debugging symbols (using the compilers `-g` flag).

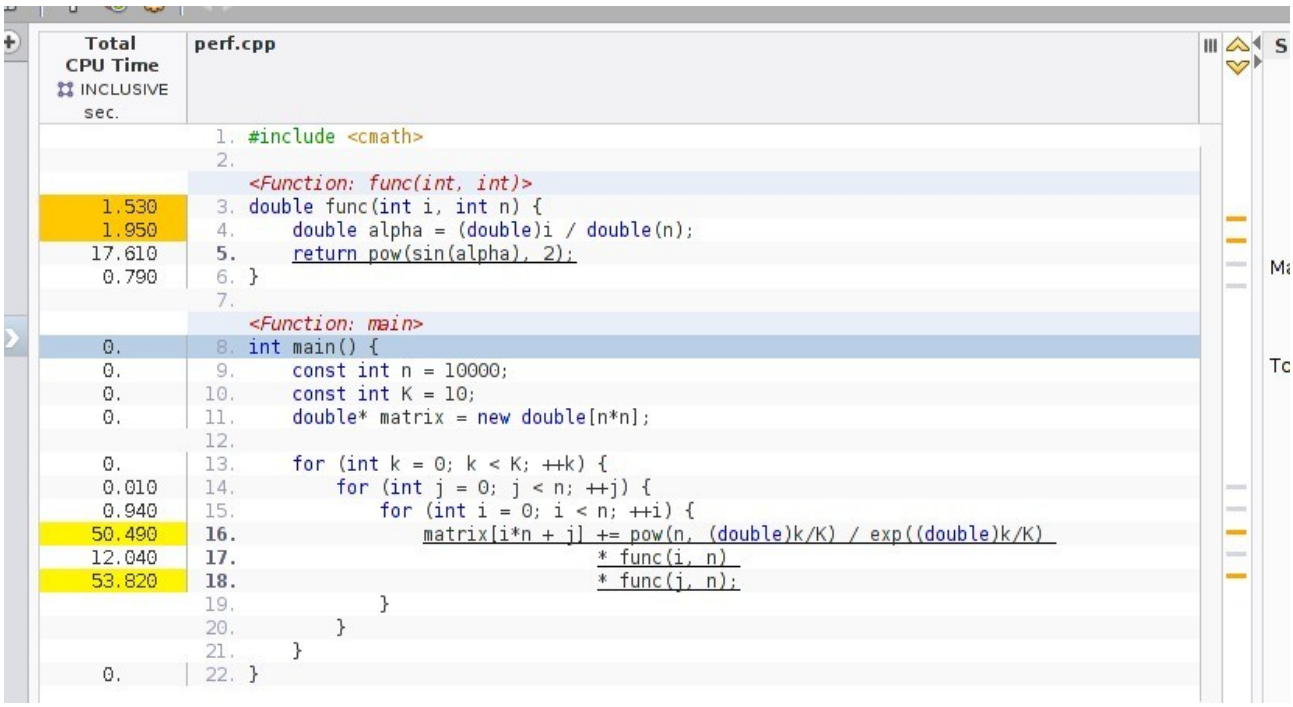


Figure 10.3: Source view showing inclusive times for each line of code.

10.4.3 Timeline

The final view we will discuss is the *Timeline*. This shows the performance of your code as a function of time. The coloured *stacks* show a backtrace of what was being executed during each sample. Selecting one of these will show the call stack on the right-hand side.

One often useful feature of this view is the ability to restrict the analysis to a portion of the code we are interested in. This is achieved by dragging a selection on the grey time axis bar at the top or bottom of the view.

10.5 Optimising Code

We now consider how to improve the performance of our code by discussing a few aspects we can look for. The use of each optimisation should be judged on a case-by-case basis. Applying optimisations for negligible performance return may just make the code more difficult to read and debug!

10.5.1 Inline simple functions

Often *inlining* a function – essentially substituting each call to a function with the body of the function – can be helpful in some cases to allow further optimisations.

In our case, `func()` is quite simple and is easily inlined, as shown in Example 10.2. We introduce two temporary variables inside the inner loop to hold the values previously held by `alpha` inside the function. This takes about 10 seconds off the runtime:

```
real    1m50.905s
user    1m50.292s
sys     0m0.612s
```

The reason for this is that calling a function itself is not “free” and, at a minimum, requires a new

entry to be added to the call stack, and removed upon function completion. Since our function was called 2 billion times, this cost added up to about 10 seconds of runtime!

Example 10.2: Optimising code by inlining simple functions.

```
int main() {
    const int n = 10000;
    const int K = 10;
    double* matrix = new double[n*n];

    for (int k = 0; k < K; ++k) {
        for (int j = 0; j < n; ++j) {
            for (int i = 0; i < n; ++i) {
                double ai = (double)i / double(n);
                double aj = (double)j / double(n);
                matrix[i*n + j] += pow(n, (double)k/K) / exp((double)k/K)
                                * pow(sin(ai),2)
                                * pow(sin(aj),2);
            }
        }
    }
}
```

10.5.2 Simplify mathematical expressions

This is actually more of an algorithmic optimisation than a code optimisation. However, be careful to avoid introducing errors. In our particular example, the only simplification we could make is to avoid a division, by taking the exponential of $-k$.

```
matrix[i*n + j] += pow(n, (double)k/K) * exp(-(double)k/K)
                 * pow(sin(ai),2)
                 * pow(sin(aj),2);
```

Floating-point division is a more computationally intensive operation than multiplication, so finding ways to minimise the number of divisions can improve performance. This optimisation takes about 3 seconds off the run-time of our test program.

10.5.3 Pre-compute repeated expressions

If a sub-expression is repeated multiple times in a mathematical expression then pre-computing that sub-expression will reduce the overall computational cost. This is particularly effective for expressions computed within loops, as in our example. We can pre-compute `sin(ai)` and `sin(aj)` and subsequently avoid the call to `pow()`:

```
double sai = sin(ai);
double saj = sin(aj);
matrix[i*n + j] += pow(n, (double)k/K)*exp(-(double)k/K)
                * sai*sai
                * saj*saj;
```

Doing this cuts a further 16 seconds off the execution time:

```
real    1m31.761s
user    1m31.072s
sys     0m0.688s
```

10.5.4 Lift loop-independent expressions

An expression (or sub-expression) within a loop which is not dependent on the loop counter (directly or indirectly) can be lifted out of the loop to reduce the number of operations.

Example 10.3 shows our test problem implementation after lifting our common expressions. We have lifted the sub-expression `pow(n,K) * exp(-k)`, which is only dependent on the `k` counter out of the `i` and `j` loops. We have also lifted the calculation of `aj` and `saj` out of the inner `i` loop. Since both the sine and exponential functions are computationally costly, this significantly reduces our runtime by a further 49 seconds.

Example 10.3: Lifting loop-independent sub-expressions out of loops

```
int main() {
    const int n = 10000;
    const int K = 10;
    double* matrix = new double[n*n];

    for (int k = 0; k < K; ++k) {
        double coeff = pow(n,(double)k/K)*exp(-(double)k/K);
        for (int j = 0; j < n; ++j) {
            double aj = (double)j / double(n);
            double saj = sin(aj);
            for (int i = 0; i < n; ++i) {
                double ai = (double)i / double(n);
                double sai = sin(ai);
                matrix[i*n + j] += coeff
                                 * sai*sai
                                 * saj*saj;
            }
        }
    }
}
```

10.5.5 Cache locality and nested loops

When the CPU requests a data from memory, it is stored in a small memory on the CPU itself, called a *cache* (see Section 1.2 to remind yourself of the computer memory hierarchy). However, when we request a variable from memory, it does not fetch just one value but a block of values of a fixed size, called a *cache line*. Typically on current computers this is 64 bytes (or 8 `double` values).

Since accessing values already in cache is many times faster than retrieving the value from the main memory (which may take a few hundred clock cycles), restructuring our code to maximise the use of values already in cache, known as *cache locality*, will boost performance.

If we take a closer look at how we currently access our matrix array, we have interpreted it as a matrix with `i` indexing the rows and `j` indexing the columns, and data are stored row-by-row:

```
matrix[i*n + j] = ...
```

If we now consider how we *access* the matrix, we can see that the inner `i` loop changes the fastest:

```
for (int j = 0; j < n; ++j) {
   ...
   for (int i = 0; i < n; ++i) {
      ...
      matrix[i*n + j] = ...
   }
}
```

On the first iteration of the `i` loop, we must go to main memory to retrieve the value. An entire *cache line* is read which includes our required value, but also the next few values along the same row of the matrix. However, on the next iteration of the loop, we request the first value from the next row. Since this is not in cache, we must again go to main memory, and retrieve a cache line. By the time we return to the first row (on the next iteration of the `j` loop), those values we first read "for free" along that first row have probably been deleted from the cache.

Example 10.4 shows how, by reordering the loops, we can populate the matrix in a different order (by row, instead of by column) and improve cache locality. Now, when we access the second entry in the first row, this value is already in cache since it was next to the first value in the row in memory and consequently read as part of a cache line.

We have also moved the `k` loop to be the inner loop, so that we need only visit each matrix entry once. We reduce our runtime by a further 13 seconds.

Example 10.4: Reordering loops to improve cache locality.

```
int main() {
    const int n = 10000;
    const int K = 10;
    double* matrix = new double[n*n];

    for (int i = 0; i < n; ++i) {
        double ai = (double)i / double(n);
        double sai = sin(ai);
        for (int j = 0; j < n; ++j) {
            double aj = (double)j / double(n);
            double saj = sin(aj);
            for (int k = 0; k < K; ++k) {
                double coeff = pow(n,(double)k/K)*exp(-(double)k/K);
                matrix[i*n + j] += coeff
                                * sai*sai
                                * saj*saj;
            }
        }
    }
}
```

10.5.6 Vectorisation

As discussed in Section 1.1, modern CPUs support a form of data parallelism called vectorisation. If certain conditions are met regarding the instructions within a loop, and the data is organised in memory in a compatible way, the compiler can generate special instructions which allow the processor to execute multiple iterations of the loop concurrently.

The following code is highly amenable to vectorisation:

```
for (int i = 0; i < N; ++i) {
    f[i] = g[i] + h[i];
}
```

Code characteristics which can **prevent** vectorisation of a loop include:

- Too many nested loops
- Dependencies between iterations of the loop, for example:

```
for (int i = 1; i < N; ++i) {
```

```
        f[i] = f[i-1] + g[i];
    }
```

- Branching within the loop (e.g. conditional statements, function calls), for example:

```
for (int i = 1; i < N; ++i) {
    if (i > N / 2)
        f[i] = f[i-1] + g[i];
    else
        f[i] = f[i-1] + g[i];
}
```

- Memory indirection:

```
for (int i = 1; i < N; ++i) {
    f[i] = f[index[i]] + g[index[i]];
}
```

To enable vectorisation in the GCC compiler, we need the following flags:

```
g++ -O2 -ftree-vectorize
```

We can also ask the compiler to explain why loops were not vectorised using the following additional flag:

```
-fopt-info-vec-missed
```

This produces output similar to:

```
perf.cpp:14:23: missed: couldn't vectorize loop
perf.cpp:14:23: missed: not vectorized: multiple nested loops.
perf.cpp:15:27: missed: couldn't vectorize loop
perf.cpp:17:39: missed: statement clobbers memory: _3 = pow (3.0e+3, _2);
perf.cpp:16:31: missed: couldn't vectorize loop
perf.cpp:17:39: missed: statement clobbers memory: _3 = pow (3.0e+3, _2);
perf.cpp:12:36: missed: statement clobbers memory: _22 = operator new [] (72000000);
perf.cpp:17:39: missed: statement clobbers memory: _3 = pow (3.0e+3, _2);
perf.cpp:17:61: missed: statement clobbers memory: _4 = exp (_2);
```

The outer loop could not be vectorised because the loop nesting was too deep. The inner loops could not be vectorised because we have function calls within the loop.

10.5.7 Revisiting earlier optimisations

Our code has changed quite significantly from its original implementation. As we have applied each optimisation, our previous optimisations may no longer be effective, or there may be additional optimisations to make.

In this case, by changing the loop orderings, we can perform some additional loop-lifting optimisations (`sai*sai` and `saj*saj`). We also notice that the `k`-loop coefficients are recomputed for every matrix entry. Since `K` is very small compared with `n`, we could pre-compute all the values at the cost of a little extra memory.

Our final optimisation of the code is shown in Example 10.5. We have reduced our runtime from over 2 minutes down to just 8 seconds!

```
real    0m7.742s
user    0m7.136s
sys     0m0.604s
```

However, notice that the code does not now closely resemble the mathematical expression and debugging errors is probably more challenging. After performance-optimising the code, we should verify it produces the same result as the unoptimised code and add some useful documentation!

Example 10.5: Final optimisation of our test problem.

```
#include <cmath>

int main() {
    const int n = 10000;
    const int K = 10;
    double* matrix = new double[n*n];
    double* coeff = new double[K];

    int i, j, k, idx;
    double ai, aj, sai, saj;

    for (k = 0; k < K; ++k) {
        coeff[k] = pow(n,(double)k/K)*exp(-(double)k/K);
    }
    for (i = 0; i < n; ++i) {
        ai = (double)i / double(n);
        sai = sin(ai);
        sai *= sai;
        for (j = 0; j < n; ++j) {
            aj = (double)j / double(n);
            saj = sin(aj);
            saj *= saj * sai;
            idx = i*n + j;
            for (k = 0; k < K; ++k) {
                matrix[idx] += coeff[k] * saj;
            }
        }
    }
}
```

Exercises

Exercise 10.1

Using the corrected code from Exercise 3.1, use a profiler to analyse the code and identify areas of poor performance.

Optimise the code and compare the runtime with the non-optimised version.

Make sure your optimised code passes the tests you defined in Exercise 8.1.

Chapter 11

Clusters and Job Scheduling (Optional)

Objectives

In this chapter you will learn about:

- the basic architecture of a distributed computing cluster;
- queuing systems on modern computing clusters;
- how to write a job-submission script to run your code on a cluster in parallel.

11.1 Queuing Systems

Computing clusters are multi-user facilities which provide a large number of computers, or *nodes*, connected together using a network. Their design will vary depending on their intended use, but usually they can be considered as either high-performance systems, where the focus is on small numbers of large parallel jobs, or high-throughput systems, where the focus is on allowing a large number of small serial or trivially parallel jobs to run. Figure 11.1 illustrates the typical hardware configuration of a distributed computing cluster. Each node is typically of identical (or similar) specification with its own memory and CPUs. They are connected together using a standard network. For high-performance clusters, an additional high-speed network may be included which complies with the Infiniband standard and provides both high-throughput (bandwidth) and low-latency communication. This is particular important for parallel jobs using the Message Passing Interface (see Chapter 12).

Definition: Job

A job is a unit of work submitted to a computer cluster which executes one or more commands, and includes meta-data specifying the resource requirements performing the work.

Clusters are usually designed with a few dedicated *login* nodes along with a large number of *compute* nodes. Users only interact with the cluster through the login nodes. The compute nodes are used to actually perform computations and the user does not log into them directly. In order for this to work, clusters run software called a batch queuing system. Users submit jobs to the queuing

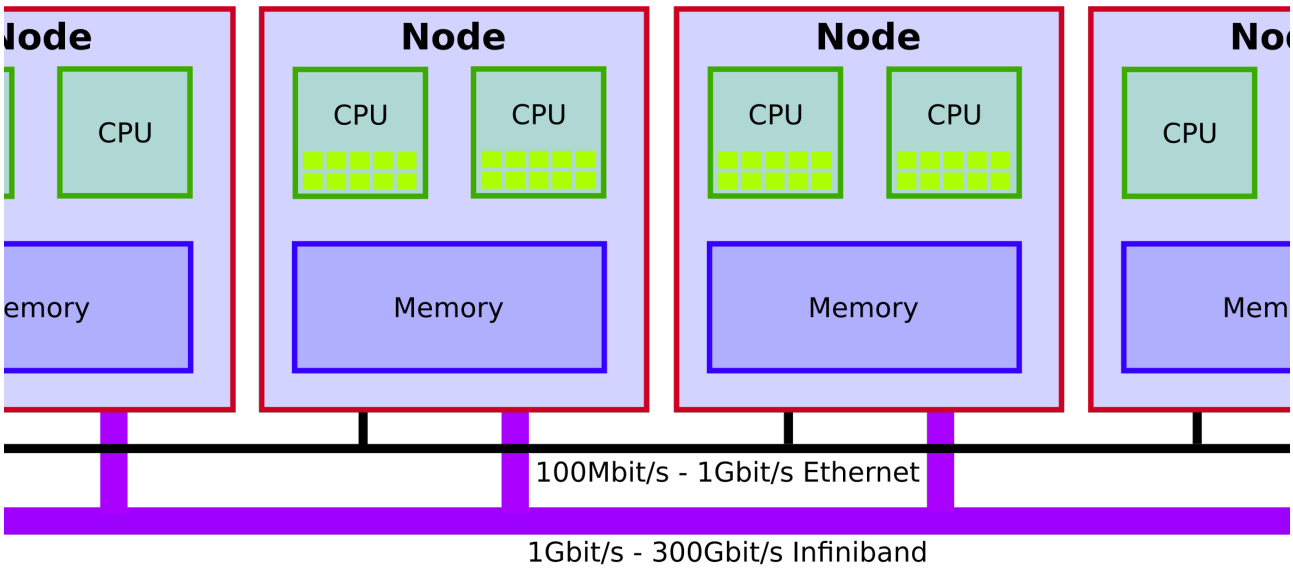


Figure 11.1: Distributed memory supercomputers connect many computer systems through a network. Infiniband is a specialist high-throughput low-latency interconnect which is used for sending messages.

system and the software handles allocating and running the job on one or more compute nodes.

Definition: Batch queuing

Job schedulers are software tools which manage job queueing on a cluster. They allocate jobs from multiple users to computing resources in an efficient and fair manner. They will schedule jobs from a queue as resources in a cluster become available.

Examples of queuing systems are Slurm and PBS. In both cases, the user specifies the resource required for their job using special statements at the top of the shell script. These may simply state how many CPUs are needed, how much memory each process requires and how long the job should take. However, it may also specify that the job requires particular hardware, such as a Graphics Processing Unit (GPU) or high-end network hardware.

Definition: Shell script

A shell script is a plain text file which specifies one or more commands to run, along with a command *interpreter*.

Often the command interpreter is the normal command-line interpreter (in the Linux environment the is called *bash*). It is specified using a single line at the top of the file as follows:

```
#!/bin/bash
```

A list of one or more commands then follow, which are written in exactly the same way as if they had been typed in a terminal window.

After writing the script, the user submits the script to the queueing system using a command-line tool. If sufficient resources are available to meet the requirements of the job the queuing system will start executing the job, otherwise it will wait in the queue until the necessary resources are available. Other constraints may be imposed to ensure fairness between users, such as limiting the number of jobs which can run concurrently, or resources which can be used, by any given user. The user can monitor the progress of their jobs in the queue and during their execution, as well as modify and cancel jobs, using other command-line tools.

Since the user does not interact directly with the computers which actually run the job, the output

from the commands run in the script are written to a file, typically in the directory from which the job was submitted.

11.2 Slurm Workload Manager

Slurm is an open-source and scalable workload management system for Linux clusters. More information can be found at https://slurm.schedmd.com/overview.html. To run a compute task on a Slurm-managed cluster, we must write a shell script and add resource specifications to the top of the script.

11.2.1 Slurm scripts

Slurm scripts are normal shell scripts with some additional metadata. To write a script to submit to Slurm, first write a shell script which performs the computational task required and test if it runs locally on your terminal.

A shell script might look like the following (the first line is the script interpreter).

`run-my-code.sh`:

```
#!/bin/bash
./mycode
```

To make the script executable, we must first change the permissions of the script:

```
chmod +x run-my-code.sh
```

We can then test the script executes our code as expected.

```
./run-my-code.sh
```

Next we modify this script to tell Slurm the resources required to run this job on the cluster, as shown in Example 11.1. We have changed the filename extension to `.slr` to make clear the script is intended for use with Slurm.

Example 11.1: A simple Slurm script for executing a serial code.
`run-my-code.slr`:

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --mem=8000MB
#SBATCH --time=06:00:00
#SBATCH --job-name=runmycode
#SBATCH --partition=medium
./mycode
```

The lines beginning with `#SBATCH` provide information to Slurm about our job and its needs.

- `#SBATCH --nodes=1`
  Specifies the number of nodes (computers) required for the job. Each node my have multiple CPUs, each with multiple cores.
- `#SBATCH --ntasks=1`

Specifies the total number of processes to run across all nodes (should match the total number of cores on the selected nodes).

- `#SBATCH --mem=8000MB`
  Specifies the total memory required *per node*.
- `#SBATCH --time=06:00:00`
  Specifies an **upper limit** on the time our job is expected to take to run. If it runs for longer, Slurm will kill it.
- `#SBATCH --job-name runmycode`
  Specifies a name for the job. By default the script filename will be used. If you submit many jobs concurrently, this is useful to keep track of which job is which.
- `#SBATCH --partition=bigqueue`
  Specifies the name of the partition (or queue) to submit the job to. Different queues have different resource limits. If the resources requested exceed the limits of the chosen queue, the job will be rejected.
- `./mycode`
  Our command to run.

To submit this script to the queuing system, we use the `sbatch` command.

```
sbatch run-my-code.slr
```

If slurm accepts the job, it will return a unique job number.

11.2.2 Parallel jobs

To run parallel jobs using MPI (see Chapter 12, we must run our code using the MPI wrapper (`mpiexec`). However, as the task will be run within the environment of the queueing system, we should not specify the number of processes to spawn; this is determined automatically by `mpiexec` from the queuing system. We might modify our script as shown in Example 11.2.

Example 11.2: A simple Slurm script for running a parallel job across two nodes.

```
#!/bin/bash
#SBATCH --nodes=2
#SBATCH --ntasks=32
#SBATCH --mem=128000MB
#SBATCH --time=06:00:00
#SBATCH --job-name=runmycode
#SBATCH --partition=medium
mpiexec ./mycode
```

In particular, we have changed the following:

- `#SBATCH --nodes=2`
  We will run the code in parallel (using MPI) across two nodes.
- `#SBATCH --ntasks=32`
  The total number of processes to run is 32, which will spawn 16 processes on each node.
- `#SBATCH --mem=128000MB`
  We have increased this to 128 GB as we are running 16 processes on each node (16 $\times$ 4 GB).

- `mpiexec ./mycode`
  Since this is a parallel job, our command to run must be prefixed with the MPI launcher. However, we do not specify the total number of processes. The launcher will get this information from Slurm.

11.2.3 Managing Slurm jobs

Viewing the queue

After submitting a job, we can monitor the queue using the `squeue` command. By default, this will print the list of currently running jobs. The output will look something like

```
JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
  212    medium  run-100    steve  R      15:36      1 linux1
```

Jobs associated with a given user can be shown with the `-u username` option, for example

```
squeue -u <username>
```

If your job is queued and not currently running, you can see the predicted start time by running

```
squeue --start
```

The `JOBID` column shows the unique ID of each job. The `PARTITION` column is the queue the job is in. The `ST` colume shows the state of the job. For example, `R` indicates the job is running.

Listing available queues

Slurm clusters may include one or more partitions. To see a summary of the available partitions on the cluster and their resource limits, we can run

```
sinfo -s
```

This command will print out a table similar to the following

```
PARTITION AVAIL  TIMELIMIT   NODES(A/I/O/T)  NODELIST
long         up 7-00:00:00          2/0/0/2  linux[1-2]
short*       up    1:00:00          3/3/0/6  hurricane,linux[1-4],spitfire
medium       up 1-00:00:00          3/1/0/4  linux[1-4]
```

From this we can see that there are three queues and the default queue, indicated by the asterisk, is the `short` queue. Each queue has a maximum time limit for jobs and the `NODES` column indicates how many nodes are active and inactive.

Cancelling jobs

To cancel a job use we can run

```
scancel <job-id>
```

11.3 Portable Batch System (PBS)

Portable Batch System (PBS) is another job scheduling service. The software was originally developed by NASA in the early 1990s and now has several variants. It uses the same principles as Slurm, whereby jobs are defined by shell scripts and submitted to the queuing system. It is used widely, including on the College HPC systems, so we include a brief summary of its usage here.

The script defined in Example 11.2 can be modified for use with PBS as shown in Example 11.3.

Example 11.3: A simple script for running a parallel code across two nodes using the Portable Batch System.
`run-my-code.pbs`:

```
#!/bin/bash
#PBS -l select=2:ncpus=16:mem=128000MB
#PBS -l walltime=06:00:00
#PBS -N runmycode
#PBS -q medium
mpiexec ./mycode
```

Note: We modified the file extension for no other purpose than to make clear it is a PBS job submission script.

The first `-l` line provides the hardware resource specification. In this example, we request two nodes, each with 16 cores and a 128 GB of memory per node. The second `-l` line specifies the maximum runtime of the job. As with Slurm, we can name our job and specify which queue to use.

11.4 PBS Commands

Job scripts are submitted to PBS using the `qsub` command:

```
qsub run-my-code.pbs
```

This will return a unique identifier (say `<job-id>`) for your job.

To view details of jobs and queues we use the `qstat` command. e.g.

- `qstat`: See list of queued jobs.
- `qstat -f <job-id>`: View details of a job.
- `qstat -q`: List available queues and their restrictions.
- `qstat -Q`: List available queues and their status.
- `qstat -Qf <queue-name>`: List details of a queue.

To remove/cancel a job (you own), use

```
qdel <job-id>
```

To modify properties of a job (after submission to the queue) use `qalter`, e.g.

```
qalter -N mynewjobname <job-id>
qalter -l walltime=08:00:00 <job-id>
```

Note: Resource requirements can only be changed before jobs starts.

Exercises

Exercise 11.1

Choose one of the programs you have written so far and write a script to execute it on a Slurm cluster. Remember to compile your code first.

1. Think about suitable resource requirements for your chosen program and specify them in the script.
2. Give your job a suitable name.
3. Submit the job to the `short` queue.

Hint: Since the cluster runs jobs in a non-interactive environment, choose a program which does not require terminal input from the user (i.e. it does not use `cin`).

Part IV

Parallel Programming

Chapter 12

Distributed Parallel Programming with MPI

Definition: Parallel Computing

Parallel Computing is the process of dividing up a large computation into smaller computations which can be undertaken concurrently.

A parallel programming model is a means of incorporating parallel computation into an algorithm.

The design and architecture of the types of systems we use with MPI programs is given in Appendix 1.3.

There are two main types of parallel programming models: *message passing* and *shared memory*.

Under the message-passing paradigm, multiple processes work on different parts of a problem in separate memory spaces and exchange messages with each other to propagate information and coordinate the solution. This approach leads to a flexible design where problems can be parallelised not only across multiple processors within a single machine, but across multiple machines connected together by a network. Typically, the programmer writes a single program which is executed multiple times on each participating machine. This is known as *Single Program, Multiple Data*, or SPMD.

In contrast the shared memory approach spawns multiple threads of execution within a single process and memory space. The threads share between them the memory allocated to the process. As a result this paradigm is restricted to a single machine.

In this section, we will explore the Message Passing Interface (MPI), which is a standard for exchanging messages in parallel programs. In Chapter 13, we look at OpenMP, which is an approach for shared-memory parallelism.

12.1 MPI Basics

When we run a program written using MPI, multiple copies of our code are executed. Each of these processes is given a unique ID, or *rank*. The processes can "communicate" with each other by calling MPI routines to send and receive messages. There are two types of communication routines: point-to-point and collective. Point-to-Point communication functions are those where messages are exchanged between a pair of processes only. These routines may wait for the other process to acknowledge receipt of the message, or could just "send and forget". Collective communication occurs between more than two processes, often between all processes. MPI routines have the form

```
error = MPI_Xxxxx(argument, ...);
```

and are defined in the `mpi.h` header file.

12.1.1 Compiling and Running

To write a parallel program using MPI requires a few changes to the design of the C++ code and how it is run.

When writing our code, we will need to use the routines provided by the MPI library. We should therefore include the MPI header file

```
#include <mpi.h>
```

at the top of our program to make the `MPI_Xxxxx` routines available.

To compile our MPI code, we should use `mpicxx` in place of `g++`:

```
mpicxx hello.cpp -o hello
```

The `mpicxx` program is actually just a wrapper script around our normal `g++` compiler, which automatically adds all the compiler flags (libraries, library locations, directories containing the headers) necessary to compile the program to use MPI.



Tip

To see which compiler options the `mpicxx` is providing, run:
`mpicxx -show` from a terminal.

Once our program is compiled, we must then run it using the `mpiexec` command. For example,

```
mpiexec -np 2 ./hello
```

This program handles launching the correct number of copies of our program (in this case two), and setting up the communication between them.

Now that we know how to compile and run a program with MPI, we look more closely at how to write a parallel program.

12.1.2 Initialising and Finalising

When we launch our program with `mpiexec`, each copy of our code is initially running in isolation; it is completely unaware of the other copies of our program. To allow the processes to communicate with each other we must first initialise MPI by calling the function

```
int MPI_Init(int* argc, char** argv[]);
```

The values of `argc` and `argv` hold command-line arguments which are given to our program by `mpiexec`. We must therefore modify the definition of our `main` function to be able to accept these. Instead of the parameter-less function we have declared so far, we add two additional parameters:

```
int main(int argc, char* argv[])
```

The first, `argc`, holds the number of arguments passed on the command-line, while the second is an *array of character arrays*. These must be passed **by reference** to `MPI_Init`, as shown in Example 12.1.



Note

We briefly met command-line arguments already in Exercise 2.3.

Also note that the first argument in the `argv` list is the name of the executable itself.

At the end of the program the parallel execution environment must be terminated by calling:

```
int MPI_Finalize();
```

No other MPI calls can be made after this statement.

Example 12.1: Initialising and Finalising an MPI program.

```
#include <iostream>
#include <mpi.h>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);                    // Initialise MPI
    std::cout << "Hello World" << std::endl;
    MPI_Finalize();                            // Finalize MPI
}
```

12.1.3 MPI Return values

All MPI routines return an error code as an integer. The MPI header file `mpi.h` includes convenient preprocessor macros for these values:

- `MPI_SUCCESS`: The MPI function completed successfully.
- `MPI_ERR_COMM`: Invalid communicator.
- `MPI_ERR_OTHER`: Another error occurred.

We should ideally always test the value after any MPI calls to check that the routine completed successfully. An example of doing this is shown in Example 12.2

Example 12.2: Testing MPI return values

```
int retval;
retval = MPI_Init(&argc, &argv);
if (retval != MPI_SUCCESS) {
    cout << "An error occurred initialising MPI" << endl;
}
```

12.2 MPI communicators

Definition: MPI Communicator

A *communicator* is a fixed ordered set of ranks which can be used to define (sub)sets of processes which participate in a communication operation.

A single communicator which contains all of the process ranks spawned by `mpiexec` is always created at the beginning of the program for you by MPI. This communicator is called `MPI_COMM_WORLD` and cannot be modified or removed. For many straightforward programs, this is the only communicator you will need. However, sometimes a subset of the processes need to perform a collective communication, which requires us to create a new communicator containing only these processes (discussed in Section 12.5). Each communicator has a *size* and each process within the communicator has a *rank*, with respect to that communicator.

12.2.1 Process Rank and Size

In partitioning a problem among multiple processes, each process needs to be able to both identify its own rank and the total number of ranks. With this information, we can programmatically and unambiguously decide which rank should work on which part of the problem. The number of processes can be retrieved using

```
int MPI_Comm_size( MPI_Comm comm, int *size );
```

and the rank of a process can be retrieved using

```
int MPI_Comm_rank( MPI_Comm comm, int *rank );
```

An example of their use is given in Example 12.3.

Example 12.3: Retrieving a communicator's rank and size.

```
int rank, size, retval_rank, retval_size;
retval_rank = MPI_Comm_rank(MPI_COMM_WORLD, &rank); // zero-based
retval_size = MPI_Comm_size(MPI_COMM_WORLD, &size);
if (retval_rank == MPI_ERR_COMM || retval_size == MPI_ERR_COMM) {
    std::cout << "Invalid communicator" << std::endl;
    return 1;
}
cout << "I am process " << rank + 1 << " of " << size << endl;
```

12.3 Point-to-point messages

In this section we look at how we can send messages between two processes. An MPI *message* contains a number of pieces of information:

- Sender rank
- Destination rank
- Message datatype
- Message size
- Message location (in memory)
- Message tag

For each message we wish to send, we must specify all of these items. Table 12.1 shows the different MPI datatypes and their corresponding C++ data types. These cover all the common types we would use in practice.

Table 12.1: MPI datatypes and their corresponding C++ data types

MPI Datatype	C++ Datatype
`MPI_CHAR`	`signed char`
`MPI_SHORT`	`signed short`
`MPI_INT`	`signed int`
`MPI_LONG`	`signed long`
`MPI_UNSIGNED_*`	`unsigned *` (as above)
`MPI_FLOAT`	`float`
`MPI_DOUBLE`	`double`
`MPI_LONG_DOUBLE`	`long double`

12.3.1 Sending messages

The first function we consider is `MPI_Send()`, which has the function prototype:

```
int MPI_Send(void *buf, int cnt, MPI_Datatype type, int dest,
             int tag, MPI_Comm comm);
```

This sends a single message from the current process, to the process with rank `dest`, with respect to the communicator `comm`. The message will send an array of size `cnt` (count), starting at the memory location pointed to by `buf`, which has elements of datatype `type`. If we wish, we can specify an integer `tag` to allow our processes to distinguish between the many messages they may receive.

An example of using `MPI_Send()` is given in Example 12.4

12.3.2 Receiving messages

A message sent by another process using `MPI_Send()`, with this process as its destination, can be received using the function `MPI_Recv()`. This has prototype

```
int MPI_Recv(void *buf, int cnt, MPI_Datatype type, int src,
             int tag, MPI_Comm comm, MPI_Status *stat);
```

Now `buf` points to an array in which we want the received message data to be placed. We must have previously allocated this to be the correct size, so the current process must have some prior idea of what data it will receive. We specify the rank from which we expect to receive a message with `src`. The `MPI_Recv()` function will wait until the message is received before returning control to the application. If no message is received (e.g. `src` was specified incorrectly), the routine will wait indefinitely!

Finally, the `stat` argument is a status field, which we often ignore by passing the argument `MPI_STATUS_IGNORE`.

An example of using `MPI_Recv()` is given in Example 12.4

12.3.3 Buffering, Blocking and Deadlocks

In Example 12.4, we intentionally ordered the send and receive commands so that they match. When the first process sends, the other receives, and when the second process sends, the first receives. We can see this more clearly if we explicitly write out which statements were executed on each process:

Example 12.4: Exchanging two numbers on two processes

```cpp
#include <iostream>
using namespace std;

#include <mpi.h>

int main(int argc, char* argv[]) {
    int a = 0;
    int b = 0;
    int rank, size, retval_rank, retval_size;
    MPI_Init(&argc, &argv);
    retval_rank = MPI_Comm_rank(MPI_COMM_WORLD, &rank); // zero-based
    retval_size = MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (retval_rank == MPI_ERR_COMM || retval_size == MPI_ERR_COMM) {
        cout << "Invalid communicator" << endl;
        return 1;
    }

    if (rank == 0) {
        a = 2;
        // Send 'a' to process rank 1
        MPI_Send(&a, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        // Receive 'b' from process rank 1
        MPI_Recv(&b, 1, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    else {
        b = 3;
        // Receive 'a' from process rank 0
        MPI_Recv(&a, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        // Send 'b' to process rank 0
        MPI_Send(&b, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }
    cout << "Rank: " << rank << "  a=" << a << ", b=" << b << endl;

    MPI_Finalize();
    return 0;
}
```

Process 0:

```cpp
int a = 0;
int b = 0;
a = 2;
MPI_Send(&a, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
MPI_Recv(&b, 1, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
cout << "Rank: " << rank << "a=" << a << ", b=" << b << endl;
```

Process 1:

```cpp
int a = 0;
int b = 0;
b = 3;
MPI_Recv(&a, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Send(&b, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
cout << "Rank: " << rank << "a=" << a << ", b=" << b << endl;
```

Definition: Blocking and non-blocking communication

In *blocking* communication, the MPI functions wait until the processes participation in the communication is complete before returning.
In *non-blocking* communication, the functions return immediately and MPI continues processing the communication in the background.

`MPI_Send()` and `MPI_Recv()` are used to perform *blocking* communication. The choice of whether to use blocking or non-blocking communications rests with us as a the programmer and will depend on the particular algorithm being implemented.

In Example 12.4, we might expect that if instead we placed the `MPI_Send()` call first **on both processes**, our code would wait indefinitely since both processes are waiting for the other process to receive the message. In this case, it actually will still run successfully. This is because the particular MPI implementation we are using (OpenMPI) *buffers* our message internally before sending it. Once it has copied the data into the buffer, the processes role in the communication is still complete and it can continue to the `MPI_Recv()` call. Other implementations may have slightly different behaviour for `MPI_Send()`.

However, if we were to place the `MPI_Recv()` first on both processes, our program would wait indefinitely (or "hang"). This is known as *deadlock*.

Definition: Deadlock

A deadlock occurs when one or more processes block indefinitely waiting on the communication subsystem.

Deadlock is one of the perils of parallel programming using MPI. There can be many causes. Some examples include:

- An `MPI_Recv()` without a matching `MPI_Send()`
- Incorrect ordering of MPI communication calls
- Not all processes in a communicator calling the same collective communication operation.

Sometimes deadlock can occur sporadically on different executions of the same program. This is usually down to asynchronous communication and slight timing differences between the processes. To ensure a reproducible behaviour, we could use *synchronous* communication.

Definition: Synchronous and Asynchronous communication

In *synchronous* communication, the sender process blocks until the message has been received.

In *asynchronous* communication, the sender process continues once the message has been sent.

However, this will cause a reduction in performance as the sending processes may be unnecessarily waiting for the communication to complete. A compromise is to use synchronous communication while debugging to eliminate potential deadlocks, and switch to asynchronous communication when your code is tested.

12.3.4 Synchronous and Non-blocking Send/Receive

The synchronous form of `MPI_Send()` is `MPI_Ssend()`,

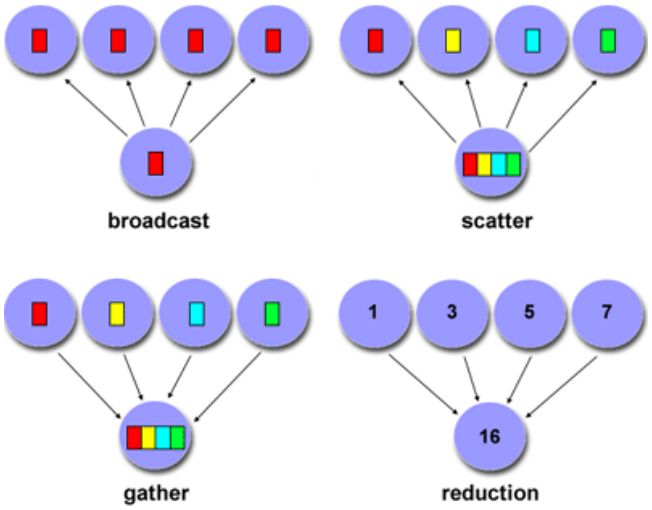


Figure 12.1: The four fundamental types of collective operation in MPI.

```
int MPI_Ssend(const void *buf, int count, MPI_Datatype datatype,
              int dest, int tag, MPI_Comm comm)
```

This will block execution from continuing until the message has been received successfully. There is also an explicitly non-blocking form of `MPI_Send()`, called `MPI_Isend()`,

```
int MPI_Isend(const void *buf, int count, MPI_Datatype datatype,
              int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

This will return immediately. The *request handle*, `request`, can be used to check at a later point if the message has been sent yet and `buf` can be reused, waiting otherwise. We would do this using a call to `MPI_Wait()`,

```
int MPI_Wait(MPI_Request *request, MPI_Status *status)
```

This has the advantage of allowing our program to do some other (unrelated) work while the communication is taking place, thereby improving efficiency. Similar versions of `MPI_Recv()` are also available.

12.4 Collective operations

Definition: Collective operation

A collective operation is a process to which all processes in a communicator contribute to simultaneously.

It is necessary that ALL processes in the communicator participate or deadlock will occur. Most collective operations are blocking, meaning that they necessarily *synchronise* all processes to the same point in the code.



Warning

Blocking/Non-blocking In the MPI-2 standard, all collective operations block until complete. However, the recently published MPI-3 standard supports some non-blocking collective operations.

The fundamental types of collective operations supported by MPI are illustrated in Figure 12.1. There are a number of variations of these. For example, a reduction can return the result to just one process, or to all processes. Some of the more common operations are listed below:

```
// Synchronise processes
int MPI_Barrier(MPI_Comm comm);
// Broadcast copies data from process 'root'
int MPI_Bcast( void *buffer, int count, MPI_Datatype datatype,
               int root, MPI_Comm comm )
// Distribute data among processes from process 'root'
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
               MPI_Comm comm)
// Gather data from processes to process 'root'
int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
               MPI_Comm comm)
// Reduce data using 'op' onto process 'root'
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
// Reduce data using 'op' onto all processes
int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
// Each process sends a chunk of data to each other process
int MPI_Alltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                 void *recvbuf, int recvcount, MPI_Datatype recvtype,
                 MPI_Comm comm)
```

The reduction routines above introduce another parameter of type `MPI_Op`. This specifies the type of operation to be used to combine the contributions from the different processes. Valid values include `MPI_MAX`, `MPI_MIN`, `MPI_SUM`, `MPI_PROD`.

We illustrate the use of collective operations with an example of computing a dot-product in parallel. This is shown in Example 12.5. We assume that the size of the global vector, `n`, is divisible by the number of processes, `p`. Each process works with a chunk of size `loc_n` of the global vectors `x` and `y`. For the purpose of this example, we have filled them with random numbers. In lines 23-26, we compute the dot product for the local chunk on each process. Remember, each process is executing these same lines of code, but on different data.

The collective operation appears in lines 31-32, where we gather the local contributions of the dot product, `loc_dot`, into an array on the root process. On the other processes, this array is never referenced, so we only allocate the array (of size `p`) on the root process. Finally, we sum together the local dot products to calculate the full dot product (but only on the root process).

The `MPI_Gather()` routine in Example 12.5 offered slightly more flexibility than we needed. The `MPI_Reduce()` function would be better suited to this particular task. We could replace lines 28-40 with:

```
double dot;
MPI_Reduce(&loc_dot,&dot,1,MPI_DOUBLE,MPI_SUM,0,MPI_COMM_WORLD);
if (my_rank == 0) {                    // First process computes result
    cout << "Dot product: " << dot << endl;
}
```

The result is only recorded on process 0, since we only wish to print the result once. If we needed the result on all processes for further calculations, we might instead use `MPI_Allreduce()`:

```
MPI_Allreduce(&loc_dot,&dot,1,MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
```

Example 12.5: Dot product using MPI

```
#include <iostream>
using namespace std;

#include <mpi.h>

int main(int argc, char* argv[]) {
    int my_rank, p, i, k;
    MPI_Init(&argc, &argv);                  // Initialise MPI program
    MPI_Comm_rank(MPI_COMM_WORLD,&my_rank); // Get rank
    MPI_Comm_size(MPI_COMM_WORLD,&p);        // Get number of procs

    const int n = 64;
    int loc_n = n/p;                         // Assume p divides n

    double* x = new double[loc_n];           // Only allocate local chunk
    double* y = new double[loc_n];
    srand(time(0)+my_rank);                  // Seed using time and rank
    for (i = 0; i < loc_n; ++i) {            // Just populate with random
        x[i] = double(rand())/RAND_MAX;      // numbers for this example
        y[i] = double(rand())/RAND_MAX;
    }

    double loc_dot = 0.0;                    // Local contribution
    for (i = 0; i < loc_n; i++) {
        loc_dot += x[i]*y[i];
    }

    double* loc_dots;
    if (my_rank == 0) loc_dots = new double[p];

    MPI_Gather(&loc_dot,1,MPI_DOUBLE,loc_dots,1,MPI_DOUBLE,0,
                  MPI_COMM_WORLD);           // Collate local contributions

    if (my_rank == 0) {                      // First process computes result
        double dot = 0.0;
        for (k = 0; k < p; k++) {
            dot += loc_dots[k];
        }
        cout << "Dot product: " << dot << endl;
    }

    MPI_Finalize();                          // Finish MPI program
}
```

12.5 Creating communicators

For many parallel applications `MPI_COMM_WORLD` is sufficient. However, in some more complex applications it is useful to allow subsets of processes to communicate independently of others.

We can partition an existing communicator (e.g. `MPI_COMM_WORLD`) using the `MPI_Comm_split()` routine.

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,
                   MPI_Comm *newcomm);
```

All processes in `comm` which supply the same value for `color` will be assigned to the same communicator. The `key` argument is used to determine the ordering of processes in the communicator.

When we have finished with our communicator, we should free it using

```
int MPI_Comm_free(MPI_Comm *comm);
```

Example 12.6 shows the use of `MPI_Comm_split()` to create two communicators: one containing all the even ranks in `MPI_COMM_WORLD`, with the other containing all the odd ranks.

Example 12.6: MPI_Comm_split

```
// Get the rank and size in the original communicator
int world_rank, world_size;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
MPI_Comm_size(MPI_COMM_WORLD, &world_size);

int color = world_rank % 2; // Determine odd (1) or even (0)

// Split communicator as odd/even. Use original rank for ordering.
MPI_Comm oddeven_comm;
MPI_Comm_split(MPI_COMM_WORLD, color, world_rank, &oddeven_comm);

int oddeven_rank, oddeven_size;
MPI_Comm_rank(oddeven_comm, &oddeven_rank);
MPI_Comm_size(oddeven_comm, &oddeven_size);

cout << "World: " << world_rank << "/" << world_size
     << "Odd/even: " << oddeven_rank << "/" << oddeven_size << endl;

MPI_Comm_free(&oddeven_comm);
```

12.6 MPI Groups

Definition: MPI Group

An MPI group contains a set of ranks which can be manipulated locally and used to create communicators. It is not itself a communicator and therefore cannot be used directly for communication between its constituent ranks.

We can start by creating a group from an existing communicator using the routine `MPI_Comm_group()`:

```
int MPI_Comm_group(MPI_Comm comm, MPI_Group *group);
```

Groups are very similar in concept to mathematical sets. We can therefore perform a range of standard set operations on them, including

```
// Extract selected members of a group
int MPI_Group_incl(MPI_Group group, int n, const int ranks[],
                   MPI_Group *newgroup);
// Intersection of two groups
int MPI_Group_intersection(MPI_Group group1, MPI_Group group2,
                           MPI_Group *newgroup);
// Union of two groups
int MPI_Group_union(MPI_Group group1, MPI_Group group2,
                    MPI_Group *newgroup);
```

Finally, when we no longer need a group, we should free it.

```
int MPI_Group_free(MPI_Group *group);
```

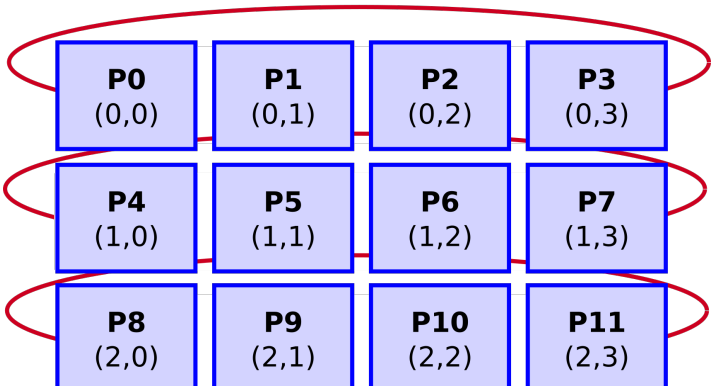


Figure 12.2: Organising processes as a 2D Cartesian grid.

Groups allow manipulation of sets of ranks. At some point, we will probably want to be able to communicate between those ranks and so we must create a communicator from the group. This is achieved using the `MPI_Comm_create()` function:

```
int MPI_Comm_create(MPI_Comm comm, MPI_Group group, int tag,
MPI_Comm * newcomm);
```

For those ranks not belonging to the group, `newcomm` will be null.

Example 12.7 shows how to generate a group containing prime-numbered ranks and then create a communicator for those ranks.

12.7 Virtual Topologies

Definition: Virtual Topology

A virtual topology is a method of associating an addressing scheme to the processes within an MPI group. It has no relation to the physical topology of the network.

A Cartesian topology organises the processes as a multi-dimensional grid. Each process has a unique tuple of coordinates. This is illustrated in Figure 12.2. A Cartesian grid topology is created using the `MPI_Cart_create()` routine:

```
int MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[],
                    const int periods[], int reorder,
                    MPI_Comm *comm_cart);
```

`ndims` is the number of dimensions in the grid (e.g. 2). `dims[]` is an array of sizes for each dimension. `periods[]` is an array specifying if grid is periodic. `reorder` allow ranks to be reordered (=1), or not (=0).

To get the tuple which represents the coordinates of a process, we use the `MPI_Cart_coords()` routine:

```
int MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims,
                    int coords[]);
```

This will compute the coordinates of the `rank` in communicator `comm` and store them in a (pre-allocated) array `coords`.

Often, we might want to know the rank of a process relative to another rank in the Cartesian grid. For this, we can use the `MPI_Cart_shift()` routine:

```
int MPI_Cart_shift(MPI_Comm comm, int direction, int disp,
int *rank_source, int *rank_dest);
```

Example 12.7: Manipulating MPI Groups (requires 16 MPI ranks).

```cpp
#include <iostream>
#include <mpi.h>

int main(int argc, char* argv[]) {

    MPI_Init(&argc, &argv);

    // Get the rank and size in the original communicator
    int world_rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    if (world_size != 16) {
        if (world_rank == 0) {
            std::cout << "This example requires 16 ranks." << std::endl;
        }
        MPI_Finalize();
        return 1;
    }

    // Get the group of processes in MPI_COMM_WORLD
    MPI_Group world_group;
    MPI_Comm_group(MPI_COMM_WORLD, &world_group);

    // We are going to select 7 ranks
    const int n = 7;

    // The ranks we select are those which are prime numbers
    int ranks[n] = {1, 2, 3, 5, 7, 11, 13};

    // Construct a group containing all of the prime ranks in
    // the world_group
    MPI_Group prime_group;
    MPI_Group_incl(world_group, n, ranks, &prime_group);

    // Create a new communicator based on the group
    MPI_Comm prime_comm;
    MPI_Comm_create(MPI_COMM_WORLD, prime_group, &prime_comm);

    int prime_rank = -1, prime_size = -1;
    if (MPI_COMM_NULL != prime_comm) {
        MPI_Comm_rank(prime_comm, &prime_rank);
        MPI_Comm_size(prime_comm, &prime_size);
    }

    // -1 indicates this world rank is not present in the prime communicator.
    std::cout << "World: " << world_rank << "/" << world_size
              << "Prime: " << prime_rank << "/" << prime_size << std::endl;

    MPI_Finalize();
    return 0;
}
```

`rank_dest` is the process at given displacement from current rank while `rank_source` is the process which, under the displacement, would get to the current rank.

Finally, we can also extract a sub-grid from a Cartesian grid using `MPI_Cart_sub()`:

```
int MPI_Cart_sub(MPI_Comm comm, const int remain_dims[],
MPI_Comm *newcomm);
```

In this routine, `remain_dims[]` indicates if each dimension is in the new subgrid.

Virtual topologies can be a very powerful tool for certain applications which lend themselves to the ranks being organised this way. Example 12.8 shows a simple usage of the above routines.

Example 12.8: Working with Cartesian topologies.

```
#include <iostream>
#include <mpi.h>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    if (world_size != 12) {
        if (world_rank == 0) {
            std::cout << "This example requires 12 ranks." << std::endl;
        }
        MPI_Finalize();
        return 1;
    }

    MPI_Comm mygrid;
    const int dims = 2;
    int sizes[dims] = {3, 4};
    int periods[dims] = {0, 1};
    int reorder = 1;
    MPI_Cart_create(MPI_COMM_WORLD, dims, sizes, periods,
                    reorder, &mygrid);

    MPI_Comm myrowcomm, mycolcomm;
    int coords[dims];
    int keep[dims];

    MPI_Comm_rank(mygrid, &mygrid_rank);
    MPI_Cart_coords(mygrid, mygrid_rank, dims, coords);

    keep[0] = 0;
    keep[1] = 1; // keep rows in subgrid
    MPI_Cart_sub(mygrid, keep, &myrowcomm);

    keep[0] = 1; // keep columns in subgrid
    keep[1] = 0;
    MPI_Cart_sub(mygrid, keep, &mycolcomm);

    MPI_Finalize();
    return 0;
}
```

Exercises

Exercise 12.1

1. Write a parallel MPI program which reads an integer from the user and distributes that value to all the other processes. All processes should then print their rank and the value entered.

   Remember, to use MPI routines, you will need to `#include <mpi.h>`.

2. Compile and link your code using the `mpicxx` command instead of `g++`.

3. Run your code in parallel (using the `mpiexec` command) for different numbers of processes (up to 16). Verify each process has printed its rank and the number entered.

Exercise 12.2

Write a parallel MPI program which reads a number from the user (on rank 0) and passes it around all other processes in a ring in order of their rank. Each process should print out the value when it receives it. Take care to avoid deadlock!

Why would using non-blocking sends and receives **not** be appropriate here?

Exercise 12.3

Write a parallel MPI program which generates two vectors of 1024 random floating-point numbers, each distributed equally across all processes (use $2^n <= 32$ processes).

Combine BLAS and MPI functions to calculate the dot-product of the two complete vectors and the norm of each vector, printing the results on the root process only.

Note: While BLAS does not itself support parallel operations, it can (should) be used to perform linear algebra operations locally, coupled with MPI calls to exchange information where necessary.

Exercise 12.4

The series expansion of π in Exercise 1.2 can also be expressed as the integral:

$$\frac{\pi}{4} = \int_0^1 \frac{1}{1+x^2}\,\mathrm{d}x$$

Write a parallel MPI program which approximates this integral using the trapezium rule. The program should request the number of intervals to use from the user. Take care of the case when the number of processes does not exactly divide the number of intervals!

Exercise 12.5

Write a parallel MPI program which generates a random matrix and vector, distributed evenly amongst the processes, and performs a matrix-vector multiplication in parallel, printing the result on the root process. Use a matrix/vector size of 2^n and run on 2^m processes, where $n >> m$.

Exercise 12.6

Extend Exercise 4.3 to perform the conjugate gradient algorithm in parallel, where $\mathbf{A}$, $\mathbf{x}$ and $\mathbf{b}$ are distributed evenly across the processes. Use a matrix/vector size of 2^n and run on 2^m processes, where $n > m$.

Chapter 13

Shared-memory Parallel Programming with OpenMP

The approach to parallelism discussed in Chapter 12 allowed us to split a computational problem into small sub-problems and execute them concurrently in a distributed manner. However, to do this requires either incorporating parallelism in the initial code design, or substantial refactoring of an existing serial code. OpenMP provides us with an approach to exploit multi-core machines and achieve parallelism by quickly specifying some *code transformation* pragmas in an existing serial code.

Definition: Definition: OpenMP

OpenMP (Open Multi-Processing) is an Application Program Interface (API) which can be used to add multi-threaded shared memory parallelism to applications.

OpenMP is only suitable for shared-memory parallelism, so it does not support parallelism across nodes connected by a network. Uses *threads* to achieve parallelism within a *single process*. In contrast to the library of routines provided by MPI, OpenMP is (mostly) used through compiler directives to specify concurrency in code. There are still a few library routines to allow our code to find out information about the scope of parallelism being used and so we should `#include <omp.h>`. Environmental variables are used to control aspects of OpenMP, such as the number of threads to use.

13.1 Compiling OpenMP programs

OpenMP is provided through the compiler itself. Programs which use OpenMP *directives* can be compiled with the addition of a flag when **both compiling and linking**. For the GNU C++ compiler, this flag is `-fopenmp`; other compilers sometimes use alternative flags. It instructs the pre-processor to interpret OpenMP directives and the linker to link to the OpenMP library routines (which means, we do not need to specify the OpenMP library manually). Example 13.1 shows how to compile an OpenMP program.

Example 13.1: Compiling with OpenMP

```
# Interpret directives
g++ -fopenmp -c -o my_omp_prog.o my_omp_prog.cpp
# Link OpenMP library
g++ -fopenmp -o my_omp_prog my_omp_prog.o
```

13.2 OpenMP Directives

An OpenMP directive has the following general syntax (in C/C++):

```
#pragma omp [directive-name] [clause, ...]
{
    [C++ code]
}
```

The `#pragma omp` line is a preprocessor directive. It must appear on its own line and is case-sensitive. The directive only applies to the subsequent C++ statement or block enclosed in curly braces. Sometimes directives can have a number of clauses and so long directives can be continued onto multiple lines using the backslash to *escape* the new line, for example,

```
#pragma omp [directive-name] [clause1] [clause2] \
                    [clause3] [clause4]
{
}
```

13.3 Parallel Regions

All of the programs we have written so far have a single thread of execution within each process. In the case of MPI, we had multiple processes, but still only a single thread of execution within each one.

A parallel region is a block of code that will be executed by multiple threads within the same process. This is the fundamental OpenMP parallel construct and is specified as

```
#pragma omp parallel [clause ...]
{ [C++ code] }
```

It creates a team of threads and the original thread of execution becomes the *master* thread. Master forms part of the team and has thread ID 0. There is an implied barrier at end of parallel region, ensuring all the threads synchronise at the end of the parallel region.

The number of threads in a team can be prescribed by one of

- `NUM_THREADS (int)` clause.
- Calling the omp_num_threads() function.
- Setting the `OMP_NUM_THREADS` env. variable.

The default number of threads is equal to the number of CPU cores.

13.3.1 Data scope attributes

OpenMP is a shared-memory model. This means that variables which are declared before a parallel region are shared by default within parallel regions. However, this can be controlled using data scope attributes. Data scope is specified by a clause to the `#pragma omp` directive.

The main data scope attributes are:

- `default (private | firstprivate | shared | none)`: This defines the default scope for all variables within the extent of a region.
- `private (list)`: A separate version of each variable in *list* is created for each thread. You must assume theses have not been initialised.

- `firstprivate (list)`: This is the same as `private` but thread-specific variables are initialised with the value of the original variable from before the parallel region.
- `shared (list)`: The variables in *list* are assumed to be shared between all threads.
- `reduction (op : list)`: A private copy of variables in *list* is created for each thread. At the end of the parallel region, a reduction operation *op* is performed on the thread-specific variables and the original variable is set to the result.

Example 13.2 illustrates the use of a parallel region. The data scope attributes specify that each thread will have its own instance of the `threadid` variable (but `nthreads` will remain shared). We can use a conditional statement, with a conditional expression dependent on `threadid`, to have only a subset of threads execute code.

Example 13.2: Parallel regions

```
#include <iostream>
using namespace std;

#include <omp.h>

int main(int argc, char *argv[]) {

    int nthreads, threadid;

    // Fork a team of threads with each thread having a private threadid
    // variable
    #pragma omp parallel private(threadid)
    {
        // Obtain and print thread id
        threadid = omp_get_thread_num();
        cout << "Hello World from thread = " << threadid << endl;

        // Only master thread does this
        if (threadid == 0)
        {
            nthreads = omp_get_num_threads();
            cout << "Number of threads = " << nthreads << endl;
        }
    }  // All threads join master thread and terminate
}
```

13.4 Work-sharing Constructs

Definition: Work-sharing Construct

A work-sharing construct divides the execution of work among members of a team of threads. These constructs do not themselves launch threads.

Work constructs are only used within parallel regions. They specify, for example, which statements are executed by which threads, or how iterative structures are distributed across threads.

The main work-sharing constructs are:

- `do / for`
  This distributes the iterations of a loop across the threads (data parallelism)

- `sections`
  Breaks work into separate sections, each executed by a thread (functional parallelism)
- `single`
  Serialises a section of code - only performed by one thread.
- `tasks`
  Queues units of work to be executed in parallel, potentially with interdependencies between tasks.

There is no implied barrier upon entry of a work-sharing construct. However, there is an implied barrier on exit. We can also combine the `parallel` and work-sharing constructs.

13.4.1 DO / FOR

The `do` and `for` work-sharing constructs can split the iterations amongst the available threads using one of a number of *schedules*. This is specified using a `schedule (TYPE [,CHUNK])` attribute. The `TYPE` argument may be one of:

- `static`: The work is split into fixed-sized chunks and assigned statically to threads at the beginning of the work-sharing construct. If a `CHUNK` size is not specified, work is approximately evenly distributed. If one thread finishes its work before the others it will sit idle.
- `dynamic`: The work is split into fixed-sized chunks and assigned dynamically to threads as they become free. Default `CHUNK` size is 1.
- `guided`: Similar to dynamic, but size of each chunk reduces as work is assigned with the aim of evenly distributing the total work. Here `CHUNK` defines minimum size.
- `runtime`: Defer to an environmental variable. In this case specifying the `CHUNK` argument is invalid.
- `auto`: Defer to operating system to decide.

The `static` schedule is good for loops in which all iterations have approximately the same amount of work as it minimises overheads. If the amount of work for each iteration might vary, using `dynamic` or `guided` scheduling may be more performant.

An example of using a loop work-sharing construct on a `for`-loop is shown in Example 13.3. We have combined the parallel and work-sharing construct for simplicity. The 100 iterations of the loop will be split up into chunks of size 10 and assigned statically. The loop iterator is private and we have indicated that we will perform a reduction on `result`.

13.4.2 Sections

This is a non-iterative work-sharing construct. Section(s) of code are divided amongst the team of threads.

The `sections` directive has the form:

```
#pragma omp sections [clause ...]
{
    #pragma omp section
    { [C++ code] }
    #pragma omp section
    { [C++ code] }
    ...
}
```

Example 13.3: Computing a dot-product in parallel using OpenMP.

```
#include <omp.h>
#include <iostream>
using namespace std;

int main(int argc, char *argv[])  {
    const int n      = 100;             // Vector size
    const int chunk  = 10;              // Chunk processed per thread
    int       i      = 0;               // Loop index
    double    result = 0.0;             // Store result
    double a[n], b[n];                  // Arrays

    for (i = 0; i < n; i++) {           // Initialise vectors
        a[i] = i * 1.0;
        b[i] = i * 2.0;
    }

    // Parallel region, 'for' work sharing, only 'i' is private
    // Split work by chunk size and sum local 'result' variables
    #pragma omp parallel for        \
        default(shared) private(i)  \
        schedule(static,chunk)      \
        reduction(+:result)
    for (i = 0; i < n; i++)
        result = result + (a[i] * b[i]);

    cout << "Final result= " << result << endl;
}
```

We can use `private`, `firstprivate`, `lastprivate` and `reduction` clauses to define the scope of variables within sections. An additional argument, `nowait`, is available which removes the implied barrier at the end of the `sections` work-sharing construct. The assignment of sections to threads is implementation specific.

Example 13.4 shows the use of the sections directive to compute two independent blocks of code concurrently. In this example, we have separated the `parallel` directive and the work-sharing construct. In this case, each `section` consists of only one statement; for sections containing multiple statement, remember to use curly braces.

13.4.3 Tasks

Tasks are units of work designed to be executed in parallel. They are relatively new in OpenMP and were only introduced in the OpenMP 3.0 standard. A `task` work-sharing construct is specified using:

```
#pragma omp task [clause ...]
```

Like work-sharing constructs, they must be in `parallel` region. When a thread encounters a `task` directive, a task (which is just a unit of work) is created, queued, and executed whenever a thread is available. Normally, only one thread in a team should generate the tasks. To do this, we might combine the task generation with a `single` directive, as shown in Example 13.5.

Example 13.4: Computing two independent operations in parallel using sections.

```
#include <omp.h>
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    const int N = 1000;
    int i;
    float a[N], b[N], c[N], d[N];

    for (i=0; i < N; i++) {
        a[i] = i * 1.5;
        b[i] = i + 20.0;
    }

    #pragma omp parallel shared(a,b,c,d) private(i)
    {
        #pragma omp sections nowait
        {
            #pragma omp section
            for (i=0; i < N; i++)       // Compute c = a + b
                c[i] = a[i] + b[i];
            #pragma omp section
            for (i=0; i < N; i++)       // Compute c = a * b
                d[i] = a[i] * b[i];
        }  /* end of sections */
    }  /* end of parallel region */
}
```

Example 13.5: Task generation within a `single` work-sharing construct.

```
#pragma omp parallel  // Create a team of threads
{
    #pragma omp single    // Only master thread generates tasks
    {
        #pragma omp task      // One unit of work
        { [C++ code] }
        #pragma omp task      // Second unit of work
        { [C++ code] }
    }
}
```

13.4.4 Tasks vs. Sections

At first glance, in simple examples, tasks look a bit like sections, as shown in the code excerpts below:

```
#pragma omp sections
{
#pragma omp section
func1();
#pragma omp section
func2();
}
```

```
#pragma omp single
{
#pragma omp task
func1();
#pragma omp task
func2();
}
```

Tasks are started at a *scheduling point*. In the above example, this is the implicit barrier at the end of

the `single` directive. We can introduce an explicit scheduling point / synchronisation with the `taskwait` directive:

```
#pragma omp taskwait
```

This is essentially a barrier for tasks. We can also remove the implicit barrier with additional `nowait` clause to `single`:

```
#pragma omp single nowait
```

Tasks in general provide us with greater flexibility in how the computation in our program is distributed in parallel.

13.4.5 Task dependencies

Dependencies between tasks can be introduced to ensure they are performed in the correct order. We do this using the `depend` directive:

```
#pragma omp task depend([in | out | inout]:<variable>) ...
```

A task for which a variable is marked as `out` must complete execution before a subsequent task with the same variable marked as `in`. Dependencies on arrays can be specific to only some elements so that different tasks may work on different portions of an array. As a result of adding task dependencies, the order the tasks are created becomes important.

An example of applying dependencies between tasks is given in Example 13.6.

Example 13.6: Tasks with dependencies.

```
#pragma omp task depend(out:a)
a = ComputePi(1);
#pragma omp task depend(out:b)
b = ComputePi(10);
#pragma omp task depend(out:c)
c = ComputePi(1000);
#pragma omp task depend(in:a,b,c)
Print(a, b, c);
```

13.5 Synchronisation

Synchronisation directives enforce the relative execution of threads.

- `master`: the block of code is executed only by the master thread (no implicit barrier at end, unlike `omp single` directive).
- `critical`: the block of code can only be executed by one thread at a time.
- `barrier`: a thread will wait for all other threads at this point.
- `atomic`: Ensures a statement is executed by only one thread at a time (a mini-`critical` region).

An example of using the `critical` directive to ensure only one thread at a time executes the statement which updates `x` is shown in Example 13.7.

Example 13.7: Using `critical` to enforce atomic operations.

```
int x = 0;
#pragma omp parallel shared(x)
{
#pragma omp critical
x = x + 1;
}
```

13.6 OpenMP directives

As we have seen, there are a number of OpenMP directives, each of which can accept a range of clauses to modify their behaviour. A summary of this is shown in the table below:

Clause	Directive					
	PARALLEL	DO/for	SECTIONS	SINGLE	PARALLEL DO/for	PARALLEL SECTIONS
IF	●				●	●
PRIVATE	●	●	●	●	●	●
SHARED	●	●			●	●
DEFAULT	●				●	●
FIRSTPRIVATE	●	●	●	●	●	●
LASTPRIVATE		●	●		●	●
REDUCTION	●	●	●		●	●
COPYIN	●				●	●
COPYPRIVATE				●		
SCHEDULE		●			●	
ORDERED		●			●	
NOWAIT		●	●	●		

For further details on OpenMP, there is a good tutorial at:
https://computing.llnl.gov/tutorials/openMP/

Exercises

Exercise 13.1

Recall from Exercise 12.4 that π can be approximated by the integral:

$$\frac{\pi}{4} = \int_0^1 \frac{1}{1+x^2}\,\mathrm{d}x$$

Compute this using OpenMP parallelisation instead performing an appropriate reduction at the end.

Exercise 13.2

Copy your solution from Exercise 4.2 and modify it to perform matrix-matrix multiplication in parallel using OpenMP.

Hint: Take care to avoid race conditions, where more than one thread tries to write to the same memory location. One way to ensure this is to split the rows of the output matrix amongst the threads.

Exercise 13.3

Copy your solution to Exercise 4.6 and parallelise with OpenMP where appropriate.

Measure the time taken to perform the time-integration step using the Boost Timer library: `http://www.boost.org/doc/libs/1_74_0/libs/timer/doc/cpu_timers.html`

Compare running your code in serial and with different numbers of threads to examine how efficiently your code scales. Also consider different numbers of grid-points (adjusting time-step to maintain stability).

Exercise 13.4

(Advanced) Use OpenMP to accelerate your solution to Exercise 5.3 and plot the speedup against number of threads.

Chapter 14

Parallel Programming on GPUs with CUDA (Optional)



Note

For the code and exercises in this chapter an NVIDIA GPU is required. Please use `typhoon.ae.ic.ac.uk`, which is equipped with two NVIDIA A40 GPUs.
You will also need to load the CUDA toolkit. To do this, run

```
module load cuda
```

from a terminal. It is recommended to compile and run your code using the terminal as you will need to use the `nvcc` compiler.

14.1 GPUs vs CPUs

Graphics Processing Units (GPUs) were originally designed for accelerating graphical rendering tasks for 3D visualisation and games. Such operations involved performing large number of identical matrix-vector operations on many data elements in order to perform geometric computations such as translations, rotations, and identifying intersections of graphical objects as they move in a scene. As a consequence, GPUs were designed to have a large number of processing cores to allow these operations to happen in parallel and give a smooth rendering with high frame rates. Since the operations were quite simple, the cores themselves were not particularly sophisticated, lacking many of the features (e.g. double-precision arithmetic, branch prediction, prefetching and cache management) essential on CPUs and comparatively slower.

CPUs and GPUs are therefore designed with different objectives in mind. A CPU is designed to execute a few sequential *threads* as fast as possible; it has a range of sophisticated flow control and caching features to allow this. A GPU, on the other hand, is designed to execute hundreds or thousands of threads in parallel, even though each thread executes significantly slower than a CPU. The total number of operations that a GPU can perform per second is considerably higher than a CPU and they also use less energy per operation. This makes them interesting for high-performance computing.

Definition: CUDA

CUDA stands for the *Compute Unified Device Architecture* and is a programming model and set of C language extensions, developed by NVIDIA for use with their GPUs.

Around 2006, NVIDIA introduced CUDA as a means to allow programmers to use GPUs for general

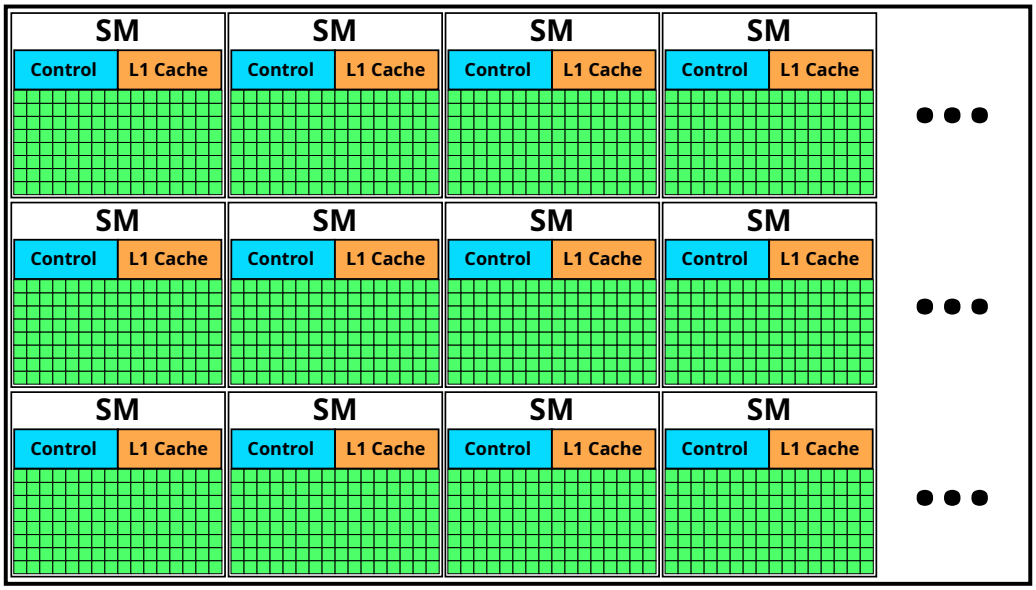


Figure 14.1: Layout of a GPU with multiple SMs grouping processing cores. In this example (part of an NVIDIA A40), each streaming multi-processor has 128 cores, an L1 cache and some control logic. The A40 has a total of 84 SMs.

purpose applications. It is a parallel computing platform and programming model which leverages the compute engine available in NVIDIA GPUs. In particular, it provided the opportunity for non-graphics software, such as scientific and engineering packages, to utilise the large number of floating-point operations available on the GPU to do computationally intensive tasks (such as linear algebra).

14.2 CUDA architecture and terminology

A GPU is an additional device that is added to a computer. It communicates with the host system (CPU and memory) over a fast Peripheral Component Interconnect Express (PCIe) bus. GPUs are composed of a large number of processor *cores*, which are the smallest processing units of the device. The cores are grouped into *streaming multi-processors* (SMs).

Definition: Definition: Streaming Multi-processor (SM)

A streaming multi-processor (SM) is a grouping of processing cores within a GPU. Each SM manages the parallel execution of off-loaded code on those cores.

The number of SMs on a GPU, and the number of cores per SM, will vary by model of GPU. For example, an NVIDIA Tesla P100 GPU has 56 SMs, each containing 64 CUDA cores, for a total of 3,584 cores. A newer generation A40 GPU (as available in `typhoon.ae.ic.ac.uk`) has 84 SMs, each containing 128 CUDA cores, for a total of 10,752 cores. This is illustrated diagramatically in Figure 14.1.

Definition: Definition: Threads and warps

A *thread* is a sequence of instructions executed on a single core. A group of 32 threads are called a *warp*.

Definition: Definition: Thread block

A thread block is a group of threads executed by a single SM. A thread block can contain a maximum of 1024 threads.

SMs execute *thread blocks*. They manage the creation, scheduling and execution of the *threads* in groups of 32, known as *warps*. Unlike, for example, OpenMP, where individual threads can follow different execution paths through the program, all threads in a warp must necessarily follow the same instructions. This means if you have data-dependent branching occurring within the program, all possible branches are executed for all 32 threads, with those for which the branch does not apply "masked out". Greatest efficiency is achieved when all threads within a warp agree on the sequence of instructions to be executed.

Definition: Definition: Kernel

A *kernel* describes a sequence of instructions which are executed by threads on the GPU.

These kernels can then be called on multiple blocks of threads, as described later in this chapter.

A GPU has a total of five types of memory space which threads can use, each with differing levels of access and performance characteristics. Each thread has its own registers and private local memory to which only it can access. Each thread block has a dedicated region of shared memory which can be accessed by all threads in the block. Since this is on the GPU processor itself, this is fast to access. There is then a single global memory space on each device, to which all threads can access. However, this is slower to access than the shared memory.

In addition, there are two special types of memory: *constant* and *texture*. However, these will not be discussed further.

14.3 Compiling CUDA code

Any code which uses the CUDA language extensions must be compiled by the CUDA compiler. To denote code which needs to be compiled by the CUDA compiler, we use the `.cu` extension for source files, and the `.cuh` extension for header files.

The CUDA C++ compiler is called `nvcc`, so to compile CUDA C++ code, we would invoke:

```
nvcc mycode.cu -o mycode
```

14.4 Writing CUDA code

For an algorithm to use a GPU efficiently it must be highly parallelisable. This is because algorithms which need instructions to be executed sequentially or with limited parallelism will not be able to effectively utilise the GPU. When we write a program using CUDA, some code will execute on the host machine (the CPU), while some will be *off-loaded* to the GPU. Only code inside of *kernels* executes on the GPU.

Before a kernel is executed, the data needed by the kernel must be transferred from the host memory to the GPU memory, if it does not already reside on the GPU. The cost of doing this can be non-negligible and repeatedly transferring data to and from the GPU can strongly compromise the performance of a code. Achieving performance on a GPU therefore requires us to minimise the number of data transfers, keep data in GPU memory where feasible, and perform as much of the algorithm as possible on the GPU.

14.5 Kernels

A kernel is a unit of work, potentially a specific operation or step in an algorithm, for example adding vectors or multiplying two matrices. It takes the form of a C++ function prefixed by the `__global__` CUDA specifier. For example, one could declare a kernel to add two vectors together as:

```
__global__ void VectorAdd(const int N, float* a, float* b, float* c) {
        for (int i = 0; i < N; ++i) {
                c[i] = a[i] + b[i];
        }
}
```

The kernel will be executed by each thread in one or more thread blocks, depending on the parameters given when it is called.

14.5.1 Calling a kernel

A kernel is called in a similar way to a normal function, passing in parameters to specify the data it should operate on. However, additional information is required to specify how many threads the kernel should be executed on. This information is given in a special CUDA language extension, called the *execution configuration*. For example:

```
VectorAdd<<<1, 256>>>(N, a, b, c);
```

The first argument in the `<<<...>>>` notation is the number of thread blocks to use, while the second argument is the number of threads in each block. So the above code will launch the `VectorAdd` kernel on 256 threads.

Remember that threads are dispatched by the SM in warps (groups of 32), so the number of threads in a block should be a multiple of 32 to use the GPU efficiently. The maximum size of a block is usually 1024 threads. The number of blocks is limited by system resources. An SM may be able to support more than 1024 threads, so multiple blocks may be distributed to each of the available SMs. Multiple blocks can also be executed concurrently by different SMs.

This is illustrated in Figure 14.2. In this example, each SM has 128 cores. Since a warp contains 32 threads, this means each SM will execute an instruction from four warps concurrently. Each SM can support up to 1,536 threads. This allows the SM to schedule other threads to execute if the current threads are blocked waiting for data from memory. Since each block is 512 threads, each SM can support three blocks at time. Thread blocks are assigned to SMs as capacity becomes available.

14.5.2 Thread and block dimensions

For some applications data may naturally be organised in a multi-dimensional way. For example, a matrix may be more naturally processed as a two-dimensional grid of two-dimensional blocks of threads. To support this, CUDA provides the `dim3` tuple for specifying the number of threads per block, and the size of the grid of blocks. The data type provides three member variables for x-, y- and z-dimension sizes.

```
// Operate on NxN matrices using a 2D grid of 2D thread blocks.
int N = 1024;
dim3 numThreads(16,16); // 256 threads per block
dim3 numBlocks(N / numThreads.x, N / numThreads.y);
MatrixAdd<<<numBlocks, numThreads>>>(A, B, C);
```

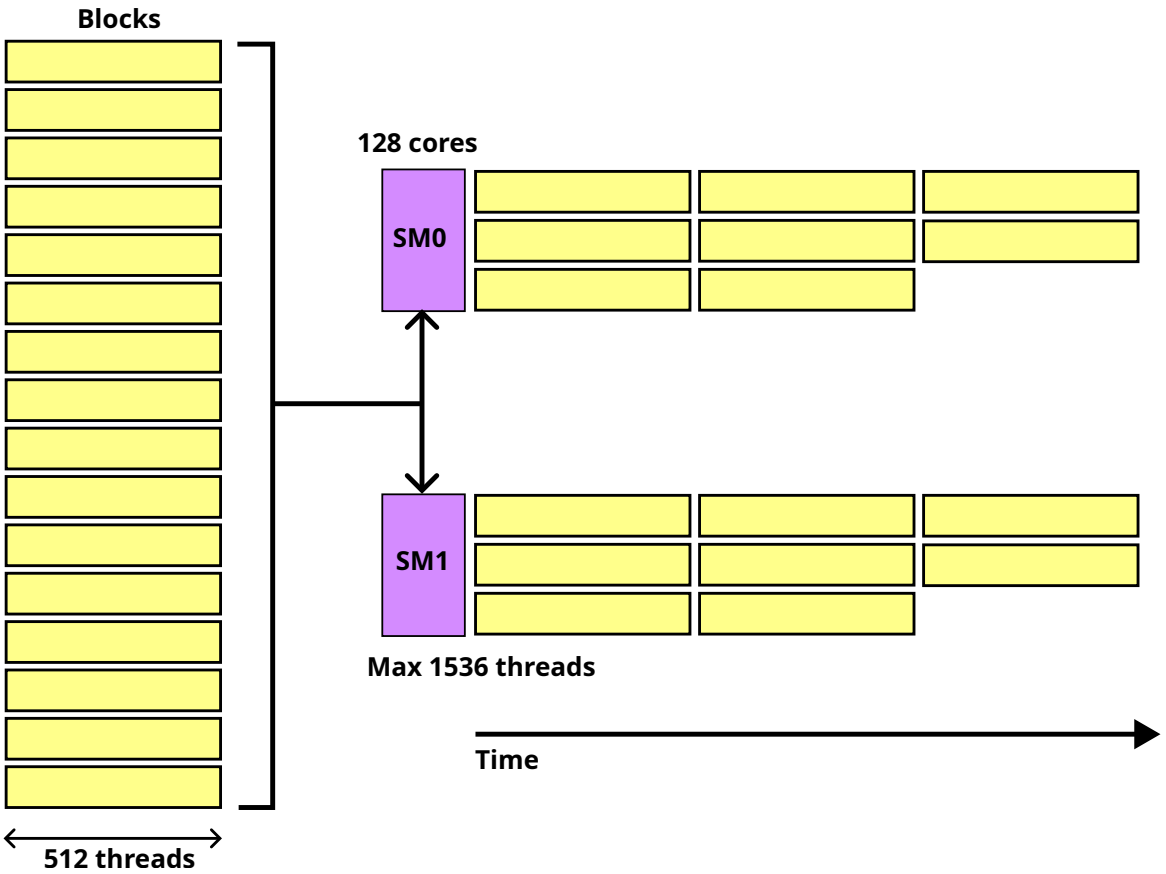


Figure 14.2: Distribution of thread blocks to SMs of an NVIDIA A40 GPU. Each SM has 128 cores, but allows up to 1536 to reside on the SM.

14.5.3 Thread and block indexing

As written above, each thread executing the `VectorAdd` kernel will perform the addition on the entire vector. This will clearly not provide any advantage with multiple threads and will introduce race conditions as multiple threads try to write to the same locations in memory. In order to allow each thread to operate on different data, CUDA provides keywords we can use within the kernel to find the thread index and the block size (much like the thread index in OpenMP).

- `threadIdx`: The index of a thread within a block;
- `blockIdx`: The index of a block within the grid;
- `blockDim`: The size of the block.

To accommodate the multi-dimensional indexing discussed above, the values are accessed through x, y and z components. For example, `threadIdx.x` will access the thread index component in the x-direction of the block. If blocks and grids are one-dimensional, the values should still be accessed using the `x` component.

14.5.4 Memory allocation on the GPU

For kernels to operate on data, that data must reside in the GPU memory. CUDA provides functions to explicitly allocate memory on the GPU, copy data between the host and the GPU, as well as freeing memory on the GPU when it is not needed. Example 14.1 shows this.

We first allocate an array on the host and populate it with suitable values. To allocate memory on the GPU, we first create a pointer to hold the memory address on the GPU. We then use

```
cudaMalloc(void** ptr, size_t n)
```

to allocate it. Note that the first parameter is a `void**` (a pointer to a pointer), so we need to use the *address-of* operator to pass the memory location of the pointer. The second parameter is the size in **bytes**, so this should be calculated appropriately, e.g. `N*sizeof(float)`.

To transfer the data in the host array to the gpu array we use

```
cudaMemcpy(void* dest, void* src, size_t n, cudaMemcpyKind kind)
```

The first three parameters are self-explanatory. The last parameter specifies the type of transfer to perform. This is usually one of `cudaMemcpyHostToDevice` or `cudaMemcpyDeviceToHost`.

Once we are finished with the memory, we should deallocate. For GPU memory allocations, this is achieved using

```
cudaFree(void* ptr)
```

Example 14.1: Memory allocation, use and deallocation on the GPU.

```
int main() {
        const int n = 1024;
        float* host_array = new float[n];

        // ...populate the host_array here...

        float* gpu_array;
        cudaMalloc(&gpu_array, n*sizeof(float));
        cudaMemcpy(gpu_array, host_array, n, cudaMemcpyHostToDevice);

        // ... call CUDA kernel to use the data ...

        cudaFree(gpu_array);

        return 0;
}
```

Unified Memory

Recent GPUs and CUDA implementations provide a *Unified memory model*, where both host CPUs and GPUs can access data in a single memory address space. Behind the scenes, CUDA or the hardware (depending on GPU model) migrates data between devices as necessary. While convenient, this provides less control over the movement of data so care must still be taken to avoid unnecessary movement.

To use unified memory we allocate the memory using:

```
cudaMallocManaged(void** ptr, size_t n)
```

We can then access the values in the array from either the host or the device, without needing to explicitly use `cudaMemcpy`. The managed memory is freed using `cudaFree` as before.

14.5.5 Synchronisation

When calculations are off-loaded to the GPU, the host must wait to ensure that the kernel has completed execution before attempting to access the computed results. To do this, we call

```
cudaDeviceSynchronize();
```

between the kernel calls and the host access to the result. This will block execution until the kernels have finished executing.

14.5.6 A complete example

Example 14.2 shows two simple kernels which populates, and then adds, two vectors together on a GPU.

Example 14.2: Example CUDA program which initialises and sums two vectors using a GPU. The arrays are allocated using CUDA-managed memory.

```
#include <iostream>
using namespace std;

__global__
void VectorInit(const int N, float* a, const float x) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        a[i] = x;
    }
}

__global__
void VectorAdd(const int N, float* a, float* b, float* c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    constexpr int n = 1024; // max 1024 threads
    float *a, *b, *c;

    cudaMallocManaged(&a, n*sizeof(float));
    cudaMallocManaged(&b, n*sizeof(float));
    cudaMallocManaged(&c, n*sizeof(float));

    VectorInit<<<1, n>>>(n, a, 1.1);
    VectorInit<<<1, n>>>(n, b, 2.2);
    VectorAdd <<<1, n>>>(n, a, b, c);
    cudaDeviceSynchronize();

    for (int i = 0; i < n; ++i) {
        float diff = fabs(c[i] - 3.3f);
        if (diff > 1e-06) {
            cout << "Index " << i << ", difference " << diff << endl;
        }
    }

    cudaFree(a);
    cudaFree(b);
    cudaFree(c);

    return 0;
}
```

Exercises

Exercise 14.1

Extend Example 14.2 to execute vector addition for vectors of arbitrary length N, and arbitrary number of threads M, such that $N > M > 1024$.

Exercise 14.2

Write a CUDA program to perform matrix-matrix multiplication on square matrices of rank N. Use a two-dimensional grid of two-dimensional blocks.

Exercise 14.3

Extend Exercise 4.3 and implement the conjugate gradient algorithm using CUDA on a system of size N.

1. Generate the $N \times N$ matrix $\mathbf{M}$ on the CPU.
2. Write a CUDA kernel to compute the product $\mathbf{M}^\top \mathbf{M}$.
3. Write CUDA kernels to perform the dot-product, daxpy and matrix-vector multiplication operations.
4. Implement the conjugate gradient algorithm, using the developed kernels.

Try to maximise the number of threads used when calling the CUDA kernels and minimise the amount of data transfers between the host and device. Think about how the threads will map onto the available SMs/cores for the GPU you are using.

Chapter 15

ScaLAPACK (Optional)

The BLAS and LAPACK libraries provide a range of routines for performing common linear algebra operations in serial. Explicit operations, such as matrix-vector or matrix-matrix multiplication are relatively easy to construct in parallel using a combination of serial BLAS operations on each process and MPI operations. However, implicit operations, such as matrix-vector solves are more complex to implement in parallel on distributed architectures.

The **ScaLAPACK** library provides parallel MPI implementations of many BLAS/LAPACK routines for use in a distributed computing environment. Besides BLAS and LAPACK, ScaLAPACK builds on two other libraries. PBLAS is a library of parallel implementations of (some) BLAS routines. BLACS is a library of basic linear algebra communication subroutines. Parallel linear algebra also introduces additional complexities to the storage formats used to represent vectors and matrices in parallel.

Full documentation for ScaLAPACK is available from
http://netlib.org/scalapack/

15.1 Linking to ScaLAPACK

Linking to ScaLAPACK requires linking a number of libraries to the executable. As well as the main ScaLAPACK library, this also includes two BLACS libraries and the BLAS/LAPACK libraries we have already seen. We might compile a program which uses ScaLAPACK as follows.

```
mpicxx myprog.cpp -lscalapack-openmpi -lblacs-openmpi \
                    -lblacsCinit-openmpi -llapack -lblas
```

15.2 BLACS communication

BLACS organises MPI processes as a Cartesian grid and provides support for row and column communication within that grid. It also provides *contexts* (which is a similar concept to MPI communicators) in which to perform linear algebra operations.

15.2.1 Initialising BLACS

To initialise BLACS we can use the code shown in Example 15.1 (note BLACS has a C interface where all routines are prefixed with Cblacs_).

The `Cblacs_pinfo()` routine retrieves the rank and size of the communicator, while `Cblacs_get()` creates a new context. We then initialise a grid within this context which has 1 row and `npe` columns.

Example 15.1: Initialising CBLACS for parallel linear algebra

```
int mype, npe, ctx, nrow, ncol, myrow, mycol;
// Initialises the BLACS world communicator (calls MPI_Init if needed)
Cblacs_pinfo(&mype, &npe);
// Get the default system context (i.e. MPI_COMM_WORLD)
Cblacs_get( 0, 0, &ctx );
// Initialise a process grid of 1 rows and npe columns
Cblacs_gridinit( &ctx, &order, 1, npe );
// Get info about the grid to verify it is set up
Cblacs_gridinfo( ctx, &nrow, &ncol, &myrow, &mycol);
```

Finally, we can retrieve information about the newly created grid using the `Cblacs_gridinfo()` routine. For each process, this gets the row and column of that process in the grid, as well as the number of rows and columns.

15.2.2 Finalising BLACS

Much like MPI requires us to finalise our use of MPI operations before the program terminates, we should also call a routine to terminate our grid. This frees the context handle we created with `Cblacs_get()`.

```
Cblacs_gridexit( mContext );
```

Optionally, if we also do not need to perform any further MPI communication, we can call

```
Cblacs_exit( 0 );
```

This also calls `MPI_Finalize()` internally.

15.3 Parallel matrix storage

15.3.1 Block-cyclic distribution

ScaLAPACK distributes a matrix amongst processes using block-cyclic data distribution to balance work across the processor cores. Block-cyclic distribution allows for the use of efficient matrix-matrix operations to still be used locally.

Figure 15.1 shows the how data is distributed in block-cyclic distribution for one-dimensional and two-dimensional data. The global matrix is divided into blocks. The first row of blocks are assigned to the first row of processes and, if there are more blocks than processes across the grid, the blocks are assigned in a round-robin manner. Subsequent rows of blocks are assigned to subsequent rows of processes in a similar manner. If there are more block rows than process rows, the same round-robin approach is applied.

15.3.2 Array descriptors

Due to the extra complexity introduced by storing a matrix in a distributed manner, ScaLAPACK captures the data layout for vectors and dense matrices in an 9-integer descriptor array. The elements of this array, which we will call `desc` are as follows:

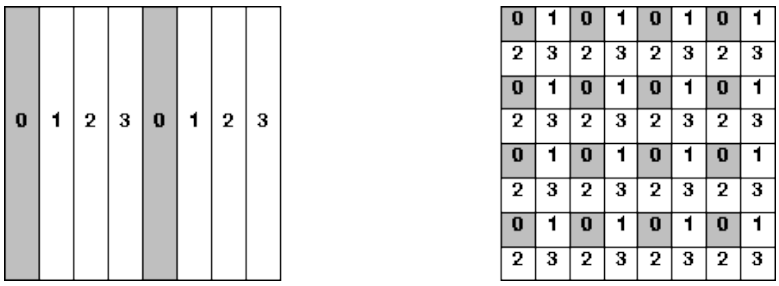


Figure 15.1: Assignment of process IDs to blocks of a matrix for one-dimensional and two-dimensional block-cyclic distributions.

`desc[0]`	Descriptor type: `1` = dense
`desc[1]`	BLACS context handle.
`desc[2]`	Number of Rows in the global matrix.
`desc[3]`	Number of Columns in the global matrix.
`desc[4]`	Row blocking factor.
`desc[5]`	Column blocking factor.
`desc[6]`	Process row over which first row is distributed.
`desc[7]`	Process column over which first column is distributed.
`desc[8]`	Leading dimension of local matrix.

15.3.3 Parallel Banded Storage

ScaLAPACK also generalises the same banded storage format used by BLAS and LAPACK. This is now distributed across the processes. However, for banded matrices, the BLACS grid layout should be set to either $1 \times P$ or $P \times 1$. A banded matrix, split across three processes, using a block size of 3 would therefore be distributed as

$$\left(\begin{array}{ccc|ccc|c} x & x & x & x & x & x & x \\ x & x & x & x & x & x & x \\ x & x & x & x & x & x & x \\ * & * & a13 & a24 & a35 & a46 & a57 \\ * & a12 & a23 & a34 & a45 & a56 & a67 \\ a11 & a22 & a33 & a44 & a55 & a66 & a77 \\ a21 & a32 & a43 & a54 & a65 & a76 & * \end{array}\right) \tag{15.3.1}$$

As for BLAS/LAPACK, we have denoted the padding with asterisks. The x's denote the additional *fill* storage required for the partial pivoting during LU factorisation. Note this is more than is required for LAPACK.

15.3.4 Banded matrix descriptors

For banded matrices, the assumptions in the structure of the matrix make the descriptor (7-integer) slightly simpler. We always use a one-dimensional block-cyclic distribution for banded solves. There are now two types of descriptor: one for the banded matrix, and one for the RHS that will be used in the banded solve:

- `501` = banded matrix (1-by-P process grid)
- `502` = banded RHS (P-by-1 process grid)

The content of the descriptor is as follows:

```
int descb[7];
descb[0] = 501;          // Type
descb[1] = ctx;          // Context
descb[2] = N;            // Problem size
descb[3] = NB;           // Blocking of matrix
descb[4] = 0;            // Process row/column
descb[5] = 1+2*BWL+2*BWU; // Local leading dim
descb[6] = 0;            // Reserved
```

To conclude, we give an outline example of how to perform a banded matrix-vector solve using ScaLAPACK in Example 15.2. In this example, we assume a matrix with the structure shown in Equation (15.3.1). We suppose our matrix has an upper and lower bandwidth of 2 and 1, respectively, with the total problem size of 7. We choose a blocking-size of 3 when distributing the columns of the matrix amongst the available processes. In this case, we also assume we are only solving for a single RHS vector and so set `RHS=1`.

The offsets in the matrix and RHS vector are given as Fortran indices and therefore start at 1 (as opposed to 0 as in C/C++). We then specify the size of the matrix on each process, computed as the total number of diagonals, plus the extra padding, multiplied by the block size. Finally, we have to allocate some workspace for ScaLAPACK. The complicated formula is lifted straight from the ScaLAPACK documentation and is the *minimum* storage we must provide for the routine to work.

After allocating the necessary storage arrays for the matrix, pivoting values, RHS and workspace, we set up the descriptors (as described earlier). We would then populate our matrix and RHS vector. The last task before performing the solve is to configure the CBLACS grid.

In this example we call `pdgbtrs` which is the parallel version of the LAPACK `dgbtrs`, and performs both the factorisation and solve step. If we were to perform the solve repeatedly using the same matrix but different RHS vectors, we might instead consider using `pdgbtrf` to first factorise the matrix, followed by `pdgbtrs`

The final tasks are, of course, to verify the routine completed successfully by testing the value of `info`, finalise CBLACS and deallocate the memory.

Example 15.2: Banded matrix solve using ScaLAPACK.

```
int info;                // Status value
const int N    = 7;      // Total problem size
const int NB   = 3;      // Blocking size (number of columns per process)
const int BWL  = 1;      // Lower bandwidth
const int BWU  = 2;      // Upper bandwidth
const int NRHS = 1;      // Number of RHS to solve
const int JA   = 1;      // Start offset in matrix (to use just a submatrix)
const int IB   = 1;      // Start offset in RHS vector (to use just a subvector)
const int LA   = (1 + 2*BWL + 2*BWU)*NB
const int LW   = (NB+BWU)*(BWL+BWU)+6*(BWL+BWU)*(BWL+2*BWU)
                  + max(NRHS*(NB+2*BWL+4*BWU), 1)   // ScaLAPACK documentation

double* A    = new double[LA];   // Matrix banded storage
int*    ipiv = new int   [NB];   // Pivoting array
double* x    = new double[NB];   // In: RHS vector, Out: Solution
double* work = new double[LW];   // Workspace

int desca[7];           // Descriptor for banded matrix
desca[0] = 501;         // Type
desca[1] = ctx;         // Context
desca[2] = N;           // Problem size
desca[3] = NB;          // Blocking of matrix
desca[4] = 0;           // Process row/column
desca[5] = 1+2*BWL+2*BWU; // Local leading dim
desca[6] = 0;           // Reserved

int descb[7];           // Descriptor for RHS
descb[0] = 502;         // Type
descb[1] = ctx;         // Context
descb[2] = N;           // Problem size
descb[3] = NB;          // Blocking of matrix
descb[4] = 0;           // Process row/column
descb[5] = NB;          // Local leading dim
descb[6] = 0;           // Reserved

// ... Populate A and x here
// ... Set up CBLACS grid

// Perform the parallel solve.
F77NAME(pdgbsv) (N, BWL, BWU, NRHS, A, JA, desca, ipiv, &x[0],
                 IB, descb, work, LW, &info);

// Verify it completed successfully.
if (info) {
    cout << "Error occurred in PDGBTRS: " << info << endl;
}

// ... Finalize CBLACS, clean up memory
```

Exercises

Exercise 15.1

Extend your solution to Exercise 5.1 solve the Helmholtz matrix system directly using MPI parallelism and routine(s) from ScaLAPACK.

Part V

Supplementary and Advanced Material

Appendix A

Unix/Linux Operating System

A.1 Overview

Unix and Linux are both operating systems. They are the software that controls the hardware of a computer and manages the software applications run by one or more users. Unix is an commercial operating system developed in the 1970s by AT&T. It introduced the concept of a hierarchical filesystem that we are familiar with today, but also the idea of having many small programs which can be connected together to perform more complex tasks. Later versions of Unix supported multiple users and applications running concurrently through time-sharing of the CPU.

Linux is a Unix-like operating system built on the same principles as Unix. Unlike Unix, it is *open-source* meaning it is free to download and use. The core of the Linux operating system is the Linux kernel, initially released in 1991 by Linus Torvalds (hence the name). Since then many flavours – or *distributions* – of Linux have been developed through contributions from thousands of programmers around the world. The Linux environment is highly customisable, efficient, robust and secure, and is used for a wide array of purposes. For example, over 80% of the world's websites run on servers which use Linux, as do most of the fastest supercomputers. In the past decade, it has gained increasing popularity as a desktop operating system, particularly through the Ubuntu distribution.

The computing systems provided for this course run the Debian Linux operating system, on which Ubuntu is based.

A.2 Desktop environments

Modern Linux distributions support desktop environments. These enable applications to provide an intuitive graphical interface for users. Most Linux graphical environments (and indeed Microsoft Windows and Mac OSX environments) provide common features:

- A *desktop* with icons for quick access to common applications or files;
- A *taskbar* showing the running applications;
- An *applications menu* for launching applications.

Figure A.1 highlights these features on the Linux environment used in this course.

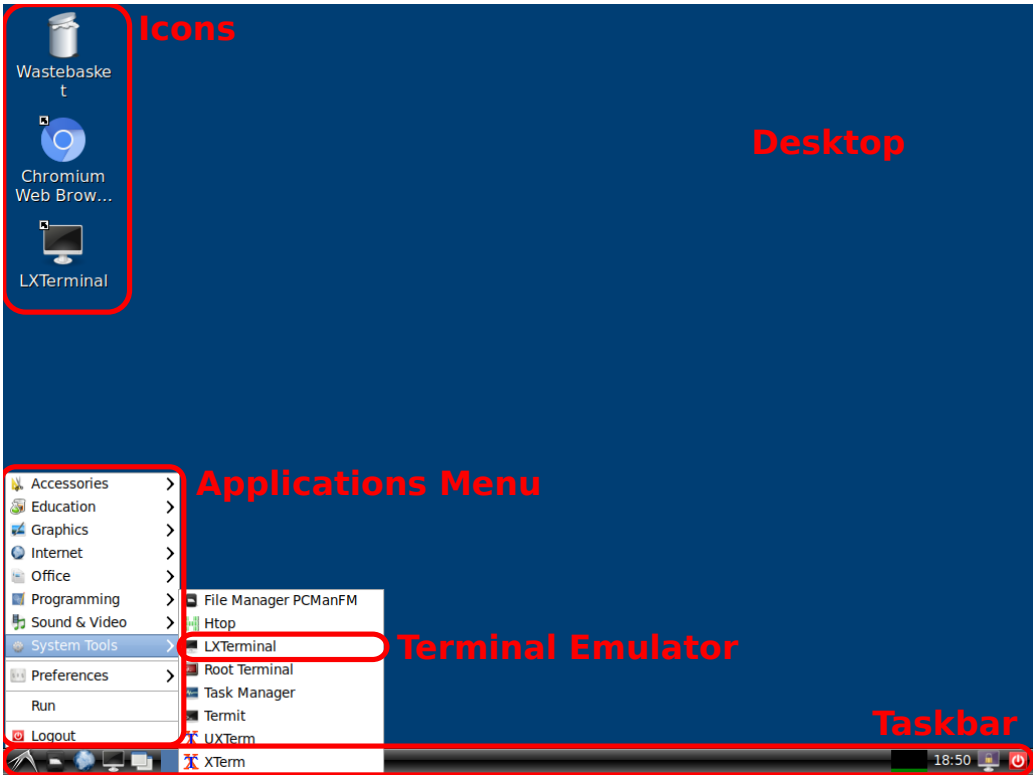


Figure A.1: Key components of a graphical environment.

A.3 Command-line

A command-line interface enables instructions to be given to the operating system through textual commands. These commands may include accessing or modifying files, executing applications and controlling hardware devices. While initially quite daunting to the novice user, with experience the command-line interface can be more powerful and efficient than the graphical environment. We will briefly introduce some of the basic operating system commands in this section.

A.3.1 The Shell

A command-line interface is provided through a *Terminal emulator*. An example is LXTerminal (Figure A.2), which can be launched from the applications menu, as shown in Figure A.1. Initially the terminal shows a *prompt* and awaits the user to type a command. This is in fact produced by a *command-interpreter*, or *shell*, which is a program which will process any input given at the prompt, performing the requested operations or running another program as needed. The format of prompt can vary (and be customised), but the default shown here has the form `user@host:path$`

- `user` - the id of the user under which commands will run. The permissions given to the user will affect which commands can be run and on which files.
- `host` - the name of the computer
- `path` - the current *working directory*

Every program running will have a working directory which is the default place it will look to for reading and writing files. The tilde (~) symbol is a short-hand notation for the user's home directory.

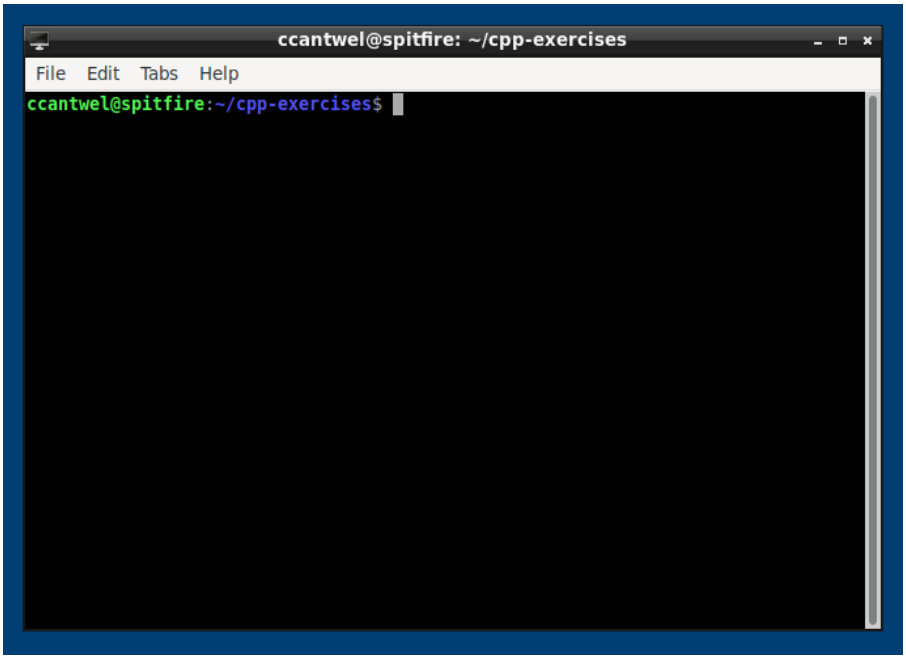


Figure A.2: LXTerminal application



Note
We use the $ symbol to indicate the shell prompt going forward.

A.3.2 Commands and Arguments

The first word entered at a prompt is taken to be the command to be run. This may be an executable program, or a built-in command of the command-interpreter. In order to customise the behaviour of a command, the user can specify zero or more arguments (separated by spaces) after the command. For example, the following is the command to move a file, where the source and destination filenames are the arguments given:

```
mv myfile.txt ~/mydirectory
```

The shell would replace the ~ with the user's home directory location and then run the `mv` command with the two arguments given. In the case of `mv` requires two or more arguments or it will return an error.

A.3.3 Getting help – man-pages

There are many commands, some of which have many possible arguments. To find out about a particular command, there is the `man` program, which displays the manual for a given command. For example,

```
man ls
```

shows a description of the `ls` command and all the possible arguments to customise the command's behaviour. These are known as man-pages.

A.3.4 Filesystems

There is only a single filesystem hierarchy on a Linux operating system (known as the root filesystem, or `/`). Other filesystems (e.g. disks or network drives) can be attached to a specific location in the hierarchy so we can access them. For example user home directories are on a network filesystem (i.e. the data actually is stored on a different computer on the network) but are attached at the child folder `/home`. Levels in the hierarchy are separated by the `/` symbol. So the files for a user `abc123` would be found in the hierarchy at `/home/abc123`.

A.3.5 Filesystem commands

These commands can be used to browse and manage files in the filesystem hierarchy using the shell.

- `ls`
  Displays a list of the files in the current working directory.
- `cp <source> <dest>`
  Copies the file or directory `<source>` to `<dest>`.
- `rm <file>`
  Deletes a file.
- `pwd`
  Prints the current working directory.
- `cd <dir>`
  Changes the current working directory.
- `mkdir <dir>`
  Creates a directory.
- `du`
  Calculate the size of a file or directory.
- `cat <file>`
  Prints (con`cat`enates) a text file to the terminal.
- `less <file>`
  Display a text file in the terminal, allowing scrolling.

You can find out more details on how to use these commands using the man-pages. Example A.1 illustrates basic usage of some of these commands. The first thing to note is that the hash symbol (#) is used to indicate the start of a comment. As well as interactive usage, the shell can also run sequences of commands from a file and comments can be used to help explain the purposes of commands.

The command `ls -l` displays a *long* listing of files, which includes various details about file attributes:

- the first 'abc123' refers to the user which owns the file.
- the second 'abc123' refers to the group which owns the file (in this case it has the same name as the user and is the user's default group).
- `-rw-r--r--` indicates file permissions. If the first character is a 'd', then this is a directory. The remaining characters are read in groups of three, each indicating whether the file can be read (r), written (w) or executed (x) by the associated user, group or anyone else. In Example A.1, the file is readable and writeable by the user 'abc123' but only readable by anyone else.

```
$ cd ~          # Change to the user's home directory (~ is optional)
$ pwd           # Show the current working directory
/home/abc123
$ ls            # List files
countries.csv
$ ls -l         # List files with details
-rw-r--r-- 1 abc123 abc123   782 Nov 15 10:39 countries.csv
$ mkdir data    # Make a directory
$ mv countries.csv data/
$ cd data
$ pwd
/home/abc123
$ ls -l
-rw-r--r-- 1 abc123 abc123   782 Nov 15 10:39 countries.csv
```

Example A.1: Example usage of some filesystem commands. The dollar ($) symbol represents the shell prompt. Text to the right of the hash (#) symbol is a comment and is not processed by the shell.

- the 782 is the number of bytes in the file.
- Nov 15 10:39 was when the file was last modified.

A.3.6 Text manipulation

Commands that can be used to manipulate text include:

- `echo`
  Repeat a string to the terminal.
- `printf`
  Format strings of characters and numbers.
- `head`
  Display the first N lines from a file.
- `tail`
  Display the last N lines from a file.
- `grep`
  Only display lines matching a given string.
- `cut`
  Select fields or columns from lines of text.
- `paste`
  Concatenate lines from multiple files side-by-side.
- `sed`
  Find and replace sub-strings within a file.
- `sort`
  Sorts lines in a file.
- `wc`
  Counts characters, words (with `-w`) or lines (with `-l`) in a file.

```
$ echo "This is a string"
This is a string
$ head -n 5 countries.csv
Country,Area(km^2)
Albania,28748
Andorra,468
Armenia,29743
Austria,83858
$ grep "Luxembourg" countries.csv
Luxembourg,2586
$ cut -f 1 -d, countries.csv
...
$ sort -k 2 -t, -n -r countries.csv
...
$ sed -e 's/,/: /' countries.csv
```

Example A.2: Illustrative usage of text manipulation commands. The ... indicates output is not shown. The `cut` command extracts the first column of values, assuming a comma as the delimiter. The `sort` command shown orders the rows of the file by land area from largest to smallest. The `sed` command replaces commas with a colon and a space.

Example A.2 shows how these commands can be used to manipulate a comma-separated text file (`countries.csv`, available from Blackboard Learn). The sort example shown orders the rows of the file by the land area column from largest to smallest. The `sed` command is used here with the '-e' (expression) argument to find and replace sequences of characters. The forward-slash separate the components: 's' indicates search and replace; ',' is the string to find; ': ' is the string to replace each instance of the search string.



Tip

Use the man-page to find out about the arguments of the `cut` and `sort` commands.

A.3.7 Text editors

Text editors are interactive tools for editing text files (such as source code) within a shell. Some of the more popular choices include:

- `nano`
- `vim`
- `emacs`

The `nano` editor is relatively simple to use, while `vim` and `emacs` are more powerful (but more complex to learn to use).

A.3.8 Pipes and redirection

By default programs executed from the terminal receive their input from the keyboard and their output is sent to the terminal to be displayed. These are known as *standard input* and *standard output*, respectively. There is a third stream known as *standard error*, which is where programs can write error messages. This is usually also connected to the terminal to be displayed.

These can of course be changed. We can use *redirection operators* to change from where a program receives input, and to where it sends its output. For example,

```
$ echo "C++ is awesome and redirection is cool"
C++ is awesome and redirection is cool
$ echo "C++ is awesome and redirection is cool" > message.txt
$ cat message.txt
C++ is awesome and redirection is cool
```

The first line prints the message to the terminal, since this is the default place to send output. The second command redirects the output to a file instead (therefore nothing is printed to the terminal).

Redirection becomes really powerful when we redirect the output from one command to the input of another. For example:

```
$ cat message.txt | xargs -n 1 | sort | uniq | grep s
```

This reads the text from the file `message.txt` and passes it to the `xargs` program, which splits the text into separate words, after which they are sorted and only one of each word is retained (note 'is' appears only once). Finally, only those words containing an 's' are printed. The output looks like:

```
awesome
is
```

We can even combine this with file redirection:

```
$ xargs -n 1 < message.txt | sort | uniq | grep s > words.txt
```

The first command reads from `message.txt` and the last command writes out to `words.txt`.

Example A.3 gives a less-contrived example and pipes together three commands to extract and format the land area of Luxembourg from the text file `countries.csv`. The `xargs` command runs the command which follows it for each word of input, appending the word as an argument to the command.

```
1 $ grep "Luxembourg" < countries.csv | cut -f 2 -d, | xargs printf "%'.f km^2"
2 2,586 km^2
```

Example A.3: Extracting and formatting the area of Luxembourg from a data file.

Exercise A.1

Download the file `FCDO_Geographical_Names_Index.csv` from Blackboard Learn. This contains the UK Government list of country letter codes, names, official names and citizen names. The first line contains the column headings and columns are separated by commas.

Use shell commands and redirection, as necessary, to:

1. List the country names beginning with a 'G'.
2. Sort the file by citizen name.
3. Find the number of countries in the list.
4. Print the country code for 'Croatia' to the terminal.
5. How many official country names begin with 'The'?
6. How many words are in the official name of 'Sri Lanka'?
7. How many non-whitespace characters are in the official name of 'Sri Lanka'?

Appendix B

List of Common File Types

Extension	Type	Description
.cpp	C++ Source file	Contains implementations of functions, classes, etc. Exactly one such file in the project must contain the `main()` routine.
.h	Header file	Contains function declarations, class definitions, etc. Used to define prototypes which are then included in one or more C++ source files.
.hpp	Header file	These files are often used to distinguish header files containing templated code from those containing non-templated code.
.so	Shared library	Libraries contain collections of compiled code which can be reused by other programs. There are often one or more header files associated with a library, which declares the available functions. Shared libraries use dynamic runtime linking, and the object code is not included in the executable.
.a	Static library	Like a Shared Library, but the functions and classes used by a program are copied into the programs executable. A static library is therefore only required during compilation.
.o	Object code	These files are the result of compiling a source code file to machine code.
.s	Assembly code	This is the result of halting the compiler part-way through the compilation process before running the Assembler.